UNIVERSITÀ POLITECNICA DELLE MARCHE
FACOLTÀ DI INGEGNERIA
Dipartimento di Ingegneria dell'Informazione
Corso di Laurea Magistrale in Ingegneria Informatica e dell'Automazione



TESI DI LAUREA

Progettazione e implementazione di un framework multi-agente per la gestione di un chatbot basato su LLM a supporto degli operatori di una compagnia assicurativa

Design and implementation of a multi-agent framework for managing a LLM based chatbot to support operators of an insurance company

Relatore

Prof. Domenico Ursino

Candidato

Denil Nicolosi

ANNO ACCADEMICO 2024-2025

Scientists study the world as it is,
engineers create the world that never has been.

Theodore von Karman

Sommario

La crescente adozione dei Large Language Model sta trasformando il modo in cui le aziende automatizzano i processi operativi, aprendo nuove opportunità per lo sviluppo di assistenti virtuali intelligenti. In questa tesi è stata progettata, implementata e valutata una Proof of Concept denominata Bluey, un sistema multi-agente basato su LLM per il supporto degli operatori di back-office nel settore assicurativo. La soluzione, sviluppata presso l'azienda Blue Reply, si basa su un'architettura Supervisor che coordina quattro agenti specializzati per l'interrogazione del database, la consultazione della documentazione, la gestione del calendario e l'invio di email. L'implementazione adotta il framework LangGraph, integrando il meccanismo Human-in-the-Loop per il controllo delle operazioni critiche e la piattaforma LangSmith per il monitoraggio e l'analisi delle conversazioni. La valutazione con il paradigma LLM-as-a-judge ha validato l'ipotesi che un'architettura multi-agente ben progettata possa compensare le limitazioni di modelli meno performanti attraverso la specializzazione dei compiti, garantendo flessibilità di distribuzione anche in presenza di vincoli infrastrutturali e requisiti di riservatezza.

Keyword: Large Language Model, sistema multi-agente, LangGraph, LangChain, Human-in-the-Loop, LLM-as-a-judge, chatbot, prompt engineering

Indice

Introduzione **1**

1 Introduzione ai chatbot **4**
1.1 Cos'è un chatbot . . . 4
1.2 Storia dei chatbot . . . 6
1.3 Tipologie di chatbot . . . 8
1.4 Vantaggi e svantaggi dei chatbot . . . 10
1.5 Applicazioni dei chatbot . . . 11
1.5.1 Domotica e IoT . . . 12
1.5.2 Ambito bancario e finanziario . . . 13
1.5.3 Telecomunicazioni . . . 14
1.5.4 E-commerce . . . 16
1.5.5 Assistenza sanitaria . . . 17

2 Introduzione ai Large Language Model **18**
2.1 Cos'è un Large Language Model . . . 18
2.1.1 Architettura di un Large Language Model . . . 18
2.1.2 Fase di addestramento . . . 23
2.1.3 Fase di Fine-Tuning . . . 24
2.1.4 Prompt engineering . . . 25
2.2 Storia dei Large Language Model . . . 26
2.3 Tipologie di Large Language Model . . . 26
2.4 Vantaggi e svantaggi dei Large Language Model . . . 29
2.5 Applicazioni dei Large Language Model . . . 31
2.5.1 Retrieval-Augmented Generation . . . 32
2.5.2 Architettura di un agente LLM . . . 34
2.5.3 Architettura multi-agente . . . 36

3 Definizione del contesto di riferimento e specifica dei requisiti **38**
3.1 Contesto aziendale . . . 38
3.1.1 Il gruppo Reply . . . 38
3.1.2 L'azienda Blue Reply . . . 40
3.2 Definizione dei requisiti del progetto . . . 40
3.2.1 Obiettivi funzionali del progetto . . . 40
3.2.2 Obiettivi tecnici del progetto . . . 41

4 Progettazione degli agenti **42**
4.1 Premessa . . . 42
4.2 Tecnologie utilizzate . . . 42
4.2.1 Il framework LangChain . . . 43
4.2.2 L'orchestrazione tramite LangGraph . . . 44
4.2.3 La piattaforma LangSmith . . . 47
4.3 Architettura del sistema . . . 48
4.3.1 Meccanismo di comunicazione tra agenti . . . 51
4.3.2 Agente Supervisor . . . 52
4.3.3 Agente SQL . . . 52
4.3.4 Agente email . . . 53
4.3.5 Agente appointment . . . 53
4.3.6 Agente documentation . . . 55
4.3.7 Flusso di elaborazione delle richieste . . . 56
4.3.8 Considerazioni sulla sicurezza . . . 57

5 Implementazione degli agenti **60**
5.1 Struttura del progetto . . . 60
5.2 Gestione dello stato del grafo . . . 61
5.3 Implementazione degli agenti specializzati . . . 61
5.3.1 Agente Supervisor . . . 62
5.3.2 Agente SQL . . . 64
5.3.3 Agente email . . . 68
5.3.4 Agente appointment . . . 70
5.3.5 Agente documentation . . . 72

6 Analisi del funzionamento degli agenti **75**
6.1 Agent Chat . . . 75
6.1.1 Architettura e funzionamento . . . 75
6.1.2 Funzionalità principali . . . 76
6.2 Analisi di esempi di conversazione . . . 77
6.2.1 Esempio di richiesta della ricerca della procedura del portale dei clienti 77
6.2.2 Esempio di verifica dello stato delle polizze e reportistica . . . 80
6.2.3 Esempio di richiesta della riunione con il cliente per il numero della polizza . . . 82

7 Valutazione e benchmark **86**
7.1 Premessa . . . 86
7.2 Metodologia di valutazione . . . 87
7.2.1 Definizione delle metriche di valutazione . . . 88
7.3 Creazione del dataset di test . . . 91
7.3.1 Generazione delle tracce di conversazione . . . 92
7.4 Esecuzione delle valutazioni . . . 93
7.5 Analisi dei risultati . . . 94
7.5.1 Confronto dei risultati . . . 95
7.5.2 Visualizzazione dei risultati in LangSmith . . . 95
7.5.3 Discussione . . . 96

8 Conclusioni e sviluppi futuri **99**

Bibliografia **101**

Sitografia **103**

Ringraziamenti **104**

Elenco delle figure

1.1 L'elaborazione del linguaggio naturale e dei suoi sottogruppi 5
1.2 Trend di ricerca relativo ai chatbot dal 01/01/2009 al 18/07/2024 5
1.3 Interfaccia grafica del chatbot ELIZA 6
1.4 Cronologia degli eventi nella storia dei chatbot 8
1.5 Grafico sulla crescita di ChatGPT in confronto ad altre piattaforme popolari . 8
1.6 Grafico sulla crescita del mercato dei chatbot nel periodo 2023-2028 11
1.7 Grafico sull'utilizzo dei chatbot nei diversi settori 12
1.8 Schema sul funzionamento di Google Assistant 13
1.9 Interfaccia ed esempio di chat del chatbot di Unicredit 14
1.10 Interfaccia ed esempio di chat con l'assistente TOBi di Vodafone tramite WhatsApp 15
1.11 Interfaccia ed esempio di chat con Zalando fashion assistant 16

2.1 Gerarchia dei Large Language Model 19
2.2 Struttura del funzionamento di un Large Language Model 20
2.3 Schema concettuale dell'architettura di un modello GPT 21
2.4 Schema concettuale dell'architettura Scaled Dot-Product Attention (a sinistra) e del meccanismo di attenzione Multi-Head (a destra) 22
2.5 Ordine cronologico dei rilasci dei modelli di LLM 27
2.6 Schema di funzionamento di un modello di Retrieval-Augmented Generation 34
2.7 Meccanismo di interazione di un agente basato su Large Language Model con l'ambiente circostante 35
2.8 Struttura di un agente basato su Large Language Model 36
2.9 Struttura gerarchica di un sistema multi-agente basato su Large Language Model 37

3.1 Logo del gruppo Reply 39
3.2 I principali clienti del gruppo Reply 39

4.1 Bilanciamento tra controllo del flusso e autonomia decisionale 43
4.2 Architettura di un agente ReAct in LangChain 45
4.3 Compromesso tra il controllo del flusso e autonomia decisionale in LangGraph 45
4.4 Esempi di pattern di flussi in LangGraph 46
4.5 Interfaccia utente di LangSmith 48
4.6 Architettura del modello multi-agente Supervisor 49
4.7 Architettura complessiva del sistema Bluey 51

4.8 Architettura di alto livello dell'agente Supervisor 52
4.9 Architettura di alto livello dell'agente SQL 54
4.10 Architettura di alto livello dell'agente email 55
4.11 Architettura di alto livello dell'agente appointment 56
4.12 Architettura di alto livello dell'agente documentation 57
4.13 Flusso di elaborazione sequenziale delle richieste tra gli agenti 58

5.1 Grafo di stato dell'implementazione dell'agente SQL 66

6.1 Interfaccia di Agent Chat durante un'interazione con il sistema Bluey 76
6.2 Flusso temporale della conversazione in esame 79
6.3 Flusso di lavoro per l'agente SQL 80
6.4 Richiesta di approvazione per esecuzione query SQL 81
6.5 Risposta finale da parte del Supervisor con il file Excel 82
6.6 Invocazione dell'agente SQL tramite tool mostrata in chat 83
6.7 Invocazione dell'agente appointment tramite tool mostrata in chat 84
6.8 Invocazione dell'agente email tramite tool mostrata in chat 84
6.9 Visualizzazione delle email ricevute dal cliente 85

7.1 Dashboard di LangSmith con le metriche di valutazione aggregate 96
7.2 Listino prezzi di alcuni modelli OpenAI GPT-5 97
7.3 Confronto tra le prestazioni e il numero di parametri di alcuni modelli sul benchmark SWE-bench Verified 98

Elenco delle tabelle

2.1 Lista dei principali modelli di Large Language Model in ordine ascendente di rilascio 27

7.1 Distribuzione degli esempi nel dataset di test 92
7.2 Risultati del test con modelli economici (GPT-5 nano/mini e Qwen3-coder) . 95
7.3 Risultati del test con modelli performanti (GPT-5 e GPT-5 Codex) 95
7.4 Confronto di latenze e costi tra le configurazioni di modelli 97

Introduzione

Negli ultimi anni i Large Language Model (LLM) hanno rivoluzionato il panorama dell'Intelligenza Artificiale, aprendo nuove frontiere nell'interazione uomo-macchina e nell'automazione dei processi aziendali. Dalla fine del 2022, con il rilascio di ChatGPT al pubblico, si è assistito a una crescita esponenziale dell'interesse verso queste tecnologie che hanno dimostrato notevoli capacità nella comprensione e nella generazione di testo in linguaggio naturale. Il chatbot di OpenAI ha raggiunto oltre 100 milioni di utenti in soli due mesi, stimolando investimenti massicci nel settore e accelerando lo sviluppo di nuovi modelli sempre più sofisticati. Questa evoluzione tecnologica sta progressivamente trasformando il modo in cui le aziende concepiscono l'automazione dei propri processi operativi, offrendo opportunità concrete per lo sviluppo di assistenti virtuali capaci di intraprendere azioni in autonomia e affiancare gli operatori umani nelle loro attività quotidiane.

Il settore dei chatbot, che affonda le proprie radici negli anni '60 con progetti pionieristici come ELIZA, ha subito una trasformazione radicale grazie all'avvento degli LLM. I chatbot tradizionali, basati su regole statiche e flussi di conversazione predefiniti, presentavano limiti significativi nella gestione di richieste complesse o non previste. I modelli linguistici di grandi dimensioni, invece, grazie alla loro capacità di comprendere il contesto e di generare risposte coerenti, consentono di superare questi vincoli e di offrire un'esperienza conversazionale più naturale e flessibile. Tuttavia, l'utilizzo di un modello LLM per gestire compiti articolati che richiedono l'accesso a sistemi esterni, la manipolazione di dati e l'esecuzione di azioni concrete presenta ancora sfide rilevanti. Il modello potrebbe, infatti, commettere errori di ragionamento, inventare informazioni non supportate dai dati o non integrarsi perfettamente con l'esigenza di controllo dei processi.

Queste difficoltà derivano, in parte, dalla natura stessa delle reti neurali su cui i Large Language Model si fondano. Gli sviluppatori che intendono integrare tali modelli in applicazioni aziendali si trovano, infatti, a dover governare sistemi che si comportano come *black-box*: i meccanismi interni attraverso cui il modello elabora le informazioni e produce le risposte non sono direttamente osservabili né interpretabili. Questa mancanza di trasparenza rende difficile comprendere le cause di determinati comportamenti, riprodurli in modo affidabile e applicare strategie mirate per prevenire con certezza risposte errate o indesiderate.

Un approccio che consente di mitigare, almeno in parte, le problematiche sopra descritte è rappresentato dai sistemi multi-agente. In queste architetture, più agenti specializzati collaborano tra loro per risolvere problemi complessi, ciascuno dotato di competenze specifiche e strumenti dedicati per il proprio ambito di competenza. Il vantaggio principale di questo approccio risiede nella sua natura modulare: ogni agente costituisce un'unità autonoma, intercambiabile e testabile singolarmente. Questa struttura rende il processo decisionale

più trasparente, poiché consente di ispezionare i comportamenti dei singoli componenti, individuare le cause di eventuali malfunzionamenti e definire strategie di intervento mirate. Tali benefici risultano ulteriormente amplificati quando questi sistemi vengono combinati con meccanismi di controllo del flusso di lavoro più strutturati, che consentono di governare con precisione le transizioni tra i diversi stati del sistema e le interazioni tra gli agenti. Queste architetture rappresentano, dunque, un'evoluzione significativa rispetto all'utilizzo di un singolo modello, offrendo maggiore controllo, affidabilità e possibilità di ottimizzazione.

In questo contesto si colloca il presente lavoro di tesi, che nasce dall'esigenza concreta di supportare il personale di back-office nel settore assicurativo attraverso un assistente virtuale. Durante il periodo di lavoro presso l'azienda Blue Reply, una società di consulenza informatica specializzata nel settore finanziario e assicurativo, ci siamo confrontati con le sfide operative quotidiane degli addetti alla gestione delle polizze. Questi ultimi devono frequentemente interrogare database aziendali per recuperare informazioni su clienti, polizze e sinistri, consultare documentazione tecnica relativa alle procedure del portale, gestire appuntamenti con i clienti e inviare comunicazioni via email. Tali attività, seppur consuete, richiedono tempo e attenzione in contesti ripetitivi, sottraendo risorse a compiti più complessi e strategici.

Il progetto si pone l'obiettivo di dimostrare, attraverso una Proof of Concept, le potenzialità di un'architettura multi-agente applicata a questo dominio operativo. In questa tesi progettiamo, implementiamo e valutiamo un sistema multi-agente basato su LLM destinato al supporto degli operatori assicurativi. La soluzione adotta un'architettura di tipo Supervisor, nella quale un agente centrale coordina più agenti specializzati, ciascuno progettato per gestire un dominio specifico di competenza. Questa struttura consente di bilanciare l'autonomia decisionale del modello con un controllo del flusso più rigoroso, partizionando la complessità del dominio in componenti autonomi e testabili.

L'implementazione si basa sul framework LangGraph, che fornisce le primitive per la gestione del flusso di esecuzione e dello stato conversazionale. Ogni agente viene progettato seguendo la metodologia CO-STAR per la costruzione dei prompt, garantendo comportamenti coerenti e prevedibili. Il meccanismo Human-in-the-Loop viene integrato per garantire l'approvazione esplicita dell'utente prima dell'esecuzione di operazioni critiche, quali l'invio di email, la creazione di appuntamenti e l'esecuzione di query sul database. La piattaforma LangSmith viene utilizzata per il monitoraggio e l'analisi delle conversazioni in tempo reale.

Per la valutazione delle prestazioni adottiamo il paradigma LLM-as-a-judge, nel quale un modello linguistico viene impiegato come giudice automatico per valutare la qualità delle risposte. Vengono definite cinque metriche specifiche per misurare il completamento delle richieste, l'assenza di allucinazioni, la gestione dell'ambiguità, la qualità dell'orchestrazione e la preservazione del contesto. Attraverso test condotti su un dataset di esempi rappresentativi, confrontiamo, infine, le prestazioni del sistema utilizzando configurazioni di modelli economici e modelli più performanti, analizzando il compromesso tra qualità delle risposte, costi operativi e tempi di latenza.

Questa tesi è strutturata come di seguito specificato:

- Nel Capitolo 1 verrà fornita una panoramica sui chatbot, analizzandone la storia, le tipologie, i vantaggi e le applicazioni nei diversi settori.
- Nel Capitolo 2 verranno introdotti i Large Language Model, descrivendone l'architettura, le tecniche di prompt engineering e le applicazioni avanzate, tra cui la RAG e le architetture multi-agente.
- Nel Capitolo 3 verrà descritto il contesto aziendale, presentando l'azienda Blue Reply e definendo i requisiti funzionali e tecnici del progetto.

- Nel Capitolo 4 verrà illustrata la progettazione del sistema multi-agente, descrivendo le tecnologie selezionate e le scelte architetturali adottate.
- Nel Capitolo 5 verrà descritta l'implementazione tecnica del sistema, analizzando la struttura del progetto, i prompt di sistema e i tool sviluppati.
- Nel Capitolo 6 verrà analizzato il funzionamento pratico del sistema, presentando alcuni esempi di conversazione e il meccanismo Human-in-the-Loop.
- Nel Capitolo 7 verrà descritta la metodologia di valutazione LLM-as-a-judge, le metriche definite e i risultati dei test condotti.
- Nel Capitolo 8 verranno tratte le conclusioni del lavoro svolto e delineati alcuni possibili sviluppi futuri.

CAPITOLO 1

Introduzione ai chatbot

In questo primo capitolo introduttivo verrà fornita una panoramica sui chatbot, definendo cosa sono e come funzionano grazie all'elaborazione del linguaggio naturale.

Si analizzerà la storia dei chatbot, partendo dai primi esempi sviluppati negli anni '60 fino ai chatbot moderni (come ChatGPT) basati su Intelligenza Artificiale. Si discuteranno le principali tipologie di chatbot in base ad alcune differenziazioni e le tecnologie utilizzate per la loro realizzazione. Si esamineranno i vantaggi e gli svantaggi dei chatbot e, quindi, i benefici che possono trarre le imprese con il loro utilizzo.

Infine si riporteranno alcune applicazioni e casi d'uso dei chatbot suddivisi per settore, analizzandone i principali punti di forza e quale impatto hanno avuto nel mercato.

1.1 Cos'è un chatbot

Un chatbot (contrazione di "chat" e "robot") è un software progettato per interagire con gli esseri umani utilizzando un linguaggio naturale simulando un comportamento umano. Questa tecnologia è utilizzata in vari settori, tra cui il servizio clienti, il marketing e l'assistenza sanitaria. I chatbot possono operare tramite diverse piattaforme, ad esempio siti web, applicazioni di messaggistica e dispositivi vocali intelligenti.

L'interfaccia utente è il punto di contatto tra l'utente e il chatbot. Può essere una finestra di chat, un'applicazione di messaggistica o un dispositivo vocale. La comprensione del linguaggio naturale è fondamentale per il funzionamento di un chatbot. Si utilizzano diversi metodi di interpretazione del linguaggio umano tra cui:

- *Natural Language Processing (NLP)*: tecnologia che consente ai computer di comprendere e interpretare il linguaggio umano. Essa comprende le seguenti fasi:
  - *analisi lessicale*: scomposizione di una espressione linguistica in token;
  - *analisi grammaticale*: analisi della struttura grammaticale di una frase;
  - *analisi sintattica*: ordinamento dei token in una struttura sintattica;
  - *analisi semantica*: interpretazione del significato di una frase a partire dalla struttura sintattica.
- *Natural Language Understanding (NLU)*: è una branca dell'NLP che si occupa di comprendere il significato delle parole e delle frasi.

- *Natural Language Generation (NLG)*: è una branca dell'NLP che si concentra sulla generazione di testo in linguaggio naturale, accedendo a database e altre fonti di informazioni per produrre risposte accurate e pertinenti.
- *Conversational AI Technology*: essa permette di utilizzare la memoria, le preferenze e la comprensione contestuale per creare conversazioni fluide e personalizzate.

Questi metodi sono relazionati tra loro come mostrato in Figura 1.1.

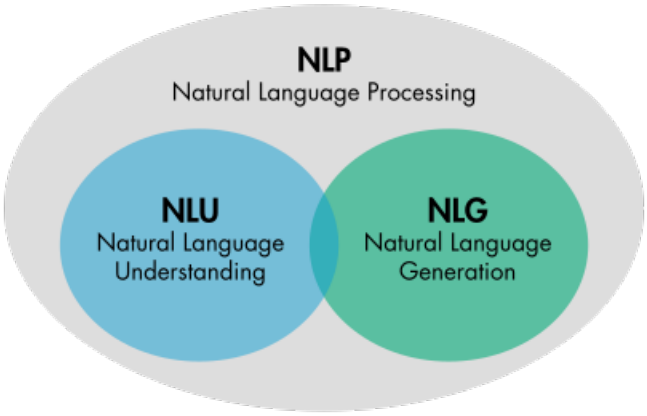


Figura 1.1: L'elaborazione del linguaggio naturale e dei suoi sottogruppi

Ad oggi il trend dei chatbot è in continua crescita. Dal sito Google Trends è possibile estrapolare i dati relativi all'argomento "Chat bot" negli ultimi anni. Come si può notare dal grafico in Figura 1.2, il numero di ricerche relative ai chatbot è aumentato costantemente nel corso degli anni, confermando l'interesse crescente per questa tecnologia.

Con il continuo progresso delle tecnologie di Intelligenza Artificiale, è probabile che i chatbot diventino sempre più sofisticati e capaci, offrendo nuove opportunità per le aziende di interagire con i loro clienti in modi innovativi e personalizzati.

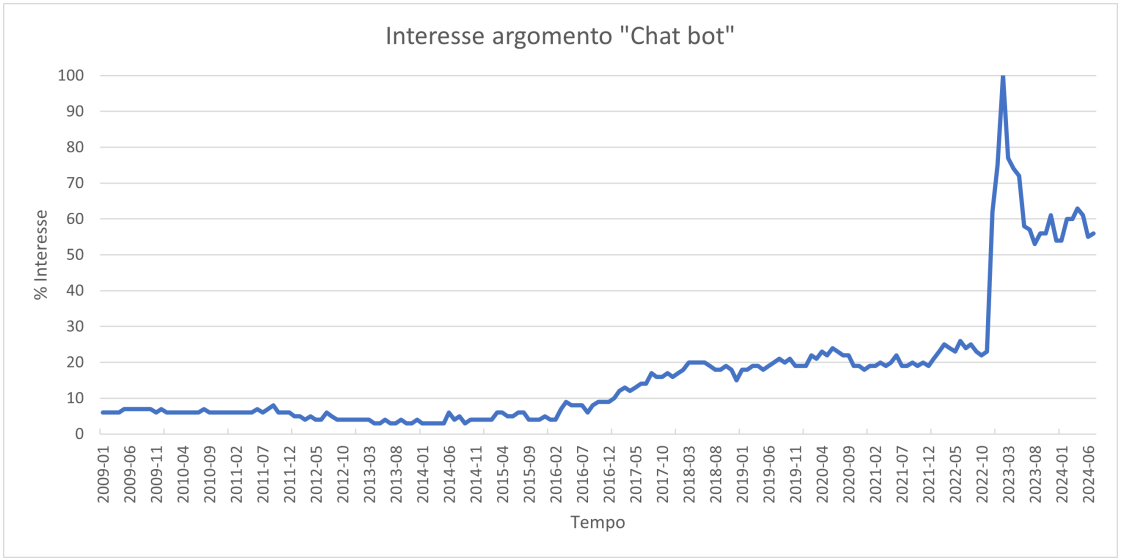


Figura 1.2: Trend di ricerca relativo ai chatbot dal 01/01/2009 al 18/07/2024

1.2 Storia dei chatbot

Nonostante il termine "chatbot" fu coniato solo nel 1994 da Micheal Mauldin, l'idea di creare macchine in grado di comunicare con gli esseri umani risale a molto tempo fa. Fin dal 1950, grazie al test di Turing, ci si è iniziati a porre la domanda "Può una macchina pensare?". Questo è descritto nel documento "Computing Machinery and Intelligence", dove fu proposto un test in cui un interrogatore, tramite una serie di domande, doveva stabilire se stesse conversando con un essere umano o una macchina. Il test di Turing viene eseguito regolarmente ancora oggi.

Nel 1966 iniziò il progetto ELIZA, un primo esempio di chatbot sviluppato da Joseph Weizenbaum. ELIZA era in grado di condurre una conversazione con gli utenti, utilizzando semplici regole di riconoscimento del linguaggio naturale. Questo chatbot simulava una conversazione con un terapeuta, rispondendo alle domande degli utenti con domande e riflessioni. L'interfaccia di ELIZA, riportato in Figura 1.3, era molto semplice e da linea di comando, in linea con i computer dell'epoca.

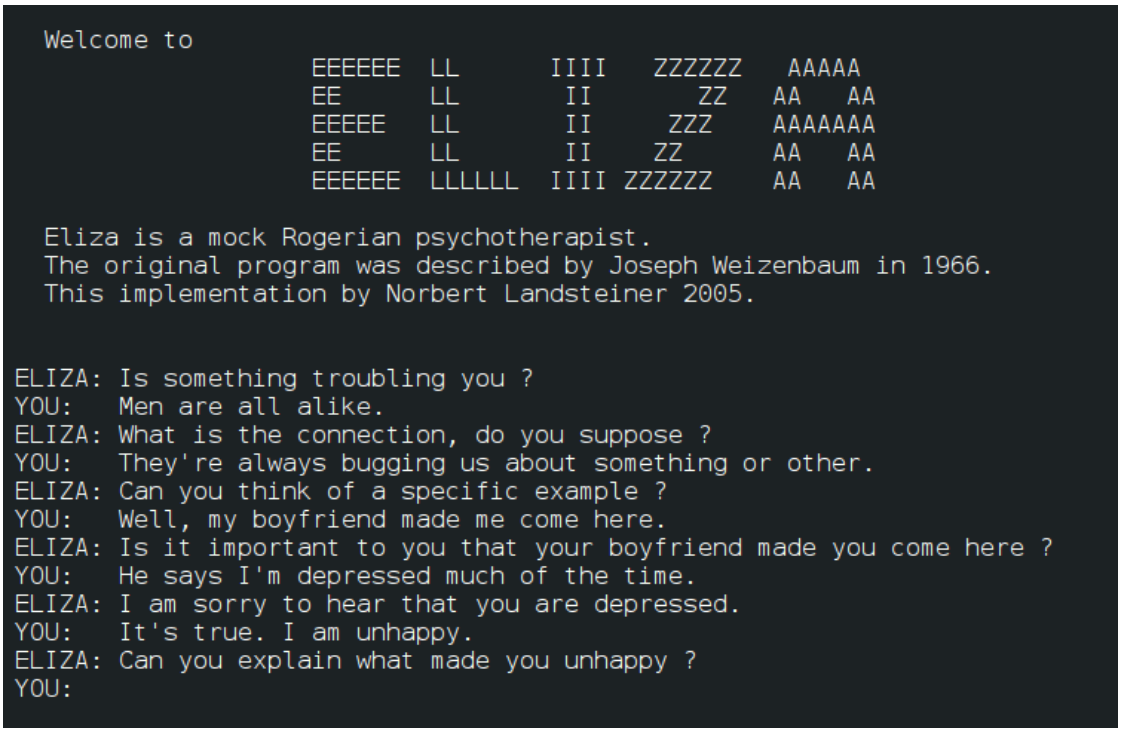


Figura 1.3: Interfaccia grafica del chatbot ELIZA

Il successo di ELIZA portò allo sviluppo di altri chatbot. Nel 1972, presso la Stanford University, lo psichiatra Kenneth Colby creò PARRY, un algoritmo più sofisticato rispetto a ELIZA. Questo programma era progettato per simulare il comportamento di una persona affetta da schizofrenia paranoide, basandosi sui comportamenti tipici osservati in tali pazienti. PARRY fu sottoposto a una variazione del test di Turing, durante il quale un gruppo di psicoterapeuti analizzò conversazioni generate da pazienti reali e dal programma tramite telescriventi. I terapeuti riuscirono a distinguere correttamente PARRY dai pazienti umani in meno della metà dei casi, superando così il test di Turing.

Nel 1988, mentre tutti i chatbot precedenti si basavano su database statici per generare risposte, fu sviluppato Jabberwacky, che utilizzava un linguaggio appreso dalle interazioni con le persone. Quest'ultimo aggiungeva al suo database le frasi utilizzate dagli utenti, espandendo, così, i propri contenuti e migliorando durante l'utilizzo. Partendo da Jabberwacky,

nel 2008 Rollo Carpenter sviluppò il celebre chatbot Cleverbot, che nel 2011 superò il test di Turing, venendo giudicato un essere umano nella maggioranza dei casi.

Il chatbot Dr. Sbaitso sviluppato nel 1991, sempre in ambito terapeutico, utilizzava la sintesi vocale per comunicare con gli utenti.

Nel 1995 Richard Wallace, ispirandosi al lavoro di ELIZA, sviluppò ALICE (Artificial Linguistic Internet Computer Entity), un chatbot programmato utilizzando il linguaggio AIML (Artificial Intelligence Markup Language). Questo linguaggio è impiegato per definire regole di corrispondenza che collegano le parole e le frasi fornite in input dagli utenti con specifiche categorie di argomenti. AIML è basato su XML (eXtensible Markup Language) ed è compatibile con numerose piattaforme e servizi. ALICE fu uno dei primi chatbot a utilizzare l'Intelligenza Artificiale per simulare una conversazione umana.

Nel 2001 la startup ActiveBuddy Inc. sviluppò il chatbot SmarterChild che venne distribuito su AOL Instant Messenger, fornendo agli utenti informazioni in tempo reale su notizie, meteo, sport e altro ancora. SmarterChild è stato uno dei primi chatbot a diventare popolare tra gli utenti di Internet.

Il primo chatbot ad utilizzare l'NLP è stato IBM Watson, lanciato nel 2006; esso prese il nome dal primo CEO di IBM, Thomas John Watson. Nel 2011 questo chatbot partecipò allo show televisivo statunitense "Jeopardy!"(una gara a quiz di cultura generale) riuscendo a sconfiggere i suoi avversari umani, tra cui il detentore del record di permanenza nello show. Grazie alla sua velocità superiore, Watson si aggiudicò il premio dimostrando una straordinaria capacità di rispondere alle domande più rapidamente rispetto ai concorrenti umani. Durante lo show il chatbot non era connesso ad Internet, ma disponeva di oltre 4 terabyte di dati da utilizzare per ricercare le informazioni. Ancora oggi Watson è utilizzato in vari settori, tra cui la sanità, il marketing e l'assistenza clienti.

Nel 2010 iniziò l'era dei chatbot intesi anche come assistenti virtuali personali; sono stati sviluppati numerosi chatbot sofisticati, come Siri di Apple (dapprima in forma di app per iPhone), integrato nel sistema operativo iOS nel 2011, Google Now nel 2012 per l'applicazione mobile di Google Search, infine nel 2015 Microsoft Cortana (oggi conosciuto come assistente nel sistema operativo Windows) e Amazon Alexa associato agli speaker per le smart home. Questi assistenti virtuali erano sempre connessi ad Internet, utilizzavano i comandi vocali e inviavano ogni domanda a un server remoto per l'elaborazione di una risposta.

Dal 2016, con il lancio della piattaforma di messaggistica Messenger di Facebook, i chatbot sono diventati sempre più popolari. Facebook ha introdotto la possibilità di creare chatbot per Messenger, consentendo alle aziende di interagire con i propri clienti tramite la piattaforma di messaggistica. Questi chatbot sono in grado di rispondere alle domande degli utenti, fornire informazioni sui prodotti e aiutare gli utenti a effettuare acquisti.

L'ordine cronologico degli eventi è riportato in Figura 1.4.

Dalla fine del 2022 fu rilasciato al pubblico ChatGPT, un chatbot e assistente virtuale sviluppato da OpenAI basato su modelli linguistici di grandi dimensioni, chiamati LLM (acronimo di Large Language Model). Esso divenne estremamente popolare raggiungendo più di 1 milione di utenti in soli 5 giorni, superando nettamente il confronto con le altre piattaforme divenute più popolari, come illustrato in Figura 1.5. A ChatGPT è riconosciuto il merito di aver innescato il boom dell'Intelligenza Artificiale, stimolando investimenti continui e attirando l'attenzione del pubblico sul settore dell'AI. A gennaio 2023 ChatGPT era diventata l'applicazione software per i consumatori con la crescita più rapida nella storia, raggiungendo oltre 100 milioni di utenti e contribuendo all'attuale valutazione di OpenAI stimata a 86 miliardi di dollari; ciò stimolò lo sviluppo e il rilascio in breve tempo di molti altri chatbot basati su LLM. Ad esempio Google rilasciò il chatbot Gemini solo pochi mesi dopo il lancio di ChatGPT. A seguire, venne rilasciato anche il chatbot Claude AI della società Anthropic e il chatbot Copilot di Microsoft.

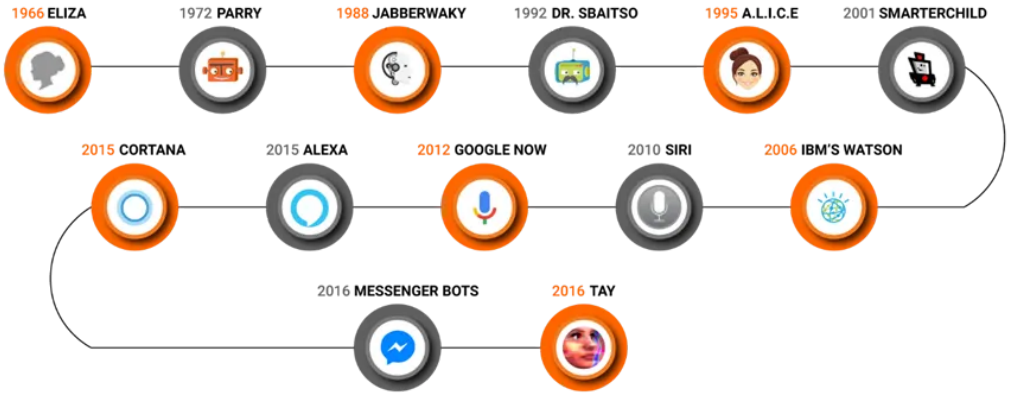


Figura 1.4: Cronologia degli eventi nella storia dei chatbot

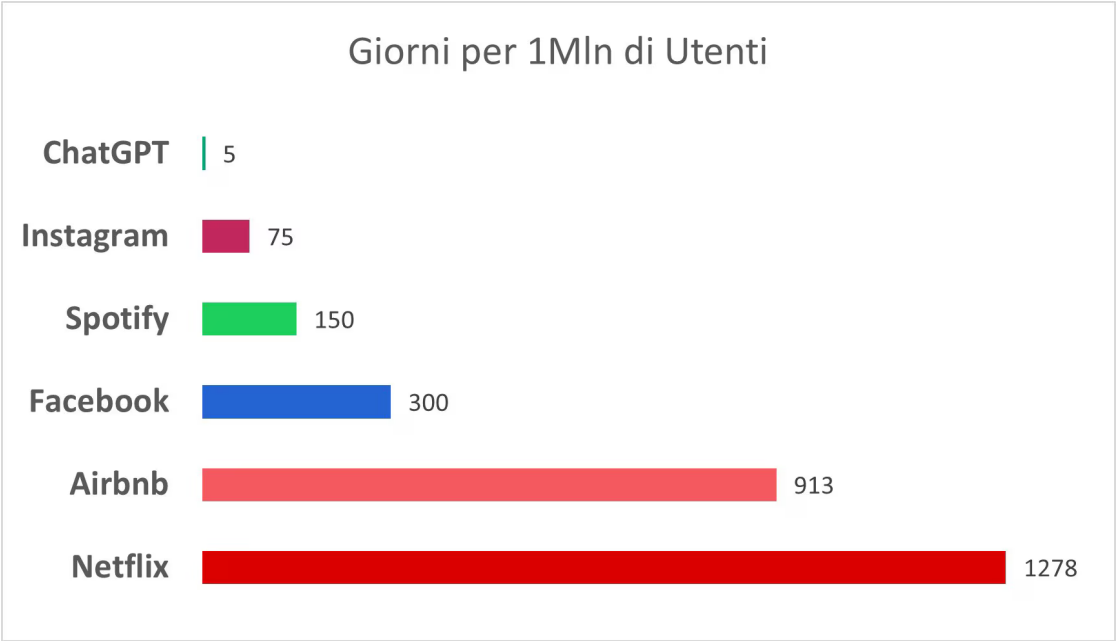


Figura 1.5: Grafico sulla crescita di ChatGPT in confronto ad altre piattaforme popolari

Attualmente i chatbot basati su LLM stanno gradualmente sostituendo i chatbot basati su regole statiche di NLP. Ad esempio nel 2023 l'assistente virtuale Cortana di Microsoft è stato dismesso e sostituito da Microsoft Copilot, un chatbot basato su LLM GPT-4 della società OpenAI. Allo stesso modo Google, con l'introduzione del chatbot Gemini (anch'esso basato su LLM), ha annunciato di voler sostituire Google Assistant con questo nuovo assistente virtuale.

1.3 Tipologie di chatbot

Non tutti i chatbot sono dotati di AI ma i chatbot moderni utilizzano sempre più le tecniche di AI conversazionale, come l'elaborazione del linguaggio naturale (NLP), per comprendere le domande degli utenti e automatizzare le risposte.

I chatbot possono essere classificati in diverse categorie in base al loro grado di sofisticazione e alle loro funzionalità. Le principali tipologie di chatbot sono le seguenti:

- *Chatbot basati su regole (modelli linguistici)*: questi chatbot utilizzano regole predefinite per rispondere alle domande degli utenti. Le regole sono definite a priori e possono includere parole chiave, frasi o modelli di conversazione. Essi lavorano sulla base di flussi di conversazione e logica "if/then" per tenere conversazioni strutturate con gli utenti. I chatbot basati su modello puramente linguistico possono essere rigidi e lenti da sviluppare a causa di questo approccio di definizione esplicito e dettagliato delle regole, ma consentono, per questo, un maggiore controllo e una migliore previsione della risposta. Questa tipologia di chatbot, molto comune, viene spesso utilizzata per le FAQ interattive o per assistenza di base nei servizi clienti.

- *Chatbot basati su Machine Learning (modelli AI)*: sono chatbot più complessi ma tendono ad essere più conversazionali e adattabili rispetto ai chatbot basati su regole. Possono comprendere il contesto delle conversazioni e fornire risposte personalizzate agli utenti. Essi imparano dalle interazioni precedenti e possono utilizzare le valutazioni degli utenti per migliorare nel tempo. Questo sistema richiede che vi sia a disposizione una elevata quantità di dati di training per poter apprendere correttamente. Tale processo di apprendimento comporta che il chatbot funzioni come una scatola nera (anche detto black-box): non è possibile sapere esattamente come il chatbot ha raggiunto una determinata risposta ed è, quindi, difficile intervenire per correggere eventuali errori. L'imprevedibilità di tale approccio rende questa tipologia di chatbot più difficile da controllare, e quindi meno adatta a contesti in cui è richiesta una risposta specifica e programmata.

- *Chatbot ibridi*: adottare un approccio ibrido tra le due tipologie precedenti consente di sfruttare i vantaggi di entrambe. I chatbot ibridi possono essere più flessibili e adattabili rispetto ai chatbot basati su regole e più controllabili rispetto ai chatbot basati su Machine Learning. È possibile, infatti, costruire chatbot anche senza dati, mantenendo, comunque, la trasparenza e il controllo nel funzionamento del sistema. Ciò consente delle integrazioni di Machine Learning per ottenere inferenze intelligenti e complesse, al contrario dei chatbot basati su regole che sono limitati a risposte predefinite. Adottando l'approccio ibrido, le aziende sono più confidenti nell'implementazione di chatbot in contesti in cui, comunque, è richiesta una risposta specifica e programmata.

È possibile classificare i chatbot anche in base alla modalità di interazione con gli utenti. Le principali tipologie di chatbot sulla base di tale classificazione sono i seguenti:

- *Chatbot testuali*: interagiscono con gli utenti tramite messaggi di testo. Possono essere integrati in siti web, applicazioni di messaggistica e piattaforme di social media. I chatbot testuali sono utilizzati per fornire assistenza ai clienti, rispondere alle domande degli utenti e automatizzare i processi di comunicazione.

- *Chatbot vocali*: utilizzano la tecnologia di riconoscimento vocale per comprendere il linguaggio parlato dagli utenti. Allo stesso modo rispondono utilizzando una sintesi vocale. Questo risultato viene ottenuto applicando una funzionalità di speech-to-text, per convertire l'input vocale in testo da far processare all'assistente e, viceversa, applicare il text-to-speech per riprodurre tramite audio l'output del chatbot. Questa tipologia di assistenti è utilizzata in applicazioni come i servizi clienti telefonici e i dispositivi vocali, quali gli altoparlanti intelligenti per le case domotiche.

- *Chatbot multicanale*: sono in grado di interagire con gli utenti su diversi canali, come siti web, applicazioni di messaggistica e dispositivi vocali intelligenti. Possono mantenere la coerenza e il contesto delle conversazioni su più canali e fornire un'esperienza utente uniforme.

- *Chatbot sociali*: sono integrati con piattaforme di social media come Facebook, Twitter e Instagram per interagire con gli utenti e fornire assistenza personalizzata. Possono rispondere alle domande degli utenti e fornire informazioni sui prodotti.

1.4 Vantaggi e svantaggi dei chatbot

I chatbot sono particolarmente utili per le aziende che desiderano automatizzare i processi di comunicazione e fornire assistenza ai clienti in modo efficiente. I principali vantaggi dei chatbot per le aziende includono:

- *Disponibilità*: i clienti possono utilizzare i chatbot per avere un servizio immediato in qualsiasi momento, 24 ore su 24, 7 giorni su 7.
- *Riduzione dei costi*: le aziende possono ridurre il personale, e quindi i costi del servizio clienti, utilizzando i chatbot per rispondere alle domande degli utenti in modo automatico.
- *Risposta immediata*: i clienti si aspettano una risposta immediata alle loro domande e i chatbot possono fornire assistenza istantanea.
- *Sponsorizzazione*: i chatbot possono essere utilizzati per promuovere i prodotti e i servizi dell'azienda e generare lead.
- *Automatizzazione*: i chatbot possono automatizzare le risposte alle domande frequenti, permettendo agli operatori umani di concentrarsi su compiti più complessi e strategici.
- *Espansione in nuovi canali*: i chatbot possono essere integrati in diversi canali di comunicazione, come siti web, applicazioni di messaggistica e social media, per raggiungere un pubblico più ampio.
- *Scalabilità*: i chatbot, a differenza degli operatori umani, possono gestire un numero illimitato di conversazioni simultanee, consentendo alle aziende di scalare facilmente le risorse in caso di necessità.
- *Rispetto delle normative*: i chatbot assicurano che le note legali non vengano trascurate e possono agire in trasparenza rispetto alle normative.
- *Aumento del coinvolgimento*: i clienti preferiscono metodi di comunicazione online e i chatbot possono aiutare le aziende a coinvolgere i clienti in modo più efficace.
- *Raccolta di informazioni sui consumatori*: i chatbot possono raccogliere dati sui comportamenti e le preferenze dei clienti per poterli analizzare e profilare.

Tuttavia, nonostante i numerosi vantaggi, i chatbot presentano alcuni limiti che frenano il loro potenziale. I principali svantaggi dei chatbot includono:

- se è necessario un chiarimento, piuttosto che richiedere informazioni all'utente tendono a fornire una risposta seppur imprecisa;
- non sempre riescono a comprendere il contesto della conversazione e possono fornire risposte fuorvianti;
- non sono in grado di mostrare empatia o comprendere le emozioni degli utenti;

- sono dipendenti dalla qualità dei dati di training e possono essere influenzati da bias e pregiudizi presenti nei dati;
- il rispetto della normativa GDPR e la privacy dei dati degli utenti è un problema per i chatbot, in quanto possono raccogliere e memorizzare informazioni personali;
- i chatbot non hanno la capacità umana di fare parallelismi con esperienze simili per risolvere circostanze complesse.

Per queste motivazioni è essenziale informare l'utente che sta interagendo con un chatbot e non con un operatore umano. In questo modo, se vi è la funzionalità di trasferire la conversazione a un operatore umano, l'utente può scegliere di farlo nel caso in cui non si senta compreso o non sia soddisfatto della risposta fornita dal chatbot.

1.5 Applicazioni dei chatbot

I chatbot sono utilizzati in una vasta gamma di settori e applicazioni e il mercato è in continua crescita. Secondo uno studio della società Stratview Research, illustrato nella Figura 1.6, il mercato globale dei chatbot è stimato a 5.27 miliardi di dollari nel 2023 e dovrebbe crescere a un tasso di crescita annuo composto (CAGR) del 23.28% fino al 2028, raggiungendo un valore di 14.97 miliardi di dollari.

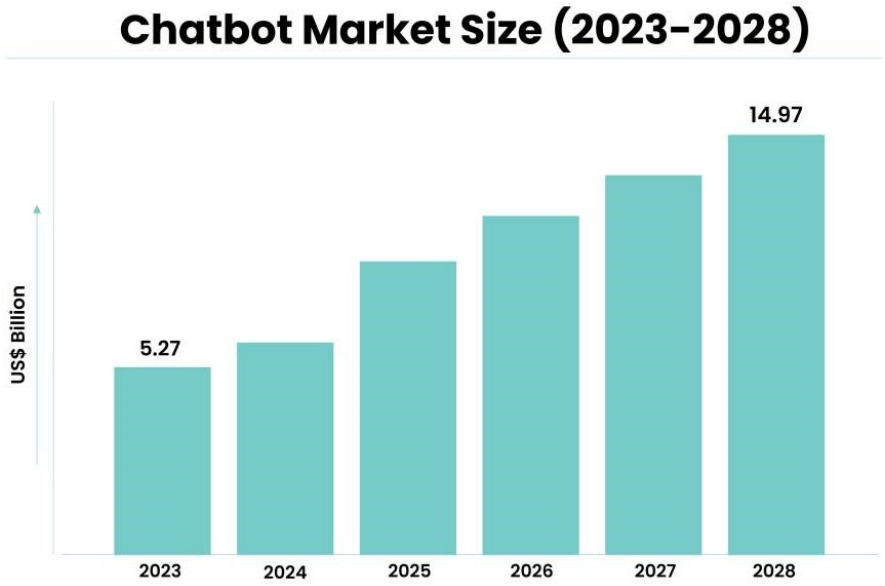


Figura 1.6: Grafico sulla crescita del mercato dei chatbot nel periodo 2023-2028

Una prima differenziazione dei casi d'uso dei chatbot può essere fatta in base all'utente di riferimento. In questo caso possiamo distinguere tra:

- *supporto ai clienti*: ad esempio, chatbot utilizzati per assistere i clienti nella ricerca di informazioni sui prodotti, nell'acquisto di beni e servizi e nell'assistenza post-vendita;
- *supporto ai dipendenti*: ad esempio, chatbot utilizzati per fornire informazioni sui processi aziendali e automatizzare i compiti amministrativi;
- *supporto all'accesso ai dispositivi*: ad esempio, chatbot utilizzati per il controllo della casa domotica e dei dispositivi connessi IoT.

I principali settori in cui i chatbot sono presenti, ordinati per percentuale, sono illustrati nella Figura 1.7. Nelle prossime sezioni verranno analizzati questi ambiti introducendo alcuni esempi che spaziano nella domotica e dell'IoT, nell'ambito bancario e finanziario, nelle telecomunicazioni, nella vendita online e nell'assistenza sanitaria.

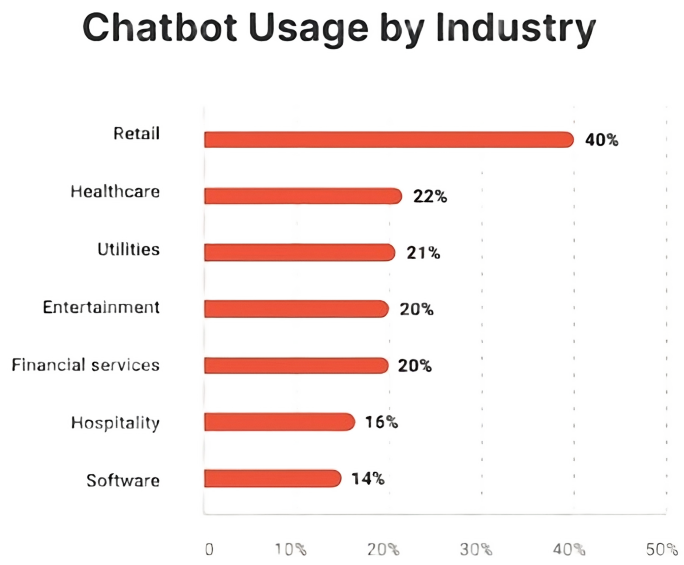


Figura 1.7: Grafico sull'utilizzo dei chatbot nei diversi settori

1.5.1 Domotica e IoT

L'Internet of Things (IoT) rappresenta un insieme di tecnologie che permettono di collegare e far interagire tra loro diversi dispositivi intelligenti, utilizzando le architetture Internet esistenti e in evoluzione; questi dispositivi lavorano insieme per raggiungere obiettivi comuni. L'ecosistema IoT include dispositivi con diverse funzioni e caratteristiche, come sensori di temperatura, luce, movimento, display, motori, calcolatori e interfacce di comunicazione. Le tecnologie IoT sono in costante sviluppo e sempre più diffuse grazie all'uso frequente di Internet nella vita quotidiana, per esempio nella condivisione e ricerca di informazioni e nell'interconnessione di oggetti fisici e virtuali. Essendo un campo in continua evoluzione, l'IoT presenta ancora diverse sfide. Una delle principali difficoltà è il controllo dei vari dispositivi, applicazioni e dati nell'IoT, che stanno diventando sempre più numerosi e complessi.

Pertanto una delle sfide più importanti in questo settore è la creazione di interfacce utente (UI) unificate, capaci di gestire simultaneamente diversi task e dispositivi, migliorando l'esperienza degli utenti e rendendola più intelligente e semplice. In questo contesto i chatbot possono essere una soluzione promettente grazie alla loro diffusione, facilità di adozione e capacità di interpretare comandi complessi attraverso il linguaggio naturale. Utilizzando un singolo chatbot è possibile gestire contemporaneamente dispositivi con funzioni simili (come due luci smart) e dispositivi con funzioni diverse (come una luce e un sensore di temperatura). Inoltre è possibile visualizzare i dati provenienti da differenti sistemi IoT.

Ad esempio, i chatbot sono utilizzati per controllare i dispositivi connessi nella casa intelligente, come luci, termostati, telecamere di sicurezza ed elettrodomestici. I chatbot possono essere integrati con assistenti vocali, come Amazon Alexa e Google Assistant, per controllare i dispositivi con comandi vocali. Per esempio, un utente può chiedere all'assistente Google di accendere le luci o regolare la temperatura del termostato utilizzando un dispositivo Google Home. In Figura 1.8 è mostrato uno schema sul funzionamento di Google Assistant dalla richiesta dell'utente in formato vocale fino all'azione sull'impianto domotico.

I chatbot sono utilizzati anche per monitorare i dispositivi connessi e inviare notifiche agli utenti in caso di problemi o anomalie. In uno scenario tipico un chatbot può avvisare un utente se la temperatura della casa è troppo alta o se una porta è rimasta aperta ed intraprendere delle azioni di conseguenza.

Recentemente i chatbot sono stati inseriti anche nei veicoli intelligenti per fornire assistenza alla guida, come informazioni sul traffico, o per regolare la climatizzazione. Nei casi d'uso più comuni un chatbot può aiutare un conducente a trovare la stazione di servizio più vicina o a pianificare un percorso alternativo in caso di traffico.

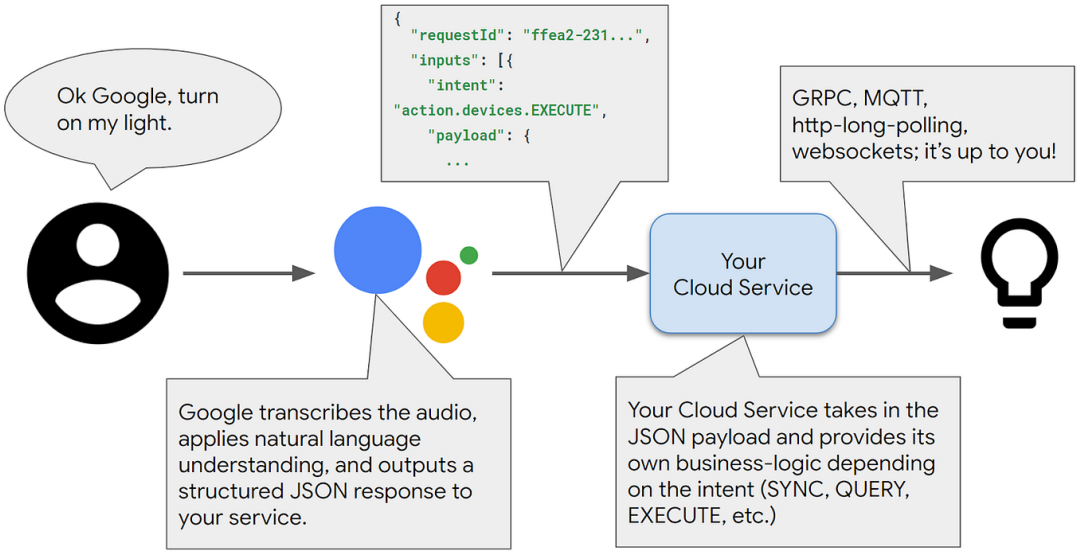


Figura 1.8: Schema sul funzionamento di Google Assistant

1.5.2 Ambito bancario e finanziario

Nell'ambito bancario e finanziario i chatbot possono essere utilizzati per semplificare le operazioni in modo colloquiale e intuitivo. Possono essere effettuate operazioni di controllo del conto, segnalazione di carte smarrite, rinnovo o gestione di una pratica. I chatbot forniscono assistenza immediata, lasciando la possibilità di essere trasferiti a un operatore umano in caso di necessità. Inoltre, essi possono essere utilizzati per fornire informazioni sui prodotti e servizi finanziari, come mutui, prestiti e investimenti.

Il chatbot può fornire un supporto anche ai dipendenti di back-office per le operazioni più ripetitive e dispendiose, come la gestione dei documenti e la verifica dei dati. Inoltre, i chatbot possono essere utilizzati per monitorare i mercati finanziari e fornire informazioni in tempo reale sugli investimenti.

Alcuni esempi sono:

- *Erica di Bank of America*; si tratta di un chatbot che fornisce assistenza ai clienti per le operazioni bancarie, come il controllo del saldo del conto, il pagamento delle bollette e il trasferimento di fondi.
- *Cleo*; è un chatbot che aiuta gli utenti a gestire le proprie finanze, fornendo consigli su come risparmiare denaro e tenere traccia delle spese.
- *PayPal*; il famoso sistema di pagamento offre un chatbot tramite Facebook Messenger per verificare lo stato delle controversie, gestire i pagamenti e controllare le limitazioni sul conto.

- *Ellis di Intesa San Paolo*; è il chatbot del servizio clienti della banca, che fornisce assistenza ai clienti accedendo anche alle informazioni sensibili, se l'utente ha effettuato l'accesso nella propria area riservata.
- *Unicredit*; in modo simile a Ellis, Unicredit ha sviluppato un chatbot per fornire assistenza ai clienti se autenticati, o comunque rispondere alle domande frequenti, come illustrato in Figura 1.9.
- Tante altre banche, come *Poste Italiane*, *Banca Sella* e *N26* hanno sviluppato chatbot per fornire assistenza ai clienti e semplificare le operazioni bancarie.

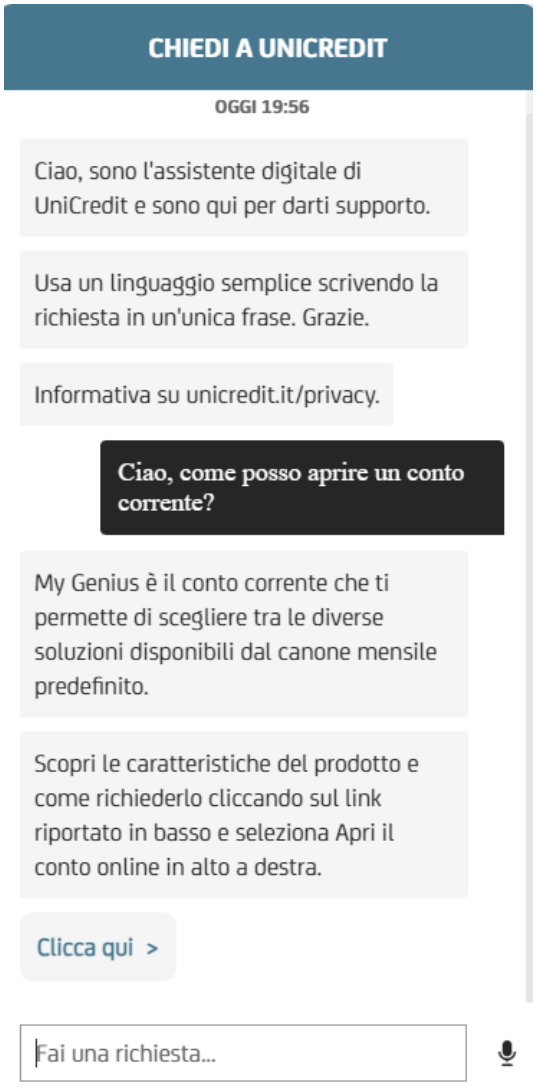


Figura 1.9: Interfaccia ed esempio di chat del chatbot di Unicredit

1.5.3 Telecomunicazioni

I chatbot sono utilizzati anche nel settore delle telecomunicazioni per fornire assistenza ai clienti, rispondere alle domande frequenti e gestire le richieste di supporto. Possono essere utilizzati per attivare nuove linee telefoniche, modificare i piani tariffari e risolvere i

problemi tecnici. I chatbot possono essere integrati con i sistemi di supporto aziendale per automatizzare i processi di assistenza e ridurre i tempi di risposta.

Grazie a questa comunicazione automatizzata è possibile ridurre i costi operativi e aumentare il cross-selling tra i clienti esistenti, offrendo servizi aggiuntivi e promozioni personalizzate.

Un'azienda di riferimento per il caso d'uso è Vodafone che utilizza l'assistente TOBi per conoscere costi e offerte, attivare servizi, cambiare il metodo di pagamento e risolvere problemi tecnici. TOBi è in grado di riconoscere il contesto della conversazione e fornire risposte personalizzate agli utenti. Il chatbot è di tipo multicanale, quindi può essere contattato tramite la piattaforma di messaggistica WhatsApp, dai normali SMS, dalla chat sul sito e anche tramite comandi vocali, grazie all'integrazione con Amazon Alexa. In Figura 1.10 è mostrato un esempio di chat con TOBi.



Figura 1.10: Interfaccia ed esempio di chat con l'assistente TOBi di Vodafone tramite WhatsApp

1.5.4 E-commerce

Uno dei settori maggiormente influenzati dall'avvento di Internet e delle nuove tecnologie è, senza dubbio, quello del commercio e del marketing. Negli ultimi anni, l'e-commerce ha conosciuto una crescita significativa. I chatbot, nel contesto dell'e-commerce, hanno lo scopo principale di gestire le richieste ridondanti e ridurre i tempi di attesa, abbattendo, così, i costi dell'assistenza clienti tradizionale e offrendo un servizio clienti immediato, disponibile 24 ore su 24. Ciò è utile a migliorare l'esperienza del cliente, il percorso d'acquisto e per personalizzare la comunicazione di marketing. In alcuni casi, i chatbot possono perfino effettuare acquisti per conto dell'utente, come nel caso di chatbot che funzionano come assistenti personali di shopping. Dall'altra parte, l'azienda può utilizzare i dati raccolti dai chatbot per analizzare il comportamento dei clienti e migliorare le strategie di marketing, per proporre offerte personalizzate e per inviare promozioni mirate.

Con la crescita dell'e-commerce e l'aumento dei dispositivi mobili, si è sviluppato anche il mobile e-commerce (m-commerce). L'm-commerce consente agli utenti di effettuare transazioni e operazioni di e-commerce tramite dispositivi mobili, favorendo l'espansione delle attività commerciali e abilitando transazioni in luoghi precedentemente inaccessibili. Le applicazioni più diffuse sono quelle di messaggistica istantanea (IM) e, proprio per questo motivo, i brand tendono sempre più a scegliere chatbot integrati nelle applicazioni IM già esistenti: "se si vuole costruire un business online, bisogna farlo dove si trovano le persone. Ora quel luogo è nelle app di messaggistica" (Matt Schlicht, CEO di Octane AI).

Recentemente, la famosa piattaforma di e-commerce di moda Zalando ha introdotto il chatbot "fashion assistant" che aiuta gli utenti a trovare i prodotti giusti, fornendo consigli di stile e suggerimenti di abbinamento. Il chatbot è in grado di comprendere i gusti stilistici dell'utente proponendo alcuni abbinamenti dei capi di moda in vendita. In Figura 1.11 è mostrato un esempio di chat ripreso dall'applicazione mobile di Zalando.

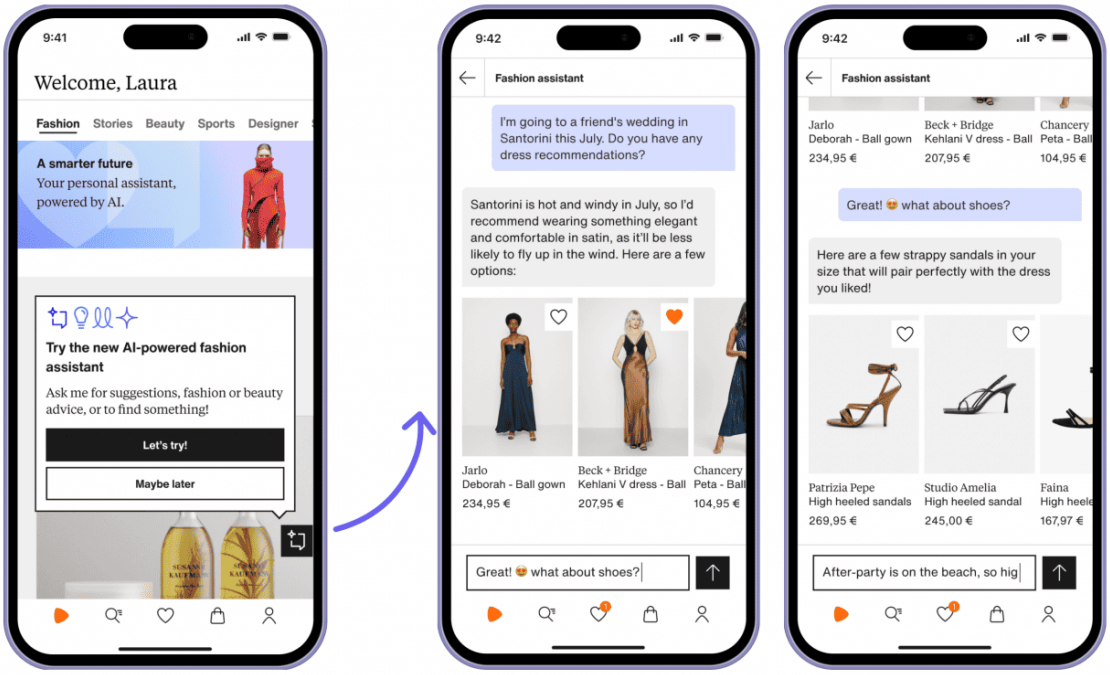


Figura 1.11: Interfaccia ed esempio di chat con Zalando fashion assistant

1.5.5 Assistenza sanitaria

L'ambito sanitario rappresenta uno dei numerosi settori in cui i chatbot possono essere implementati con vantaggi significativi. Sempre più persone, infatti, cercano informazioni relative alla propria salute attraverso servizi sanitari online. I chatbot offrono un modo semplice e gratuito per fornire agli utenti informazioni immediate e precise. Infatti, come già descritto nella sezione 1.2, i primi casi d'uso dei chatbot erano proprio nell'ambito medico e della salute mentale.

Un esempio notevole è Watson che, grazie alla capacità di analizzare i record medici di ospedali statunitensi in breve tempo, venne utilizzato per generare diagnosi mediche più accurate. Watson interagisce con i medici rispondendo a domande specifiche basate su documentazione medica disponibile e valida. Sulla base delle domande poste, Watson è in grado di generare nuove domande e rispondere utilizzando i dati estratti. Analizzando i dati medici, il sistema identifica termini chiave e ne verifica le connessioni con altri concetti, sviluppando e monitorando ipotesi basate su diversi indicatori. La capacità di migliorare la profondità delle ricerche e l'affidabilità delle risposte rende Watson un valido aiuto nel processo diagnostico, suggerendo trattamenti basati sulle pratiche mediche raccomandate. Nel 2022 Watson Health è stato venduto per 1 miliardo di dollari alla società Merative che opera ancora oggi per proseguire lo sviluppo del progetto con le nuove tecnologie presenti sul mercato.

Un'area medica in cui l'uso dei chatbot è sempre stata presente è l'assistenza alla salute mentale. Grazie ai progressi dell'Intelligenza Artificiale, i chatbot sono sempre più abili nel riconoscere e classificare le emozioni umane. Spesso, il costo di una consulenza con un esperto è elevato e non accessibile a tutti; i chatbot offrono un servizio di consulenza personalizzata e gratuita per la salute mentale, accessibile facilmente da chiunque. Essendo disponibili 24 ore su 24, essi rappresentano uno strumento funzionale per l'assistenza immediata. I chatbot si sono rilevati molto utili per una particolare categoria di pazienti introversi che hanno sempre avuto un blocco nel parlare con uno psicologo per tematiche delicate per cui temevano il giudizio altrui. Essi possono rilevare stati emotivi pericolosi, come panico o istinti suicidi, e adottare misure efficaci per contrastare tali impulsi, oltre a monitorare le condizioni psichiche dei soggetti.

I chatbot possono assistere anche i soggetti affetti da disabilità con tecnologie assistive. In questo caso essi informano gli utenti sui servizi, risorse e strumenti disponibili, riducendo il carico di lavoro degli uffici informativi. Ogni studente ha un proprio modo di apprendere e comunicare, quindi il chatbot deve essere il più flessibile possibile per adattarsi alle diverse disabilità. Per svolgere al meglio il suo compito di assistente personale, il chatbot è dotato di una personalità, rispondendo in modo empatico alle emozioni espresse dall'utente.

CAPITOLO 2

Introduzione ai Large Language Model

In questo secondo capitolo verranno introdotti i Large Language Model (LLM), modelli di Deep Learning di grandi dimensioni pre-addestrati su una grande quantità di dati. Verranno descritte le principali caratteristiche e l'architettura di un LLM, con particolare attenzione al Generative Pre-trained Transformer (GPT). Verrà illustrato il processo di addestramento e di Fine-Tuning di un LLM, e si vedrà come questo può essere sfruttato al meglio grazie alle tecniche di prompt engineering.

Si proseguirà con una disamina dell'evoluzione storica di questi modelli, mettendo in luce i progressi significativi e le innovazioni tecnologiche che hanno contribuito alla loro affermazione. Successivamente, verranno analizzate le diverse tipologie di LLM, evidenziando i vantaggi e le limitazioni delle varie configurazioni e architetture, incluse le implicazioni tecniche, etiche e operative.

Infine, si esploreranno le principali applicazioni degli LLM, tra cui la Retrieval-Augmented Generation (RAG), l'Architettura Agente e le Architetture Multi-Agente, e si vedrà come questi modelli vengano utilizzati per affrontare problemi complessi e migliorare l'efficienza in vari contesti.

2.1 Cos'è un Large Language Model

I Large Language Model (abbreviati LLM) sono dei modelli di Deep Learning di grandi dimensioni, ovvero pre-addestrati su una grande quantità di dati. L'obiettivo di questi modelli è quello di comprendere, interpretare, e quindi generare del testo. Gli LLM fanno parte della famiglia dei modelli di Generative AI, una specializzazione dell'Intelligenza Artificiale che è in grado di generare contenuti a partire da una richiesta, detta prompt. In Figura 2.1 viene illustrata la gerarchia precisa e i legami che vanno dall'Intelligenza Artificiale fino al caso d'uso più popolare: il chatbot ChatGPT. Un LLM rappresenta uno strumento versatile e utilizzabile in molti contesti. Esso può essere utilizzato per generare e riassumere testi, tradurre lingue, rispondere a domande (e quindi implementare un chatbot), scrivere del codice di programmazione e molto altro. Infatti, gli LLM rappresentano una delle tecnologie più avanzate nel campo del Natural Language Processing (NLP) e la loro funzione di base è quella di prevedere la parola successiva in una sequenza di testo. Una tipologia importante di LLM riguarda quelli basati Transformer (Generative Pre-trained Transformers), ovvero un insieme di reti neurali costituite da un encoder, un decoder e da un meccanismo di auto-attenzione.

2.1.1 Architettura di un Large Language Model

L'architettura di un Large Language Model è composta da tre parti principali:

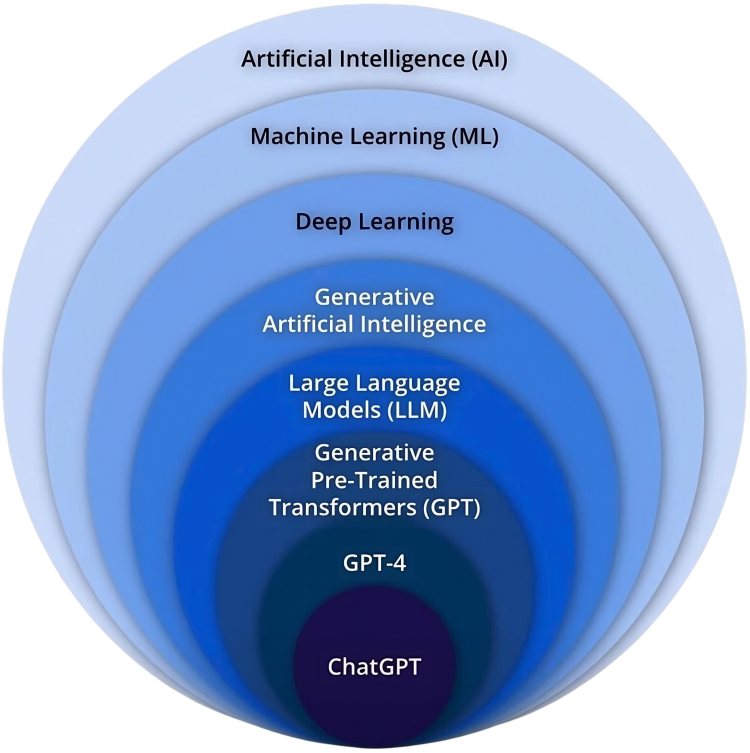


Figura 2.1: Gerarchia dei Large Language Model

- *Encoder*: è la parte del modello che si occupa di codificare il testo in ingresso. L'encoder trasforma il testo in token, ovvero una raffigurazione che il modello può comprendere e manipolare. Le precedenti forme di Machine Learning si basano su una tabella numerica per rappresentare le parole. Questa metodologia, però, non è sufficiente per riconoscere le relazioni tra le parole con significato simile. Per ovviare a questo, i Large Language Model utilizzano un vettore multidimensionale numerico che è in grado di catturare le relazioni semantiche tra le parole, consentendo al modello di comprendere il significato del testo in modo più accurato. Ciò avviene perché le parole vengono raggruppate con significati contestuali simili o con altre relazioni in modo che siano vicine tra loro nello spazio vettoriale.

- *Meccanismo di auto-attenzione*: è una tecnica basata su Transformer utilizzata nei modelli di Deep Learning per catturare le relazioni tra le parole in un testo. Questi algoritmi consentono al modello di concentrarsi su parti specifiche del testo durante la fase di addestramento. Il Transformer è un tipo di rete neurale introdotto nel 2017 e descritto nel documento intitolato "Attention is All You Need". Si distingue dalle precedenti reti neurali ricorrenti (RNN) e convoluzionali (CNN), pubblicate decenni prima e progettate per diversi tipi di compiti.

- *Decoder*: è la parte del modello che si occupa di decodificare la rappresentazione numerica (token) in uscita. In questo processo viene continuamente predetta la parola successiva nella sequenza di testo.

In Figura 2.2 è illustrata la struttura di un Large Language Model suddiviso nei relativi componenti. Il modello riceve in ingresso una grande quantità di contenuti testuali da diverse fonti, da qui viene elaborata una sequenza di token da rappresentare nello spazio vettoriale. Così, il modello è addestrato per prevedere il token successivo in base ai token precedenti.

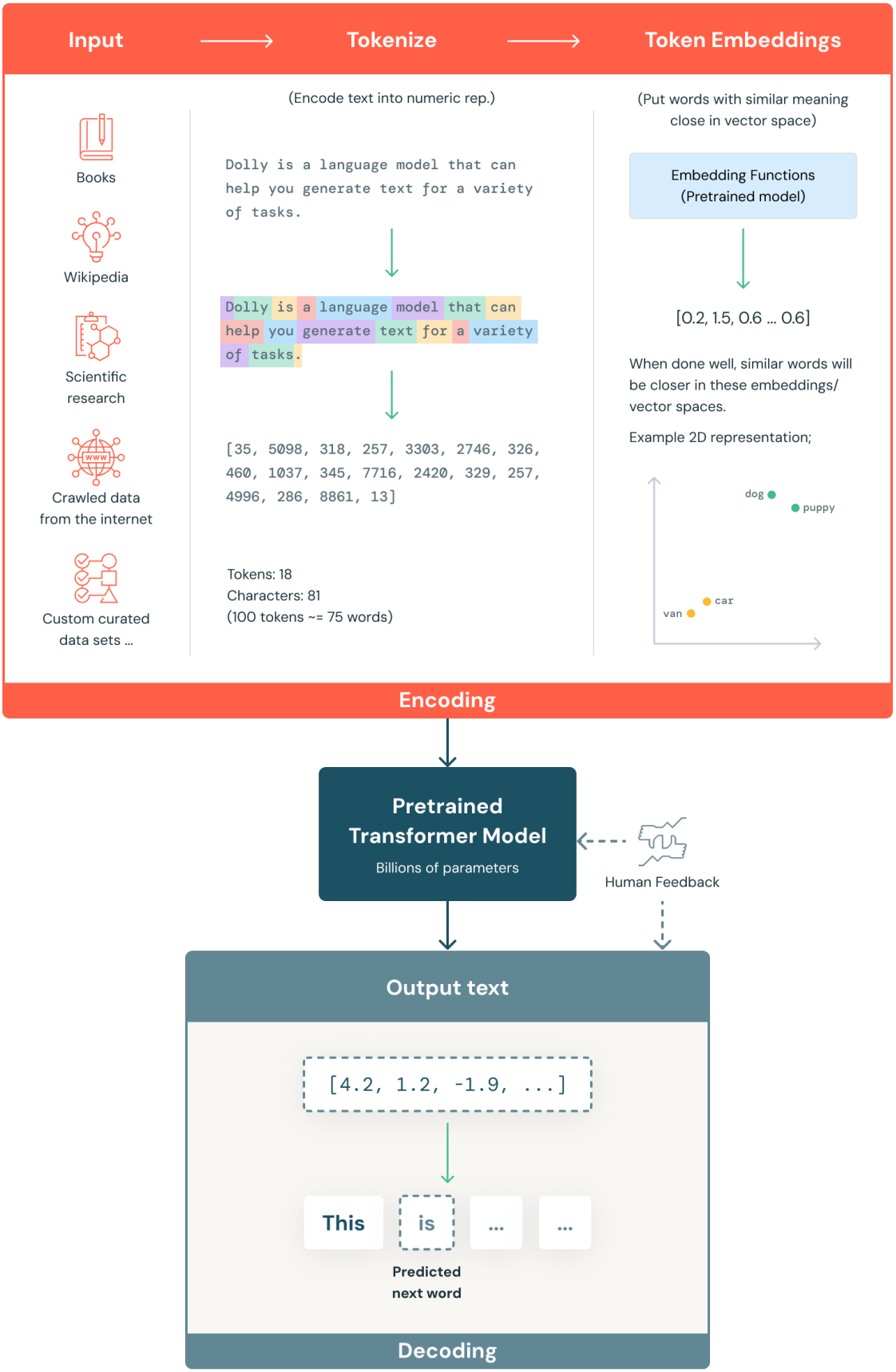


Figura 2.2: Struttura del funzionamento di un Large Language Model

Il Generative Pre-trained Transformer (GPT)

Un modello GPT è composto da L strati identici, intesi come blocchi Transformer, di cui ciascuno contenente due sottostrati: un meccanismo di auto-attenzione Multi-Head e una rete neurale Feed-Forward in base alla posizione. Ogni blocco è seguito da un livello di normalizzazione e da una connessione residuale. In Figura 2.3 è illustrata la struttura del modello GPT.

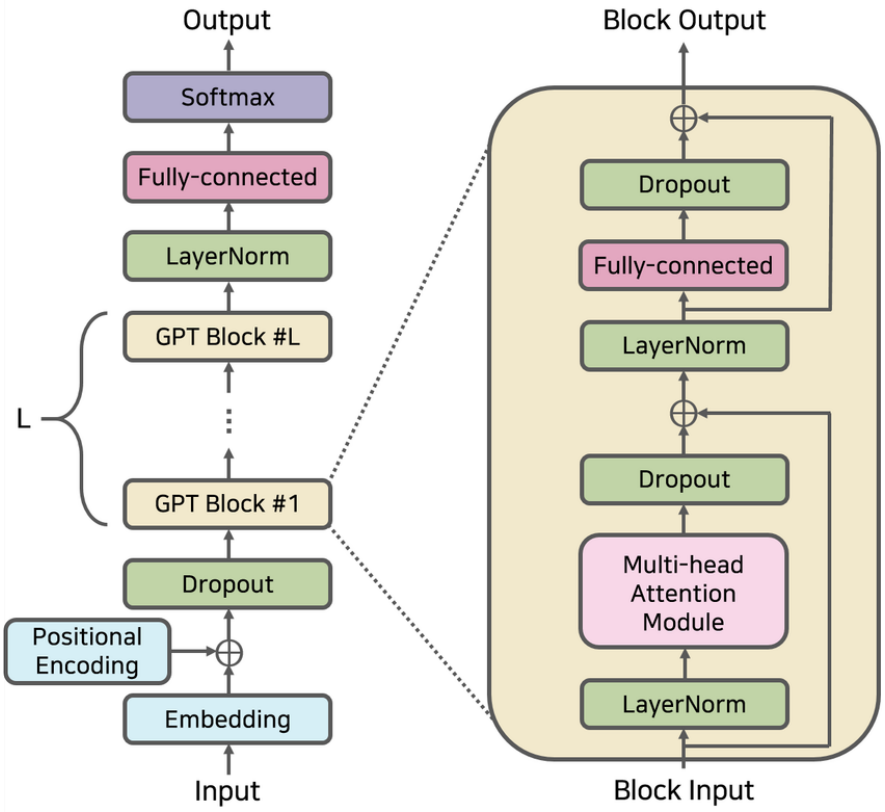


Figura 2.3: Schema concettuale dell'architettura di un modello GPT

Il meccanismo di auto-attenzione Multi-Head è un'operazione che permette al modello di concentrarsi su diverse parti del testo durante la fase di addestramento. Questo meccanismo è in grado di catturare le relazioni tra le parole in un testo, consentendo al modello di comprendere il significato del testo in modo più accurato. Per fare ciò, viene data in ingresso una sequenza di token e il modello calcola l'importanza di ciascuno di essi rispetto agli altri token nella sequenza.

Nel documento "Attention is All You Need" gli autori definiscono il processo di attenzione nel seguente modo:

> *"Una funzione di attenzione può essere descritta come la mappatura di una query e di un set di coppie chiave-valore in un output, dove la query, le chiavi, i valori e l'output sono tutti vettori. L'output è calcolato come una somma ponderata dei valori, dove il peso assegnato a ciascun valore è calcolato da una funzione di compatibilità della query con la chiave corrispondente."*

Questo processo viene chiamato "Scaled Dot-Product Attention".

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

Essa prende in ingresso la matrice delle query Q (rappresentazione vettoriale di una parola della sequenza), la matrice delle chiavi K e la matrice dei valori V. Nel caso di un modulo di attenzione Multi-Head, la matrice V è costituita dalla stessa sequenza di parole Q. Questo meccanismo viene ripetuto più volte, con proiezioni lineari di Q, K e V, per ottenere più rappresentazioni di attenzione che andranno concatenate tra loro. Durante l'addestramento, le rappresentazioni lineari vengono moltiplicate per le matrici di pesi W^Q, W^K e W^V per gli ingressi e la matrice W^O per l'uscita. Si definisce, quindi, la funzione di attenzione Multi-Head come:

$$MultiHead(Q, K, V) = Concat(head_1, ..., head_h)W^O$$

dove

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

Il meccanismo viene illustrato in Figura 2.4.

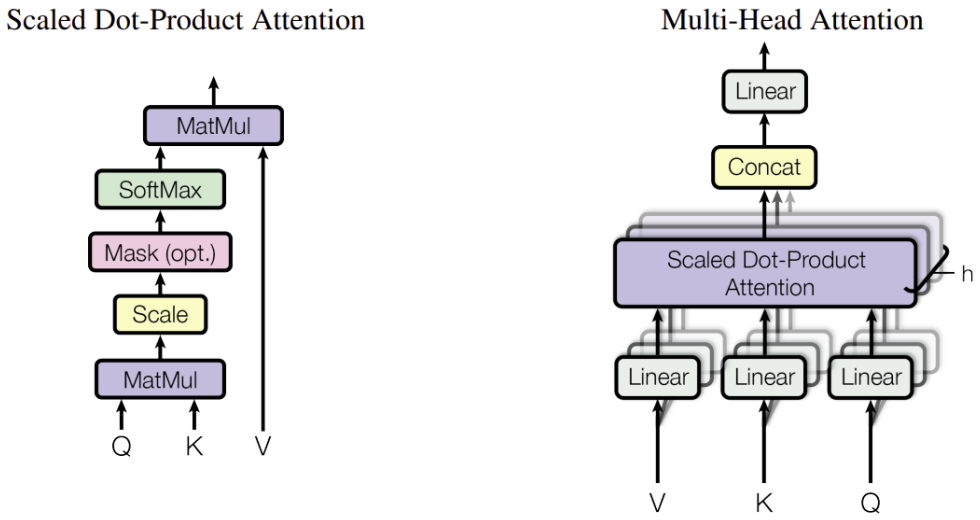


Figura 2.4: Schema concettuale dell'architettura Scaled Dot-Product Attention (a sinistra) e del meccanismo di attenzione Multi-Head (a destra)

L'operazione successiva consiste nell'attraversare la rete neurale completamente connessa Feed-Forward. Questa rete è composta da due strati lineari con una funzione di attivazione ReLU tra di essi, così definita:

$$FFN(x) = max(0, xW_1 + b_1)W_2 + b_2$$

Nell'equazione precedente, i termini W_1, W_2 sono i pesi, mentre b_1 e b_2 sono i bias della rete che vengono regolati durante la fase di addestramento. Questa operazione viene ripetuta per ogni token della sequenza. L'uscita di ciascun sottolivello nell'architettura GPT ($Sublayer(x)$) viene combinata con il suo ingresso (x) attraverso connessioni residue e viene successivamente normalizzata con la funzione LayerNorm. Essa è definita come:

$$LayerNorm(x + Sublayer(x))$$

Formalmente, si può quindi definire il modello GPT come:

$$H^{(L)} = GPT(E)$$

dove $H^{(L)}$ è la rappresentazione vettoriale della sequenza di token in uscita dal modello GPT, GPT è la funzione di trasformazione del modello formata da L livelli descritti in precedenza ed E è la sequenza di token in ingresso.

2.1.2 Fase di addestramento

Nel Machine Learning, l'addestramento di un modello è un processo di apprendimento su come eseguire una specifica attività. Le tipologie di addestramento sono le seguenti:

- *Apprendimento supervisionato*: il modello viene addestrato su un set di dati etichettati, ovvero dati che contengono sia le variabili in ingresso che le variabili di uscita del processo. Esso apprende, quindi, dalla relazione tra i dati di ingresso e i dati di uscita, cercando una mappatura che minimizzi l'errore tra l'uscita prevista dai dati di apprendimento e quella ottenuta dal modello.
- *Apprendimento non supervisionato*: il modello viene addestrato su un set di dati non etichettati, ovvero dati che contengono solo le variabili di ingresso. Esso cerca di apprendere dalle informazioni intrinseche e dalle strutture (pattern) nei dati.
- *Apprendimento semi-supervisionato*: il modello viene addestrato su un set di dati che contiene sia dati etichettati che dati non etichettati. Esso utilizza i dati etichettati per apprendere la relazione tra i dati di ingresso e i dati di uscita, mentre utilizza i dati non etichettati per trovare strutture e informazioni nei dati.
- *Apprendimento per rinforzo*: il modello viene addestrato su un set di dati che contiene una serie di azioni e di ricompense associate a tali azioni. Esso apprende in modo da massimizzare la ricompensa attraverso l'interazione con l'ambiente.
- *Apprendimento autosupervisionato*: il modello utilizza l'apprendimento non supervisionato per attività che normalmente richiedono tecniche di apprendimento supervisionato. Esso genera etichette implicite dai dati non strutturati, procedendo, quindi, con l'addestramento supervisionato.

Il punto di forza dell'architettura GPT è il suo apprendimento autosupervisionato che facilita l'apprendimento da dati non etichettati, comprendendo la grammatica, la lingua e le conoscenze generali.

Il modello viene alimentato con un vasto set di dati testuali da analizzare e apprendere. Durante la fase di addestramento, il modello regola iterativamente i propri parametri fino a riuscire a prevedere correttamente il token successivo, basandosi sul contesto fornito dai token precedenti. Questo processo di apprendimento avviene attraverso tecniche di autoapprendimento che insegnano al modello a ottimizzare i parametri, al fine di massimizzare la probabilità di prevedere correttamente i token successivi nei dati di training. Tale processo è strutturato come un problema di stima della massima verosimiglianza, con l'obiettivo di ottimizzare la probabilità di generare i dati osservati in funzione dei parametri del modello.

Le reti neurali basate sui trasformatori sono estremamente complesse e comprendono numerosi nodi e livelli. In ogni livello, ciascun nodo è connesso a tutti i nodi del livello successivo, con ogni connessione caratterizzata da un peso e un orientamento specifico che, insieme agli incorporamenti, costituiscono i parametri del modello. Le reti neurali di grandi dimensioni basate su trasformatori possono contenere miliardi di parametri. Un numero maggiore di parametri consente al modello di riconoscere strutture più complesse nei dati e di fornire prestazioni superiori nelle attività proposte. La dimensione del modello è generalmente determinata da una relazione empirica tra la dimensione del modello stesso, il volume dei dati utilizzati per l'addestramento e il numero di parametri.

Come tutti gli algoritmi di Deep Learning, disponendo di un elevato numero di parametri, per l'addestramento di un LLM sono necessarie grandi risorse computazionali. Per questo motivo, l'addestramento di un LLM richiede l'utilizzo di hardware specializzato, come le GPU.

Una volta completato l'addestramento, gli LLM possono essere facilmente adattati per eseguire una varietà di compiti utilizzando set relativamente piccoli di dati supervisionati, attraverso un processo noto come Fine-Tuning.

2.1.3 Fase di Fine-Tuning

Il Fine-Tuning è un metodo di addestramento che consente di adattare un modello pre-addestrato a un compito specifico utilizzando un set di dati di training relativamente piccolo. Questo processo consente di ottenere prestazioni migliori rispetto all'addestramento da zero, in quanto il modello pre-addestrato ha già appreso una vasta gamma di conoscenze linguistiche generali e può essere facilmente adattato a nuovi compiti più specifici. Il Fine-Tuning è basato su un processo di apprendimento supervisionato, in cui il modello viene addestrato su un set di dati strutturati ed etichettati specificando, per ogni domanda, la risposta attesa. Ad esempio, OpenAI permette il Fine-Tuning dei propri modelli ricevendo in ingresso un file in formato `jsonl`, strutturato come illustrato nel Listato 2.1. Per ogni messaggio, è necessario specificare il ruolo (utente o assistente) e il contenuto. Il modello viene, quindi, addestrato per prevedere la risposta corretta in base al contesto fornito dai messaggi precedenti.

```
{
  "messages" :
  [
    {
      "role": "user",
      "content": "Quanti sono i continenti del mondo?"
    },
    {
      "role": "assistant",
      "content": "Ci sono 7 continenti nel mondo."
    }
  ]
}
```

Listato 2.1: Esempio del file `jsonl`

Nello specifico, questa operazione consiste nel regolare i parametri del modello al fine di massimizzare la sua precisione sul set di dati di training. Solitamente, il Fine-Tuning avviene modificando solo gli ultimi strati della rete neurale, cambiando la modalità con cui vengono interpretate le informazioni ma senza alterare i livelli precedenti responsabili di estrarre le informazioni linguistiche generali. Durante questo processo, è importante tenere in considerazione alcuni fattori:

- *Dimensione del set di dati di training*: un set di dati di training più grande consente al modello di apprendere più informazioni e di ottenere prestazioni migliori. Tuttavia, anche se il Fine-Tuning richiede meno risorse rispetto all'addestramento iniziale, può comunque essere computazionalmente intensivo, soprattutto quando si lavora con grandi modelli e set di dati.
- *Regolazione degli iperparametri*: durante il Fine-Tuning è possibile regolare gli iperparametri del modello, come il tasso di apprendimento e il numero di epoche, per ottenere prestazioni migliori. I valori degli iperparametri devono essere scelti accuratamente per garantire che il modello non modifichi eccessivamente i suoi parametri originali.

- *Overfitting*: durante il Fine-Tuning c'è il rischio che il modello si adatti troppo ai dati specifici, perdendo la capacità di generalizzare ad altri contesti. È importante monitorare le prestazioni e applicare tecniche come la regolarizzazione per prevenire questo problema.

2.1.4 Prompt engineering

Il prompt engineering è una tecnica utilizzata per migliorare le prestazioni di un modello di LLM. Esso consiste nel formulare in modo accurato le richieste fatte al modello, in modo da ottenere risposte più precise e coerenti. Tali richieste, dette prompt, possono essere utilizzate per guidare il modello nella generazione di testo. Questo processo, che potrebbe sembrare immediato, richiede in realtà una comprensione profonda del funzionamento del modello e un'attenta considerazione della struttura e della formulazione del prompt stesso. Ad esempio, un prompt troppo generico potrebbe far sì che il modello produca una risposta vaga o non pertinente.

Secondo OpenAI, le principali linee guida per la creazione di prompt efficaci sono le seguenti:

- *Scrivere istruzioni chiare*: le richieste fatte al modello devono essere chiare e lasciare al modello pochi margini di interpretazione. È importante specificare esattamente il risultato atteso.
- *Fornire testi di riferimento*: i modelli possono inventare informazioni se vengono interrogati su argomenti che non conoscono. Fornire testi di riferimento può aiutare il modello a generare risposte senza inventare informazioni.
- *Suddividere le attività complesse in passaggi più semplici*: le attività complesse tendono ad avere tassi di errore più elevati rispetto alle attività semplici. Le attività complesse possono essere suddivise in passaggi più semplici in cui il risultato delle attività precedenti viene utilizzo per costruire gli ingressi delle attività successive.
- *Forzare il modello a pensare in modo specifico*: i modelli possono commettere errori di ragionamento se si chiede loro di rispondere subito a domande complesse. È possibile forzare il modello a pensare chiedendo esplicitamente di scrivere una catena di ragionamenti logici per arrivare alla risposta.
- *Utilizzare strumenti esterni*: esistono strumenti esterni che possono aiutare a compensare le debolezze del modello. Ad esempio, è possibile utilizzare dei motori di esecuzione del codice per aiutare il modello a scrivere e verificare il codice eseguendolo.
- *Testare le modifiche sistematicamente*: per migliorare le prestazioni del modello è necessario misurarle in modo sistematico. È possibile, infatti, che su un prompt il modello otterrà prestazioni migliori su alcuni esempi isolati ma peggiori su un set di esempi più rappresentativo.

Ciò ha portato allo sviluppo di tecniche avanzate di prompt engineering, tra cui:

- *Prompt zero-shot*: consiste nel progettare prompt che consentono al modello di generare risposte senza la necessità di fornire ulteriore contesto o esempi. Il modello risponde alla richiesta basandosi interamente sulla conoscenza esistente acquisita durante la fase di addestramento, il che riduce il numero di token generati e può aiutare a mantenere i costi di elaborazione più bassi.

- *Prompt few-shot*: può essere utilizzato per adattare il modello con un apprendimento dal contesto in cui vengono fornite alcune dimostrazioni pratiche e/o esempi per guidarlo ad ottenere delle prestazioni migliori.
- *Prompt chain-of-thought*: questo prompt consente di guidare il modello attraverso una serie di passaggi logici per arrivare a una risposta. Si può combinare con il prompt few-shot per ottenere risultati migliori.

2.2 Storia dei Large Language Model

La storia e l'evoluzione dei Large Language Model rappresentano uno dei capitoli più rapidi nello sviluppo dell'Intelligenza Artificiale. Questo percorso, che ha avuto inizio qualche decennio fa, ha visto una crescita esponenziale negli ultimi anni, grazie ai progressi nella tecnologia del Machine Learning e all'aumento della disponibilità di dati testuali.

Dall'anno 2014, con il paper "Sequence to Sequence Learning with Neural Networks", fu introdotto il concetto di encoder-decoder per la traduzione automatica, concetto che poi influenzerà lo sviluppo dei Transformer. Nel 2017 venne pubblicato il famoso paper "Attention is All You Need" che introdusse l'architettura dei Transformer, un'architettura basata su meccanismi di attenzione che ha rivoluzionato il campo dell'NLP. A differenza delle reti neurali ricorrenti (RNN) e delle reti neurali convoluzionali (CNN), i Transformer si basano su un meccanismo di attenzione che permetteva di considerare il contesto globale di una sequenza di parole. Questo paper ha aperto la strada a una nuova generazione di modelli di apprendimento automatico, tra cui i Large Language Model.

Da quel momento, la crescita e l'evoluzione degli LLM sono state esponenziali. Nel 2018, iniziarono i primi rilasci di modelli di LLM, tra cui il modello GPT-1 di OpenAI e il modello BERT (Bidirectional Encoder Representations from Transformers) di Google. Questi modelli hanno dimostrato la capacità di generare testo di alta qualità e di svolgere una varietà di compiti linguistici, come la traduzione automatica e la generazione di contenuti. Da allora, sono stati sviluppati numerosi altri modelli di LLM, tra cui GPT-2, GPT-3, che hanno ulteriormente migliorato le prestazioni dei modelli di LLM.

L'apice di questa evoluzione si ottenne con il rilascio di GPT-3, modello da 175 miliardi di parametri che fu reso disponibile al pubblico tramite il chatbot ChatGPT alla fine del 2022. Questo chatbot mostrò da subito una capacità conversazionale senza precedenti, in grado di generare stupore anche tra gli esperti del settore e le principali testate giornalistiche. In questo modo aumentò l'interesse generale verso i Large Language Model e le loro potenzialità e diverse aziende iniziarono a investire ingenti risorse per lo sviluppo di nuovi modelli di LLM. Nella Tabella 2.1 sono riportati i principali modelli di LLM con il numero di parametri, mentre in Figura 2.5 sono raffigurati anche gli altri modelli in ordine cronologico.

2.3 Tipologie di Large Language Model

Data la grande quantità di modelli LLM disponibili, è fondamentale comprendere le loro differenze per poter scegliere quello più adatto alle proprie esigenze. I modelli di LLM possono variare in base a diversi fattori:

- *Numero di parametri*: essi sono i pesi che il modello utilizza per prendere decisioni ed effettuare previsioni. Un modello con un numero maggiore di parametri è in grado di catturare relazioni più complesse nei dati e di ottenere prestazioni migliori in una varietà di compiti. Tuttavia, un modello con un numero maggiore di parametri richiede più risorse computazionali per l'addestramento e l'elaborazione.

Nome del modello	Azienda sviluppatrice	Data di rilascio	Numero di parametri (miliardi)
GPT-1	OpenAI	Giugno 2018	0.117
BERT	Google	Ottobre 2018	0.34
GPT-2	OpenAI	Febbraio 2019	1.5
T5	Google	Ottobre 2019	11
GPT-3	OpenAI	Maggio 2020	175
Claude	Anthropic	Dicembre 2021	52
LLaMA	Meta	Febbraio 2023	65
GPT-4	OpenAI	Marzo 2023	Sconosciuti
LLama 2	Meta	Luglio 2023	70
Cluade 2	Anthropic	Luglio 2023	Sconosciuti
Mistral 7B	Mistral AI	Settembre 2023	7.3
Claude 3	Anthropic	Marzo 2024	Sconosciuti
Qwen2	Alibaba Cloud	Giugno 2024	72
LLama 3.1	Meta	Luglio 2024	405

Tabella 2.1: Lista dei principali modelli di Large Language Model in ordine ascendente di rilascio

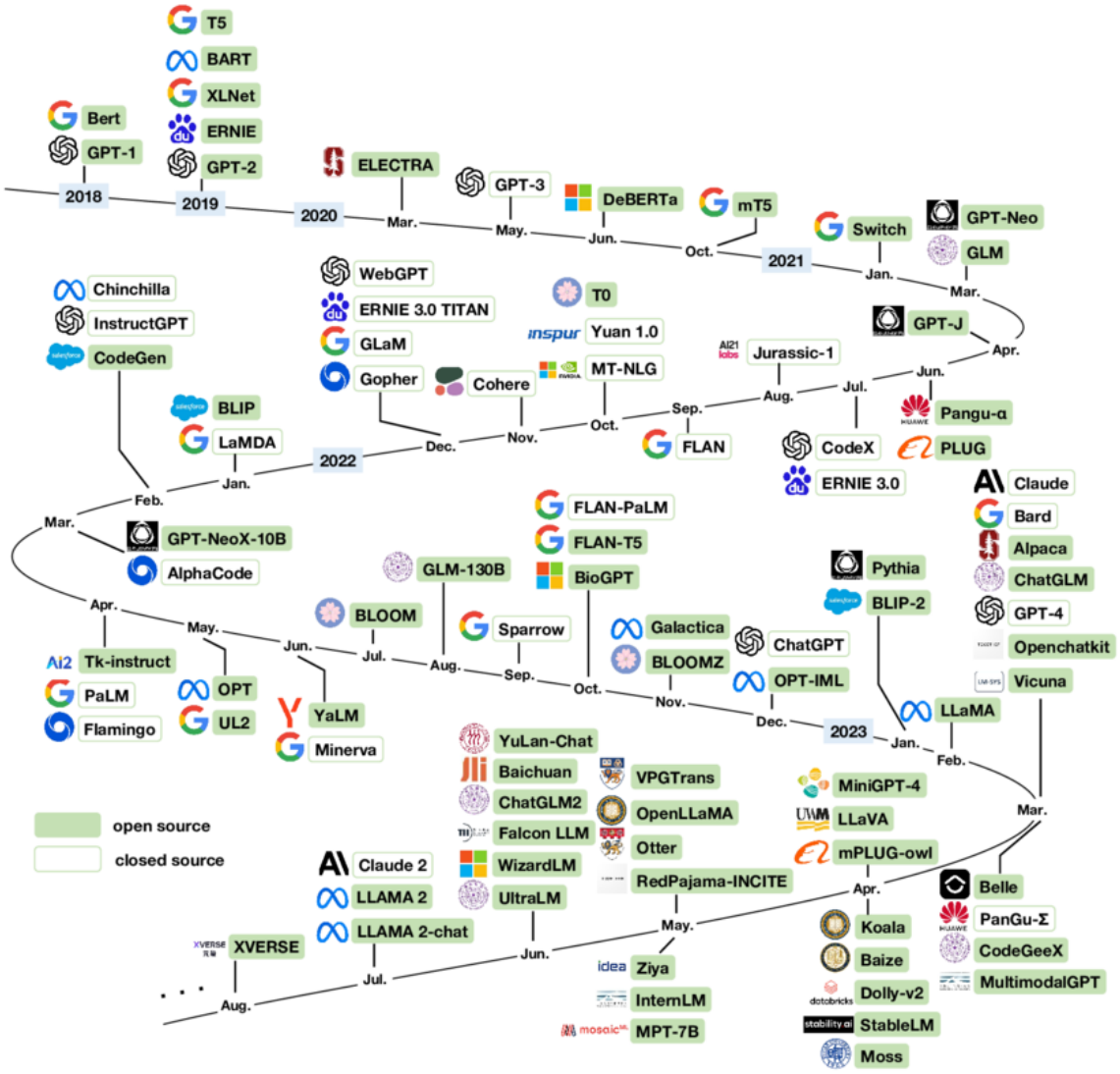


Figura 2.5: Ordine cronologico dei rilasci dei modelli di LLM

- *Dimensione del contesto*: si riferisce al numero di token che il modello può elaborare nella sequenza in ingresso per restituire delle risposte. Infatti, gli LLM hanno una lunghezza di contesto fissa e indica che può essere considerata solo una quantità limitata di testo alla volta per comprendere le richieste. Maggiore è la dimensione del contesto, maggiore è la quantità di informazioni che il modello può utilizzare per generare risposte accurate.
- *Architettura*: i modelli LLM basati su Transformer possono variare in base all'architettura utilizzata. Tra queste troviamo:
  - *LLM encoder-only*: è un modello che utilizza solo l'encoder del Transformer per generare le risposte. Esso si concentra sulla comprensione e l'elaborazione del testo in ingresso, senza generare attivamente le risposte. Questo tipo di modello è adatto per compiti di classificazione del testo e di analisi del sentiment. Un esempio di modello encoder-only è BERT sviluppato da Google.
  - *LLM decoder-only*: è un modello che utilizza solo il decoder del Transformer per generare le risposte. Esso genera la parte successiva della sequenza in ingresso in base al contesto precedente. A differenza dei modelli encoder-only, i modelli decoder-only non riescono a comprendere interamente l'ingresso ma si focalizzano sul generare la prossima parola più probabile. Questo fa sì che questi modelli siano più creativi.
  - *LLM decoder-encoder*: è un modello che utilizza sia l'encoder che il decoder del Transformer per generare le risposte. Questo tipo di modello è adatto per compiti che richiedono sia la comprensione del testo in ingresso che la generazione di risposte, come la traduzione automatica e l'elaborazione di riassunti.
- *Dataset di addestramento*: i modelli LLM vengono addestrati su grandi quantità di dati testuali provenienti da diverse fonti. Il dataset di addestramento può influenzare le prestazioni del modello in diversi compiti. Ad esempio, un modello addestrato su un dataset di notizie potrebbe essere più adatto per compiti di generazione di testi informativi, mentre un modello addestrato su un dataset di conversazioni potrebbe essere più adatto per compiti di generazione di testi conversazionali.
- *Licenza*: come avviene tradizionalmente per qualsiasi software, anche per i modelli LLM è importante considerare la licenza con cui vengono rilasciati. Essa determina come essi possano essere utilizzati, distribuiti, modificati e integrati in altre applicazioni. Alcuni modelli sono rilasciati con licenze open-source che consentono di utilizzarli liberamente e di modificarli, mentre altri modelli sono rilasciati con licenze proprietarie che limitano l'uso e la modifica del modello. A differenza dei software, però, che sono composti da codice sorgente, i modelli LLM vengono creati utilizzando dati di addestramento e parametri (pesi). Si possono identificare tre tipologie di licenze:
  - *Licenza open-source*: i modelli rilasciati con licenze open-source possono essere utilizzati liberamente e modificati da chiunque, sia per scopi personali che commerciali. Infatti viene reso disponibile il codice sorgente utilizzato per addestrare il modello, nonché tutto il dataset di addestramento. Questi modelli sono spesso rilasciati con licenze come la MIT o Apache, che consentono di utilizzare il modello per scopi commerciali e non commerciali. Un esempio di modello open-source è il modello BERT di Google.
  - *Licenza open-weight*: i modelli rilasciati con licenze open-weight sono simili ai modelli open-source, ma consentono di utilizzare solo i pesi del modello e non il

codice sorgente. Questa licenza offre, quindi, la possibilità di utilizzare i parametri del modello per operazioni di Fine-Tuning, ma non consente di modificare il modello in forma originale. Questi modelli possono essere utilizzati liberamente per scopi commerciali e non commerciali, ma non possono essere modificati. Ad esempio il modello Mistral di Mistral AI è rilasciato con una licenza open-weight.

- *Licenza proprietaria*: i modelli rilasciati con licenze proprietarie sono soggetti a restrizioni sull'uso e sulla modifica. Questi modelli possono essere utilizzati solo per scopi specifici e non possono essere modificati o distribuiti senza il permesso del proprietario del modello. Solitamente essi vengono rilasciati come servizi utilizzabili tramite API con tariffe di utilizzo. Un esempio di modello con licenza proprietaria è GPT-4 di OpenAI.
- *Licenza commerciale*: alcuni modelli sono rilasciati con licenze commerciali che consentono di utilizzare il modello solo per scopi commerciali. Questi modelli possono essere soggetti a tariffe di licenza e a restrizioni sull'uso.

2.4 Vantaggi e svantaggi dei Large Language Model

I Large Language Models (LLM) hanno rivoluzionato il campo dell'Intelligenza Artificiale, soprattutto nell'ambito dell'elaborazione del linguaggio naturale (NLP). Questi modelli hanno mostrato un grande potenziale nel risolvere una vasta gamma di compiti complessi legati alla manipolazione del testo. Grazie a questa flessibilità gli LLM offrono numerosi vantaggi. Tra questi, troviamo:

- *Comprensione del linguaggio naturale*: i modelli di LLM sono in grado di comprendere il linguaggio naturale e di rispondere a domande in modo accurato e coerente. Questo li rende utili per compiti di analisi del testo, risposta alle domande e generazione di testi conversazionali.
- *Generazione di testo di alta qualità*: i modelli di LLM sono in grado di generare testo di alta qualità che è coerente, ben strutturato e grammaticalmente corretto. Questo li rende utili per una varietà di compiti, come la generazione di contenuti, la traduzione automatica e la generazione di riassunti.
- *Adattabilità a diversi compiti*: i modelli di LLM possono essere facilmente applicati a una vasta gamma di casi d'uso e adattati ulteriormente utilizzando il Fine-Tuning. Questo li rende flessibili e adatti ad una varietà di applicazioni.
- *Risparmio di tempo e risorse*: i modelli di LLM possono automatizzare compiti che richiedono una comprensione del linguaggio naturale, riducendo il tempo e le risorse necessarie per svolgere tali compiti manualmente. Questo può aumentare l'efficienza e la produttività in una varietà di settori, come la ricerca, l'assistenza clienti e la gestione delle informazioni.
- *Scalabilità*: i modelli di LLM possono essere scalati facilmente per gestire grandi quantità di dati e compiti complessi. Questo li rende adatti per applicazioni che richiedono l'elaborazione di grandi quantità di testo, come l'analisi dei social media.
- *Facilità d'uso*: i modelli di LLM sono facili da utilizzare grazie ai comandi in linguaggio naturale. Questo li rende accessibili a una vasta gamma di utenti, anche a coloro che non hanno esperienza nel campo del Machine Learning.

- *Innovazione*: i modelli di LLM hanno il potenziale di rivoluzionare il modo in cui interagiamo con i computer e di aprire nuove opportunità per l'innovazione nel campo dell'Intelligenza Artificiale. Essi possono essere utilizzati per sviluppare nuove applicazioni e servizi che migliorano la nostra vita quotidiana e trasformano i settori esistenti.

Nonostante i numerosi vantaggi offerti dai Large Language Model, questi ultimi presentano anche alcuni svantaggi che possono limitarne l'adozione in determinati contesti. È importante considerare attentamente questi aspetti prima di utilizzare un modello di LLM per un compito specifico.

Tra i principali svantaggi dei Large Language Model troviamo:

- *Funzionamento non trasparente*: i modelli di LLM sono spesso descritti come "black-box" perché è difficile comprendere come essi forniscano una determinata risposta. Questo può rendere difficile interpretare i risultati e comprendere il ragionamento del modello per diagnosticare ed intervenire su eventuali errori. Inoltre, i modelli di LLM possono generare risposte errate o incoerenti, senza fornire spiegazioni sul perché una determinata risposta è stata generata. In applicazioni critiche, come la medicina, in caso di errori è fondamentale poter comprendere il malfunzionamento del sistema e avere una buona certezza che il modello non ripeta più lo stesso problema.

- *Risorse e impatto ambientale*: l'addestramento di modelli di LLM richiede grandi quantità di risorse computazionali, come GPU e TPU, che consumano molta energia e possono avere un impatto significativo sui costi e sull'ambiente. Questo può rendere difficile l'accesso ai modelli di LLM per le organizzazioni con risorse limitate.

- *Bias e discriminazione*: i modelli di LLM possono ereditare i bias e le discriminazioni presenti nei dati di addestramento, che possono influenzare le risposte generate dal modello. Ad esempio, un modello addestrato su dati testuali che contengono stereotipi di genere potrebbe generare risposte discriminatorie o sessiste. Questo può avere conseguenze negative in applicazioni sensibili come la selezione del personale (e quindi la valutazione automatica dei curriculum vitae) o la valutazione finanziaria per la cessione del credito bancario.

- *Dipendenza dai dati di addestramento*: i modelli di LLM sono altamente dipendenti dai dati di addestramento e possono generare risposte errate o incoerenti se esposti a dati di addestramento di bassa qualità o non rappresentativi. Ad esempio, un modello addestrato su dati testuali in lingua inglese potrebbe generare risposte errate se esposto a dati in una lingua differente. Questo può limitare l'utilità dei modelli di LLM in contesti multilingue o in applicazioni che richiedono una comprensione approfondita di un particolare dominio. Come in altri casi di Machine Learning, la qualità dei dati di addestramento è fondamentale per ottenere risultati accurati e affidabili. In base ai dati di addestramento si possono verificare fenomeni di:

  - *Overfitting*: il modello si adatta troppo ai dati di addestramento e non generalizza bene ai dati di test, non riuscendo così a catturare i pattern nei dati.
  - *Underfitting*: il modello non si adatta bene ai dati di addestramento e non riesce a catturare le relazioni presenti in essi. In altre parole, il modello è troppo semplice per catturare la complessità dei dati.

- *Privacy e sicurezza*: i modelli di LLM possono memorizzare informazioni sensibili presenti nei dati di addestramento e generare risposte che potrebbero compromettere la

privacy e la sicurezza delle persone coinvolte. Ad esempio, un modello addestrato su dati testuali contenenti informazioni personali potrebbe generare risposte che rivelano informazioni sensibili. Questo può avere conseguenze negative in applicazioni che richiedono la protezione dei dati personali, come la gestione delle informazioni sanitarie o finanziarie. Per questo motivo, l'Unione Europea ha recentemente introdotto, in aggiunta al GDPR (un regolamento che impone regole molto stringenti sulla protezione dei dati personali), l'AI Act ovvero un regolamento che impone regole molto stringenti sull'utilizzo di modelli di AI, in particolare di LLM.

- *Utilizzo improprio*: i modelli LLM possono essere utilizzati per scopi malevoli, come la generazione di contenuti falsi (deepfake), spam, disinformazione, o per automatizzare attacchi di Social Engineering. È importante monitorare attentamente l'utilizzo dei modelli di LLM e adottare misure per prevenire l'abuso e la diffusione di contenuti dannosi. Anche in questo caso, l'AI Act definisce e regolamenta l'utilizzo di questi modelli per prevenire gli abusi.

- *Allucinazioni*: i modelli di LLM possono generare risposte che non sono basate su fatti reali o che sono completamente inventate, ma che all'apparenza risultano corrette. Queste risposte, note come "allucinazioni", possono compromettere la credibilità del modello e generare confusione tra gli utenti. Principalmente, questo fenomeno è dovuto al fatto che gli LLM non hanno una comprensione del mondo reale, ma operano su correlazioni statistiche nei dati. Questo significa che possono generare testo plausibile ma non necessariamente corretto. È importante monitorare attentamente le risposte generate dai modelli di LLM e adottare misure per prevenire la diffusione di informazioni false o fuorvianti.

2.5 Applicazioni dei Large Language Model

Gli LLM, data la loro crescita esponenziale e la continua ricerca nel settore dell'Intelligenza Artificiale, stanno trovando un numero sempre crescente di applicazioni in diversi ambiti. La loro capacità di comprendere, generare e contestualizzare il linguaggio naturale li rende strumenti versatili in grado di automatizzare processi complessi, migliorare l'interazione uomo-macchina e fornire supporto in attività che richiedono la gestione di grandi quantità di testo.

Di seguito sono elencati alcuni dei principali casi d'uso degli LLM:

- *Traduzione automatica*: i modelli di LLM possono essere utilizzati per tradurre testi da una lingua all'altra in modo rapido e accurato. Questi modelli, quando addestrati su vasti dataset multilingue, sono in grado di cogliere le sfumature linguistiche e culturali, migliorando non solo la traduzione letterale, ma anche quella contestuale. Tali modelli possono essere, quindi, utilizzati per tradurre documenti, pagine web, messaggi di testo e molto altro.

- *Sentiment Analysis*: i modelli di LLM possono essere impiegati per l'analisi del sentiment in testi provenienti da fonti diverse, come recensioni di prodotti, commenti sui social media e articoli di notizie. Questi modelli possono classificare con precisione il tono e l'emozione espressi in un testo come positivi, negativi o neutri. Questa capacità è particolarmente utile per le aziende che desiderano monitorare la percezione del loro brand, analizzare feedback dei clienti in tempo reale e reagire rapidamente a tendenze emergenti.

- *Assistenti virtuali*: i modelli di LLM possono essere utilizzati per sviluppare assistenti virtuali intelligenti che vanno oltre le semplici risposte pre-programmate, comprendendo e contestualizzando le richieste degli utenti. Questi assistenti virtuali possono eseguire una vasta gamma di compiti, come rispondere a domande, fornire informazioni personalizzate, programmare appuntamenti e, persino, effettuare transazioni. Questi assistenti possono essere addestrati su dataset di conversazioni per generare risposte in base al contesto fornito. Un esempio di assistente virtuale è Gemini di Google, che è incorporato nei nuovi smartphone Pixel ed assiste l'utente nell'utilizzo del dispositivo.

- *Riassumere testi*: i modelli di LLM possono essere impiegati per generare riassunti concisi e accurati di testi lunghi, semplificando l'accesso rapido alle informazioni essenziali. Questi modelli, addestrati su grandi raccolte di testi e sui loro rispettivi riassunti, sono in grado di identificare le parti più rilevanti di un documento e di condensarle in una sintesi leggibile. Questa funzionalità è particolarmente utile in settori come l'editoria, la ricerca accademica e la gestione delle informazioni aziendali, dove la capacità di estrarre rapidamente i punti chiave da documenti complessi può accelerare i processi decisionali e migliorare l'efficienza.

- *Generazione di testi*: i modelli di LLM possono essere utilizzati per creare contenuti testuali originali e creativi, come poesie, racconti, dialoghi e canzoni. Addestrati su ampie collezioni di opere letterarie e testi creativi, questi modelli possono produrre nuove opere che imitano stili e generi diversi, stimolando l'innovazione nel campo della letteratura, del marketing e dell'intrattenimento. Questi strumenti sono particolarmente preziosi per scrittori, pubblicitari e creativi che cercano ispirazione o supporto nella generazione di idee e contenuti.

- *Assistenti per gli sviluppatori*: i modelli di LLM possono svolgere il ruolo di assistenti per gli sviluppatori, offrendo suggerimenti di codice, spiegazioni di concetti complessi e soluzioni a problemi di programmazione. Questi assistenti, addestrati su ampi dataset di codice sorgente, possono velocizzare il processo di sviluppo, ridurre gli errori e migliorare la produttività degli sviluppatori. Oltre a suggerire frammenti di codice, possono aiutare nella documentazione automatica e nel refactoring del codice, rendendo più agevole la manutenzione e l'aggiornamento dei progetti software. Un esempio di assistente per gli sviluppatori è GitHub Copilot, sviluppato da GitHub in collaborazione con OpenAI, che offre suggerimenti di codice basati su Codex, un modello di LLM derivato da GPT-3.

2.5.1 Retrieval-Augmented Generation

La Retrieval-Augmented Generation (RAG) è un approccio che combina un sistema di recupero delle informazioni con un modello di LLM. Poiché questi modelli vengono addestrati precedentemente su dei dati generici, essi possono non avere una conoscenza specifica del dominio o del contesto in cui vengono utilizzati. Per ovviare a questo problema, la RAG utilizza un sistema di recupero delle informazioni da fonti esterne per fornire, per ogni richiesta dell'utente, delle informazioni aggiuntive al modello di LLM sulla base delle quali generare risposte più accurate e pertinenti. Queste informazioni aggiuntive, che possono essere derivate da grandi raccolte di dati in più formati, solitamente vengono segmentate e indicizzate in un database vettoriale.

L'architettura di un sistema RAG è costituita da due componenti principali:

- *Modulo Retrieval*: è un sistema che, come un motore di ricerca, recupera informazioni rilevanti da una o più fonti esterne, come basi di dati, documenti o pagine web. Queste

informazioni vengono utilizzate per arricchire il contesto fornito al modello LLM e per guidarlo nella generazione di risposte accurate e pertinenti. Il sistema di recupero delle informazioni può utilizzare tecniche di ricerca basate su parole chiave o su semantiche per identificare le informazioni più rilevanti per una determinata richiesta. Solitamente, vengono utilizzati modelli di Embedding come Dense Passage Retrieval (DPR), che trasformano sia le domande che i documenti in vettori in uno spazio semantico comune. Utilizzando metriche di similarità, il sistema identifica i documenti più pertinenti che saranno utilizzati dal modulo di generazione.

- *Modulo di generazione*: è un modello di LLM che genera risposte in base al contesto fornito dal modulo Retrieval. Questo modello può essere addestrato in modo generico su un vasto dataset di testi per comprendere il linguaggio naturale e generare risposte coerenti e ben strutturate. Esso riceve in ingresso sia la query originale dell'utente che le informazioni di contesto aggiuntive. La risposta, fornita, quindi non solo riflette la capacità linguistica del modello, ma è anche arricchita dalle informazioni aggiornate e specifiche recuperate in tempo reale.

In Figura 2.6 è rappresentato uno schema di funzionamento di un modello di Retrieval-Augmented Generation. Esso è composto da cinque passaggi principali:

1. *Prompt + Query*: l'utente fornisce in ingresso un testo chiamato "prompt" insieme a una "query" che rappresenta la richiesta di informazioni aggiuntive necessarie per migliorare la risposta. Questo ingresso iniziale viene elaborato dal sistema per determinare quali informazioni siano necessarie per migliorare il contesto della risposta generata.
2. *Query*: la "query" viene inviata a un componente di ricerca delle informazioni rilevanti. Questo componente utilizza la query per cercare all'interno di una raccolta di dati aggiuntivi (Knowledge Source) come database di documenti, articoli e manuali, con lo scopo di trovare dati rilevanti che possano essere utilizzati per arricchire la risposta del modello.
3. *Informazioni rilevanti per un contesto aggiuntivo*: il componente di ricerca recupera le informazioni più pertinenti dalla raccolta dei dati aggiuntivi e le restituisce al sistema.
4. *Prompt + Query + contesto aggiuntivo*: le informazioni recuperate vengono combinate con il prompt originale e la query per creare un contesto arricchito (Enhanced Context). Esso rappresenta un ingresso più dettagliato e informato, che può migliorare la qualità e la precisione della risposta generata.
5. *Risposta testuale generata*: le informazioni precedenti vengono inviate ad un Large Language Model generico che può, quindi, generare una risposta testuale che riflette il contesto arricchito. Questa risposta è più accurata e pertinente rispetto a quella che sarebbe stata generata senza l'aggiunta delle informazioni aggiuntive.

Differenza tra RAG e addestramento

L'addestramento di un Large Language Model (LLM) è il processo iniziale in cui il modello impara dalle informazioni contenute da grandi quantità di testo in ingresso. Questo processo, che può durare settimane o mesi e richiede grandi risorse computazionali, mira a creare una comprensione generale del linguaggio. Al contrario, la RAG combina un modello pre-addestrato con un modulo di recupero di informazioni che cerca dati aggiornati da fonti esterne durante l'interazione in tempo reale. Questo approccio dinamico migliora la precisione e la pertinenza delle risposte, integrando informazioni aggiornate, anziché basarsi solo sui dati dell'addestramento iniziale.

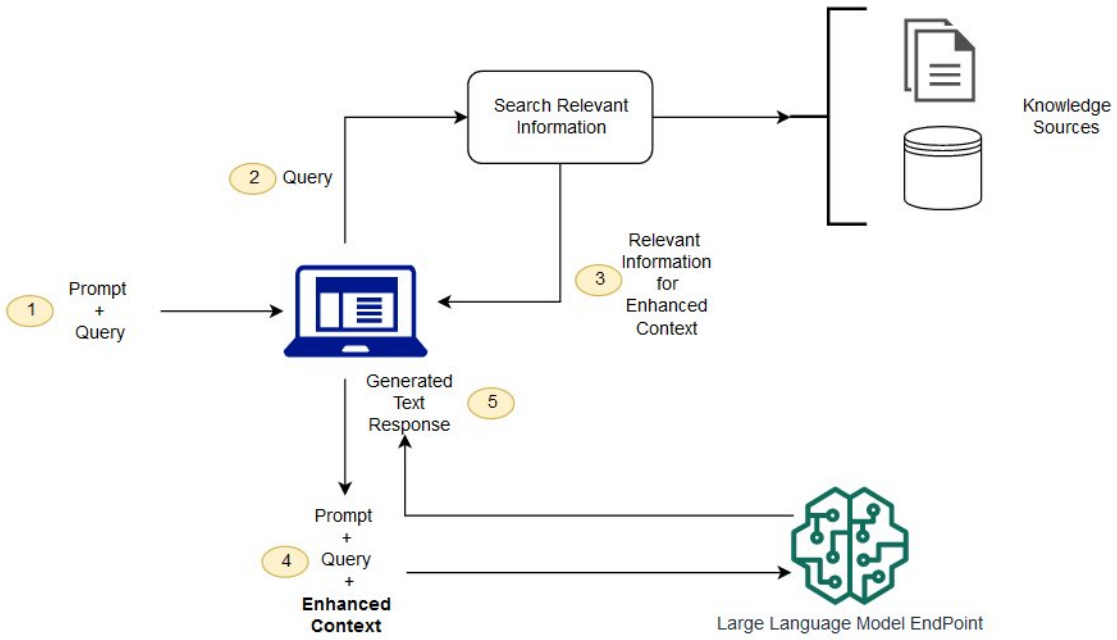


Figura 2.6: Schema di funzionamento di un modello di Retrieval-Augmented Generation

Differenza tra RAG e Fine-Tuning

Il Fine-Tuning è un passaggio successivo all'addestramento iniziale di un LLM, che consiste nell'addestrare ulteriormente il modello su un dataset specifico per ottimizzarlo su compiti particolari o domini specifici. Questo processo adatta il modello a esigenze specifiche, come rispondere a domande con un certo stile. La RAG, invece, che non richiede questo riaddestramento, arricchisce le risposte del modello pre-addestrato utilizzando informazioni recuperate in tempo reale da fonti esterne, rendendolo utile in contesti con informazioni che cambiano frequentemente o sono troppo ampie per essere catturate nel Fine-Tuning.

2.5.2 Architettura di un agente LLM

Gli agenti LLM rappresentano un'evoluzione dell'utilizzo dei Large Language Model estendendo le capacità oltre alla generazione di testo. Un agente LLM è progettato per agire in modo autonomo, utilizzando il linguaggio naturale come principale mezzo di comunicazione e interazione. Questi agenti combinano le capacità di comprensione, ragionamento e generazione del linguaggio naturale con l'esecuzione di azioni in un ambiente specifico; grazie a ciò essi possono eseguire compiti complessi che richiedono pianificazione e adattamento.

Come mostrato in Figura 2.7, un agente LLM interagisce con l'ambiente circostante attraverso un'interfaccia di comunicazione basata sul linguaggio naturale. Il modello LLM elabora delle istruzioni che l'agente trasforma in azioni da eseguire nell'ambiente. Queste azioni possono includere la ricerca di informazioni, la pianificazione di attività, la gestione di risorse e in generale l'esecuzione di codice. Il risultato di queste azioni viene restituito all'LLM che può quindi applicare un ragionamento sul processo e procedere con altre attività. L'agente utilizza una memoria per memorizzare informazioni rilevanti e un sistema di pianificazione per organizzare le azioni da eseguire.

Come illustrato in Figura 2.8, un agente LLM è costituito da diverse componenti che lavorano insieme per eseguire compiti complessi. Esse sono le seguenti:

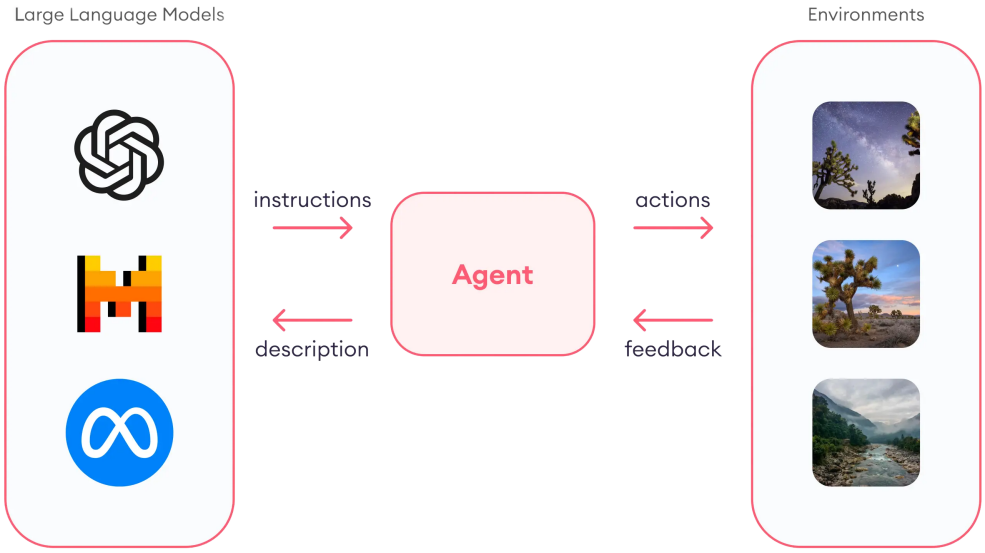


Figura 2.7: Meccanismo di interazione di un agente basato su Large Language Model con l'ambiente circostante

- *LLM*: questo rappresenta il cuore dell'agente ed è responsabile della comprensione del linguaggio naturale, della generazione di risposte e del ragionamento linguistico. Nella struttura di un agente, l'LLM non solo elabora l'ingresso fornito dall'utente, ma collabora anche con altre componenti per eseguire compiti complessi. Ad esempio, può utilizzare il contesto memorizzato o le informazioni recuperate tramite strumenti esterni per generare risposte più accurate.

- *Pianificazione*: questa è importante per determinare la sequenza di azioni che un agente deve compiere per raggiungere un obiettivo specifico. La pianificazione negli agenti LLM si divide in due fasi: formulazione del piano e riflessione del piano. Nella formulazione del piano, l'agente scompone compiti complessi in sotto-attività e crea un piano dettagliato da seguire passo dopo passo. La riflessione del piano, invece, permette agli agenti di rivedere e adattare le strategie basandosi su feedback continuo, sia interno che esterno. Questa componente interagisce strettamente con l'LLM per comprendere il contesto e definire le azioni da eseguire. Per esempio, se l'agente deve risolvere un problema matematico, il piano potrebbe prevedere la decomposizione del problema in compiti più piccoli, il calcolo di ciascuna parte, e la combinazione dei risultati per produrre la soluzione finale.

- *Memoria*: questa è una componente che consente agli agenti di gestire compiti complessi mantenendo traccia delle interazioni passate. Vi sono due tipi principali di memoria, ovvero a breve termine e a lungo termine, entrambe essenziali per fornire risposte contestuali e personalizzate. La memoria a breve termine rappresenta il "block notes", in cui vengono annotati rapidamente i dettagli rilevanti durante una conversazione. Questa memoria tiene traccia della discussione in corso, permettendo al modello di rispondere in modo pertinente e coerente con il contesto immediato. Tuttavia, è temporanea e viene svuotata una volta completato il compito o terminata la sessione. Questo tipo di memoria è fondamentale per gestire conversazioni fluide e per garantire la coerenza delle risposte all'interno di una singola interazione. D'altra parte, la memoria a lungo termine agisce come un diario che conserva informazioni significative e appro-

fondimenti raccolti nel corso di settimane o mesi. Non si limita a memorizzare dati, ma comprende anche la capacità di apprendere dai modelli delle interazioni passate. Questo tipo di memoria permette all'agente di ricordare le preferenze dell'utente e di utilizzare queste informazioni per migliorare le decisioni nelle interazioni future. Combinando memoria a breve termine e memoria a lungo termine, gli agenti LLM possono gestire efficacemente sia il contesto delle conversazioni attuali sia una ricca storia di interazioni passate.

- *Strumenti*: questi rappresentano delle risorse esterne che ampliano le capacità degli agenti LLM, consentendo loro di interagire con ambienti esterni per eseguire una vasta gamma di attività. Questi strumenti possono includere API, database, calcolatori matematici e molte altre risorse necessarie per rispondere a richieste specifiche o completare sotto-attività. L'utilizzo di strumenti permette agli agenti LLM di superare i limiti del solo modello di linguaggio e di agire in modo più efficace in contesti pratici. Per esempio, se l'utente chiede all'agente di trovare voli economici, l'agente potrebbe utilizzare un'API di ricerca voli per recuperare le informazioni necessarie e fornire una risposta dettagliata.

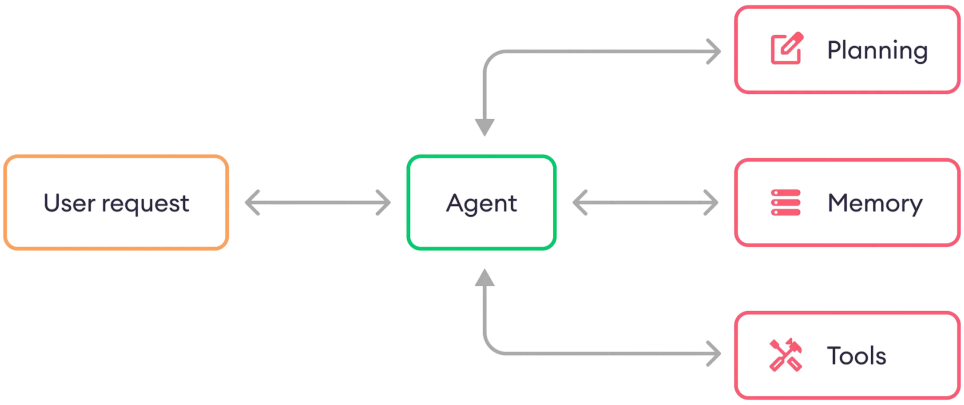


Figura 2.8: Struttura di un agente basato su Large Language Model

2.5.3 Architettura multi-agente

L'architettura LLM multi-agente prevede l'utilizzo di più agenti basati su Large Language Model che lavorano insieme per risolvere problemi complessi, gestire compiti su larga scala e interagire in ambienti dinamici. In questo contesto, ogni agente LLM ha competenze specifiche e può assumere ruoli diversi, collaborando e comunicando con altri agenti per raggiungere obiettivi comuni.

In un sistema multi-agente, gli LLM sono organizzati in una struttura in cui ogni agente può specializzarsi in un determinato compito o funzione. Alcuni agenti possono essere responsabili dell'elaborazione delle informazioni, mentre altri si occupano della generazione di risposte, della pianificazione, dell'uso di strumenti esterni o dell'interazione con gli utenti. La chiave dell'architettura multi-agente è il coordinamento efficace tra gli agenti. Tale coordinamento avviene tramite protocolli di comunicazione strutturati e meccanismi di condivisione delle informazioni.

Gli agenti LLM all'interno di un'architettura multi-agente devono comunicare e scambiare informazioni per completare i compiti in modo efficiente. Un esempio di scambio di informazioni è quando un agente specializzato nel recupero di dati raccoglie informazioni da

una fonte esterna e le condivide con un agente responsabile della generazione di risposte, che utilizza questi dati per produrre una risposta più accurata e contestualizzata.

Per garantire un'efficace collaborazione, gli agenti sono spesso coordinati tramite un sistema di ruoli e responsabilità. Ogni agente può avere un ruolo specifico in base alle sue competenze e capacità, e l'architettura può includere un agente coordinatore o un sistema centrale che assegna compiti e monitora i progressi. La pianificazione collettiva è un altro aspetto critico, dove gli agenti condividono i loro piani parziali e lavorano insieme per costruire una strategia complessiva che ottimizza il raggiungimento degli obiettivi del sistema.

Ad esempio, in Figura 2.9 è rappresentata una struttura gerarchica di un sistema multi-agente basato su LLM per la produzione di un articolo scientifico. In questo sistema, gli agenti sono organizzati in un'architettura gerarchica in cui gli agenti di livello inferiore eseguono compiti specifici e comunicano con l'agente di livello superiore per coordinare le attività e condividere le informazioni. L'agente di livello superiore può assumere ruoli di coordinamento e pianificazione, garantendo che gli agenti di livello inferiore lavorino insieme in modo efficace per raggiungere gli obiettivi del sistema.

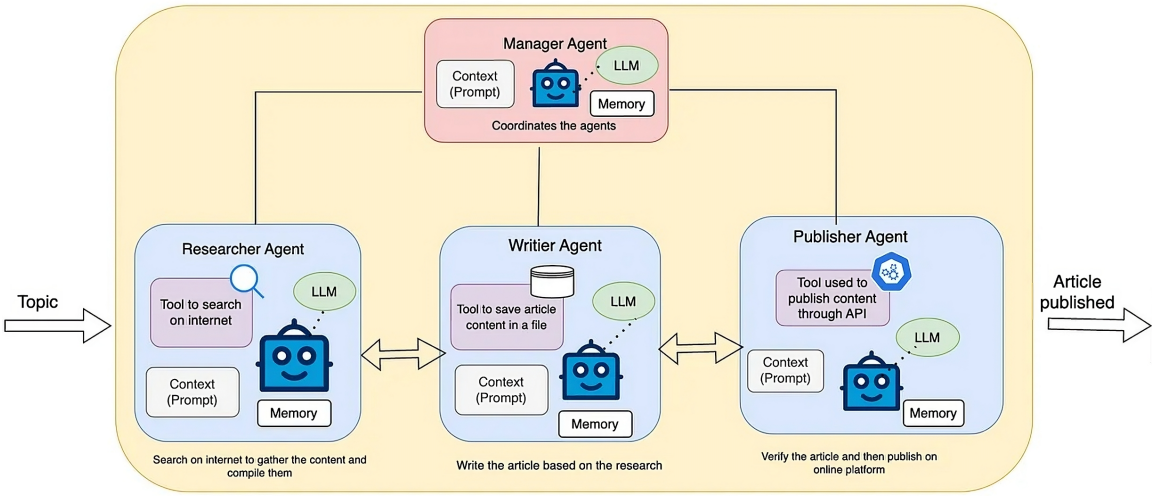


Figura 2.9: Struttura gerarchica di un sistema multi-agente basato su Large Language Model

L'architettura multi-agente offre vantaggi sostanziali; innanzitutto essa permette di suddividere compiti complessi tra agenti specializzati, migliorando l'efficienza e la scalabilità del sistema. Gli agenti possono lavorare in parallelo, aumentando la velocità di elaborazione e riducendo i tempi di risposta. Inoltre, questo approccio è più robusto in ambienti dinamici, poiché gli agenti possono adattarsi e riassegnare compiti in base ai cambiamenti del contesto.

Tuttavia, l'architettura multi-agente presenta anche sfide significative, come la necessità di un coordinamento efficace per evitare conflitti e ridondanze, e l'importanza di sviluppare protocolli di comunicazione chiari. Inoltre, la complessità di progettazione e implementazione aumenta con il numero di agenti, richiedendo sistemi di controllo più sofisticati per gestire il comportamento collettivo.

CAPITOLO 3

Definizione del contesto di riferimento e specifica dei requisiti

In questo capitolo viene descritto il contesto aziendale in cui si è svolto il progetto di tesi, illustrando brevemente l'azienda Blue Reply e il gruppo Reply di cui fa parte. Vengono, inoltre, definiti i requisiti funzionali e tecnici del progetto, delineando gli obiettivi principali che si intendono raggiungere con lo sviluppo della PoC per il chatbot per operatori nel settore assicurativo.

3.1 Contesto aziendale

Il presente progetto è stato sviluppato durante il periodo di lavoro presso l'azienda Blue Reply, una società di consulenza informatica facente parte del gruppo Reply. Blue Reply è specializzata in soluzioni di consulenza e servizi IT per il settore bancario e finanziario. In particolare, il progetto è stato svolto all'interno della divisione Blue Reply Financial Services, che si occupa di fornire servizi di consulenza e soluzioni IT per banche, assicurazioni e altri istituti finanziari. Il progetto consiste nella realizzazione di una PoC (Proof of Concept) per un chatbot per il settore assicurativo, con l'obiettivo di dimostrare le potenzialità di questa tecnologia e valutarne l'applicabilità. Il progetto ha previsto la definizione dei requisiti, lo sviluppo del chatbot, l'integrazione con i sistemi esistenti e la valutazione delle prestazioni.

3.1.1 Il gruppo Reply

Reply è una multinazionale che si occupa di consulenza, integrazione di sistemi e servizi digitali avanzati, supportando le aziende nella progettazione e implementazione di soluzioni basate sulle tecnologie più innovative. Fondata nel 1996, Reply si distingue per un modello organizzativo a rete, costituito da numerose aziende specializzate in settori specifici che collaborano per offrire competenze mirate e soluzioni di alto valore tecnologico. La struttura del gruppo consente a ciascuna divisione di concentrarsi su determinati mercati o tecnologie, garantendo, in questo modo, una forte specializzazione e una maggiore capacità di risposta alle esigenze dei clienti. Il logo del gruppo è riportato nella Figura 3.1.

Reply è quotata al segmento STAR della Borsa Italiana [EXM, STAR: REY]. L'azienda ha registrato una crescita costante: il fatturato è salito da 227,2 milioni di Euro nel 2007 a 2,11 miliardi di Euro nel 2023, accompagnato da un incremento dell'organico da 2.272 a 14.798 dipendenti nello stesso periodo.

Il gruppo è attivo in Europa con un numero sempre maggiore di sedi: ne conta 12 in Italia, 15 in Germania e 4 nel Regno Unito; è presente, seppur con un numero ridotto di sedi, anche

Figura 3.1: Logo del gruppo Reply

in Polonia, Croazia, Svizzera, Romania, Austria e Francia. Reply opera inoltre negli Stati Uniti, in Brasile, in Cina, in India e a Singapore.

Le aree di competenza dell'azienda includono l'Intelligenza Artificiale, i Big Data, il Cloud Computing, l'Internet of Things (IoT) e i media digitali. Attraverso queste tecnologie Reply affianca aziende in settori strategici, come telecomunicazioni, media, industria, servizi pubblici, energia e, in particolare, il settore finanziario. Grazie alla propria esperienza e alle competenze dei suoi team, Reply è in grado di supportare le aziende nella trasformazione digitale, aiutandole a sfruttare appieno le opportunità offerte dalle nuove tecnologie e a rimanere competitive in un mercato in continua evoluzione. In Figura 3.2 sono riportati i principali clienti del gruppo.

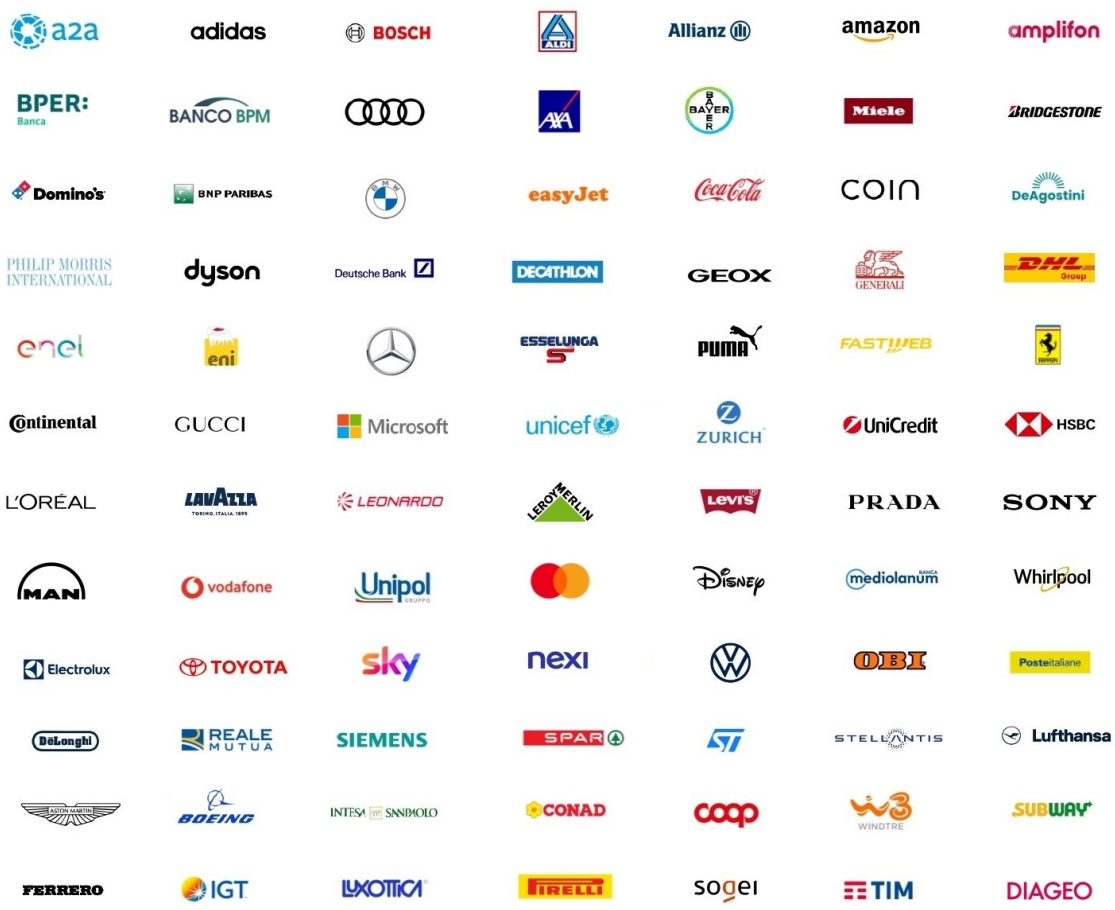


Figura 3.2: I principali clienti del gruppo Reply

Reply è fortemente impegnata nel processo di trasformazione digitale delle istituzioni finanziarie europee. Collabora con alcuni dei principali attori del settore nella definizione di

strategie multicanale, che comprendono l'intero percorso del cliente (Customer Journey) e il suo coinvolgimento (Customer Engagement). Questo approccio include attività che spaziano dal digital branding allo sviluppo di strategie per applicazioni mobili, dalla creazione di una nuova generazione di portali e punti di contatto multicanale, fino alla completa ristrutturazione delle architetture tecnologiche a supporto.

La "Financial Institution" è, insieme a Telco & Media, il settore più rilevante per Reply in termini di fatturato e dipendenti coinvolti. Tra i clienti serviti è possibile annoverare le principali banche e compagnie assicurative europee, oltre a reti di promotori e consorzi. Durante l'ultimo triennio, in particolare, Reply ha condotto diversi progetti di innovazione in ambito digitale, sia con metodologie tradizionali (seguendo end-to-end tutte le fasi, dal disegno della strategia all'analisi, all'implementazione e al lancio, oppure gestendo singole fasi della catena in collaborazione con attori terzi), sia con metodologie "agile".

Reply è partner di alcuni dei principali leader che guidano l'innovazione nel loro campo, come AWS, Google Cloud, Microsoft, Oracle, Nvidia, Intel, Qlik, Tableau, IBM, Red Hat, Pega e tanti altri.

3.1.2 L'azienda Blue Reply

Tra le numerose società del gruppo, Blue Reply rappresenta uno dei punti di forza nel settore finanziario. Questa divisione è specializzata nella consulenza e nei servizi IT per istituzioni bancarie, assicurative e finanziarie. Blue Reply supporta le organizzazioni del settore nella digitalizzazione dei loro processi, offrendo soluzioni tecnologiche che spaziano dalla progettazione di piattaforme digitali avanzate all'integrazione di sistemi complessi. Grazie alla propria esperienza consolidata e al continuo impegno nel campo dell'innovazione l'azienda si consolida come partner di riferimento per chiunque voglia affrontare con successo le sfide della trasformazione digitale.

In questo contesto, Blue Reply ha contribuito allo sviluppo di applicazioni per la gestione della firma digitale, di sistemi di pagamento, di segnalazione sinistri e di monitoraggio dei comportamenti degli utenti. Ha, inoltre, implementato portali web per la gestione di servizi assicurativi e bancari, sfruttando tecnologie di Machine Learning e Intelligenza Artificiale per ottimizzare l'esperienza utente.

La solidità di Blue Reply all'interno del gruppo Reply è evidenziata dalla sua continua crescita negli ultimi 15 anni, consolidando la propria presenza sia sul mercato italiano che su quello internazionale e diventando, così, una delle più grandi aziende del gruppo.

3.2 Definizione dei requisiti del progetto

3.2.1 Obiettivi funzionali del progetto

Il progetto ha come obiettivo la realizzazione di una PoC (Proof of Concept) per un chatbot nel settore assicurativo, in grado di rispondere a domande relative a polizze, coperture, premi e servizi offerti dall'azienda. Il sistema sarà integrato con i sistemi esistenti, in particolare con quello di gestione delle polizze. La PoC mira a dimostrare le potenzialità di questa tecnologia e a valutarne l'applicabilità. In particolare, essa dovrà essere in grado di:

- fornire assistenza riguardo all'utilizzo del portale clienti a partire da un manuale utente;
- gestire l'adeguato livello di protezione dei dati sensibili degli utenti;
- integrarsi con il database aziendale per analizzarne i dati;
- esportare i dati per essere visualizzati su Microsoft Excel (`.xlsx`);

- integrarsi con un calendario aziendale per controllare la disponibilità e fissare appuntamenti;
- inviare email a clienti e colleghi;
- fornire un'interfaccia utente semplice e intuitiva per l'utilizzo dell'assistente;
- mantenere un approccio modulare e scalabile per facilitare futuri sviluppi e integrazioni;
- implementare un'architettura che favorisca il controllo e l'orchestrazione dei vari componenti del sistema;
- richiedere, in modo deterministico, l'approvazione da parte dell'operatore, in caso di azioni critiche, come l'esecuzione di interrogazioni SQL sul database aziendale, l'invio di email e la creazione di appuntamenti sul calendario;
- fornire un sistema di monitoraggio di ogni conversazione per l'analisi delle interazioni con gli utenti.

3.2.2 Obiettivi tecnici del progetto

Dal punto di vista tecnico, per soddisfare i requisiti funzionali e garantire un'architettura moderna e manutenibile, sono stati definiti i seguenti obiettivi specifici:

- *Modularità e astrazione del modello (Model Agnosticism):* progettazione di un'architettura disaccoppiata che permetta la sostituzione rapida dei singoli agenti e l'intercambiabilità dei Large Language Model (LLM). Questo approccio garantisce la flessibilità necessaria per integrare nuovi modelli più performanti o economici senza richiedere modifiche strutturali al codice sorgente.
- *Adozione di un'architettura a grafo:* utilizzo del framework LangGraph per la gestione del flusso di conversazione e l'orchestrazione delle componenti. Questo permette di implementare cicli, ramificazioni e una gestione avanzata dello stato.
- *Implementazione del pattern Supervisor:* organizzazione dell'architettura multi-agente basata sul pattern "Supervisor", che prevede un'entità centrale di coordinamento incaricata di orchestrare il flusso di lavoro, delegando l'esecuzione delle attività ad agenti specializzati selezionati dinamicamente in funzione delle specifiche richieste dell'utente.
- *Persistenza e gestione dello stato:* utilizzo di un sistema di checkpointing (basato su database) per preservare il contesto tra le interazioni, consentendo il mantenimento di dialoghi a lungo termine e funzionalità di "time-travel" per il debugging.
- *Integrazione di servizi esterni tramite Tool:* sviluppo di strumenti personalizzati (Custom Tool) per l'interazione sicura con API di terze parti (Google Calendar, Gmail) e per la manipolazione di file locali (generazione di report Excel).
- *Prompt Engineering modulare:* centralizzazione e separazione delle istruzioni fornite ai modelli linguistici (System Prompt) in moduli dedicati, facilitando l'iterazione e l'ottimizzazione delle risposte degli agenti senza impattare sulla logica applicativa.

CAPITOLO 4

Progettazione degli agenti

In questo capitolo viene illustrata la progettazione del sistema multi-agente, descrivendo le tecnologie selezionate e le scelte architetturali adottate per soddisfare i requisiti definiti nel Capitolo 3. Verranno introdotti il framework LangChain, il sistema di orchestrazione LangGraph e la piattaforma di monitoraggio LangSmith, analizzando le motivazioni di queste scelte tecnologiche. Successivamente, verrà descritta l'architettura complessiva del sistema, basata sul modello Supervisor, approfondendo i meccanismi di comunicazione e le caratteristiche di ciascun agente specializzato che compone il team di lavoro.

4.1 Premessa

Partendo dai requisiti funzionali e tecnici definiti nel capitolo precedente, la progettazione del sistema multi-agente si concentra sulla realizzazione di un'architettura modulare, estensibile e facilmente manutenibile. La scelta di adottare un approccio multi-agente deriva dalla necessità di gestire diverse tipologie di richieste, ciascuna con caratteristiche e complessità specifiche, quali l'interrogazione del database, la consultazione della documentazione, la gestione del calendario e l'invio di comunicazioni via email. Per ogni richiesta è fondamentale individuare il miglior compromesso tra un controllo del flusso e un'autonomia decisionale demandata al modello LLM. Questi due aspetti sono inversamente proporzionali: un controllo più rigido del flusso di elaborazione limita la capacità del modello di adattarsi a richieste complesse o non previste mentre un'eccessiva autonomia può portare a comportamenti imprevedibili o non conformi ai requisiti aziendali. La progettazione mira, quindi, a bilanciare questi aspetti, garantendo, al contempo, flessibilità e affidabilità. Tali aspetti sono rappresentati graficamente nella Figura 4.1.

4.2 Tecnologie utilizzate

La realizzazione del sistema ha richiesto la selezione di tecnologie che consentissero di implementare un'architettura multi-agente stabile per lo sviluppo in ambiente di produzione. Sul mercato sono presenti diverse soluzioni enterprise e framework open-source affermati come CrewAI, Microsoft AutoGen e Google ADK (Agent Development Kit).

Per questo progetto sono state selezionate tre componenti tecnologiche principali: il framework LangChain per la costruzione degli agenti, il sistema LangGraph per l'orchestrazione dei flussi di lavoro e la piattaforma LangSmith per il monitoraggio e l'osservabilità. Questa

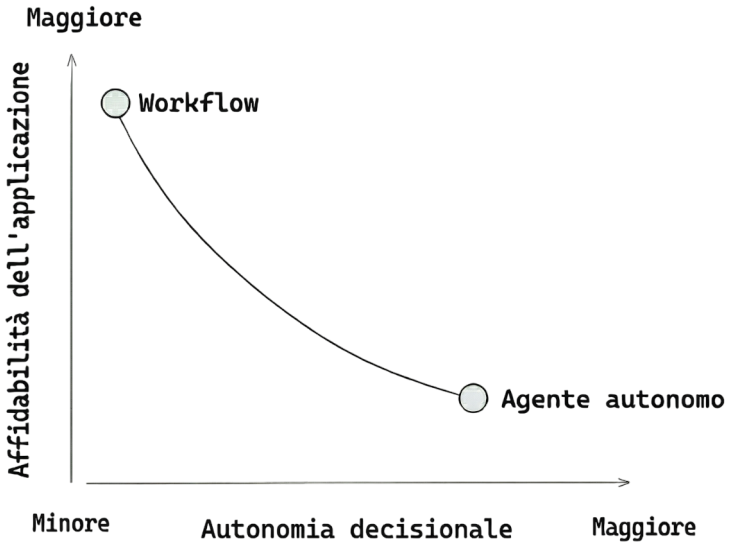


Figura 4.1: Bilanciamento tra controllo del flusso e autonomia decisionale

scelta deriva dalla coerenza dell'ecosistema, dalla maturità delle soluzioni e dalla compatibilità con i requisiti definiti nel capitolo precedente. Queste tecnologie sono sviluppate dalla società LangChain Inc. e, grazie anche al contributo di una forte community, rappresentano lo stato dell'arte nel panorama degli strumenti per lo sviluppo di sistemi che utilizzano gli LLM.

4.2.1 Il framework LangChain

LangChain è un framework open-source progettato per semplificare lo sviluppo di applicazioni basate su Large Language Model, come chatbot e agenti AI. Il progetto è stato lanciato nell'ottobre 2022 da Harrison Chase e Ankush Gola e ha acquisito subito popolarità: a giugno 2023 era il progetto open-source con la crescita più rapida su GitHub. La sua caratteristica principale è la capacità di fornire un'interfaccia standardizzata per l'interazione con diversi fornitori di modelli linguistici, permettendo agli sviluppatori di costruire applicazioni quanto più indipendenti possibile dal modello sottostante.

Il framework si articola in diversi componenti chiave che collaborano per offrire un'infrastruttura completa.

Interfaccia standardizzata per i modelli

Uno dei principali vantaggi offerti da LangChain consiste nell'astrazione delle differenze tra le API dei vari fornitori di modelli linguistici. Ciascun fornitore presenta, infatti, interfacce proprietarie con formati di richiesta e risposta differenti. LangChain risolve questa eterogeneità mettendo a disposizione un'interfaccia unificata che permette di interagire con modelli di OpenAI, Anthropic, Google e altri fornitori attraverso le medesime chiamate di metodo. Tale caratteristica risulta particolarmente rilevante in questo contesto in rapida evoluzione, consentendo agli sviluppatori di mantenere il sistema sempre aggiornato con i modelli più recenti e performanti senza impattare la logica applicativa.

Gestione dei tool

LangChain supporta il concetto di tool, ovvero funzioni che gli agenti possono invocare per interagire con sistemi esterni ed eseguire operazioni che il modello linguistico non potrebbe compiere autonomamente. Ogni tool viene definito specificando il nome, la descrizione e lo schema dei parametri attesi. Il framework gestisce automaticamente la conversione delle chiamate nel formato richiesto dal modello linguistico e offre molte integrazioni con librerie di strumenti esterne. Nel sistema in esame, i tool costituiscono il meccanismo attraverso cui gli agenti comunicano con il database aziendale, le API di Google Calendar, il servizio Gmail e il sistema di gestione documentale.

Gestione dei messaggi

I messaggi rappresentano l'unità fondamentale per la gestione del contesto nelle applicazioni basate su LLM. LangChain definisce un formato standardizzato per i messaggi che include il ruolo del mittente, il contenuto e i metadati opzionali. I ruoli principali comprendono i messaggi di sistema, che definiscono il comportamento dell'assistente, i messaggi utente, le invocazioni dei tool e le risposte del modello. Anche questo aspetto è gestito dal framework in modo uniforme per permettere l'integrazione indipendente dal fornitore del modello.

Astrazione degli agenti

LangChain fornisce astrazioni di alto livello per la creazione di agenti. Il framework consente di specificare il modello linguistico, la lista dei tool disponibili, il prompt di sistema e altre configurazioni opzionali. Per configurazione predefinita, gli agenti su LangChain seguono il pattern ReAct ("Reasoning + Acting"), alternando brevi fasi di ragionamento con chiamate di tool e integrando le osservazioni risultanti nelle decisioni successive finché non riescono a fornire una risposta definitiva. L'agente così definito, la cui architettura è rappresentata in Figura 4.2, è in grado di ricevere messaggi in linguaggio naturale, ragionare sulla richiesta, selezionare i tool appropriati ed eseguirli in sequenza fino al completamento del compito. Questa astrazione semplifica notevolmente lo sviluppo, consentendo di definire agenti complessi in modo dichiarativo.

4.2.2 L'orchestrazione tramite LangGraph

Mentre LangChain fornisce le primitive per la costruzione di singoli agenti, LangGraph si occupa dell'orchestrazione di flussi di lavoro complessi che coinvolgono molteplici componenti. Questo è un framework di basso livello progettato specificamente per la gestione di processi basati su agenti che richiedono un controllo del flusso di esecuzione, della gestione dello stato e dell'integrazione di interventi umani. Riprendendo il grafico illustrato precedentemente in Figura 4.1, LangGraph si pone l'obiettivo di migliorare il compromesso tra affidabilità e autonomia del sistema, come mostrato in Figura 4.3.

L'architettura di LangGraph si basa sulla modellazione dei processi come grafi orientati, dove i nodi rappresentano le unità di elaborazione e gli archi definiscono il flusso di controllo tra di essi. Questo approccio consente di rappresentare flussi di lavoro che includono ramificazioni condizionali, cicli e parallelismo, superando i limiti delle architetture puramente sequenziali. La Figura 4.4 illustra alcuni esempi di pattern di flussi che possono essere modellati con LangGraph.

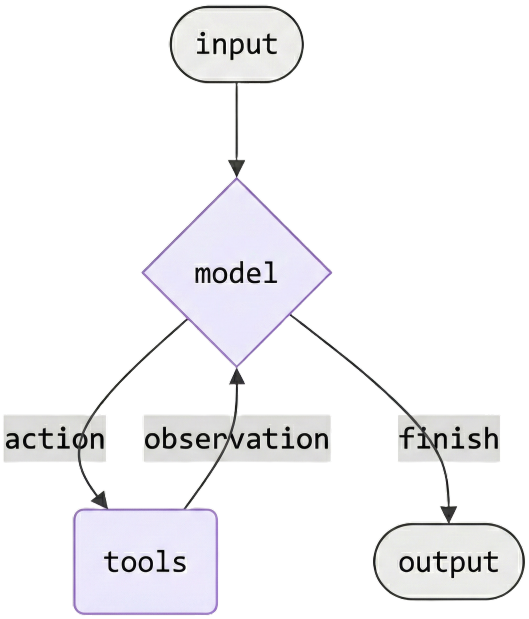


Figura 4.2: Architettura di un agente ReAct in LangChain

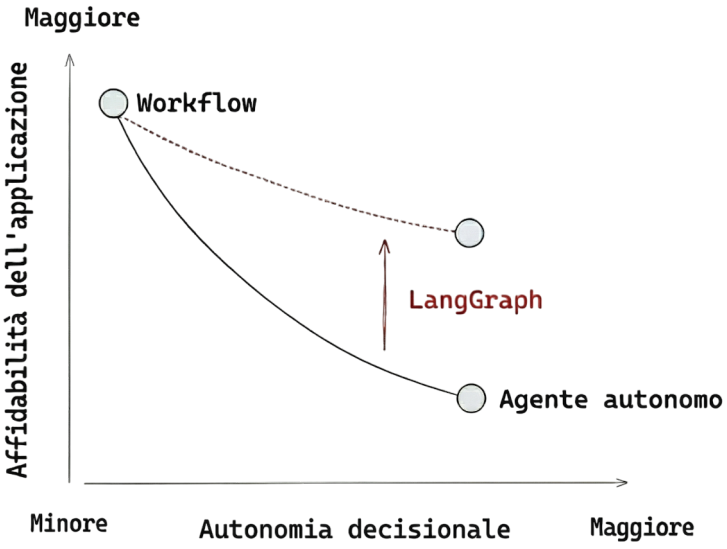


Figura 4.3: Compromesso tra il controllo del flusso e autonomia decisionale in LangGraph

Modello a grafo

In LangGraph un'applicazione viene modellata come un grafo di stato che può operare su un contesto ben definito. I nodi del grafo sono unità di elaborazione che ricevono lo stato corrente e lo aggiornano. Essi possono essere implementati come funzioni sincrone o asincrone e possono essere eseguiti in parallelo quando non esistono dipendenze tra di essi, consentendo di ridurre la latenza complessiva. Gli archi connettono i nodi e possono essere di due tipologie: archi semplici, che definiscono transizioni fisse verso un nodo prestabilito, e archi condizionali, che selezionano il nodo successivo in base a una funzione di instradamento applicata allo stato corrente.

Il framework prevede due nodi speciali, ovvero il nodo di ingresso, che rappresenta il

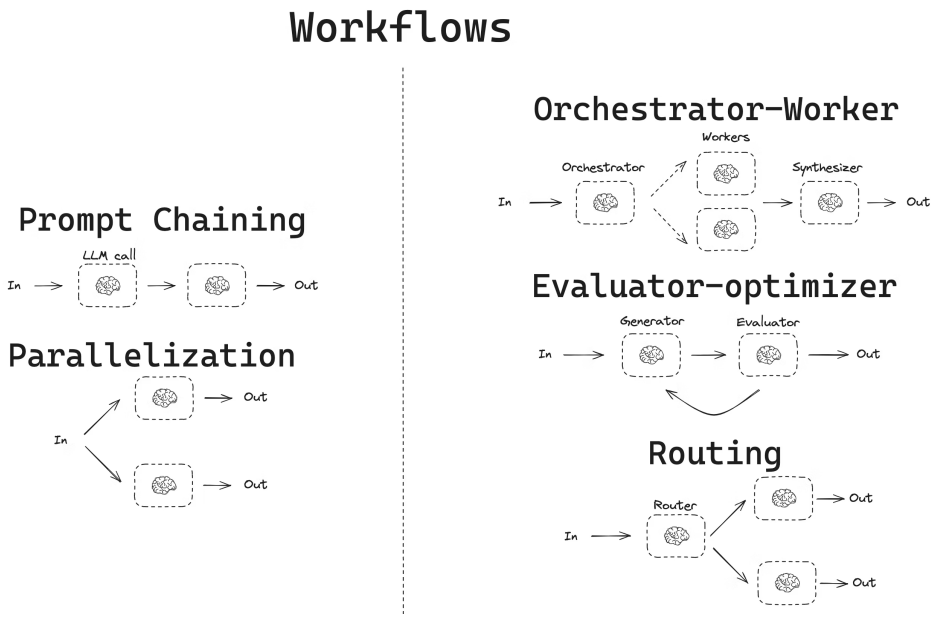


Figura 4.4: Esempi di pattern di flussi in LangGraph

punto di partenza del grafo, e il nodo di uscita, che indica la terminazione dell'esecuzione. Ogni grafo, per essere valido, deve specificare almeno un arco dal nodo di ingresso verso il primo nodo applicativo, mentre l'esecuzione deve terminare con il raggiungimento del nodo di uscita.

Gestione dello stato

La gestione dello stato rappresenta uno degli aspetti distintivi di LangGraph. Lo stato del grafo viene definito attraverso uno schema ben strutturato che specifica quali valori sono ammessi. Ogni nodo può leggere lo stato corrente e restituire aggiornamenti parziali; il framework provvede ad applicare tali aggiornamenti secondo regole configurabili mediante apposite funzioni di aggregazione. Tali funzioni determinano come combinare gli aggiornamenti restituiti dai nodi con lo stato esistente. Per impostazione predefinita, un aggiornamento sovrascrive il valore precedente della chiave corrispondente; tuttavia, è possibile definire logiche personalizzate per implementare comportamenti di concatenazione, come la formazione di code di messaggi in una lista. LangGraph mette a disposizione funzioni predefinite per i casi d'uso più comuni, tra cui la gestione delle conversazioni basate su messaggi.

Sistema di persistenza

LangGraph implementa un livello di persistenza nativo attraverso il meccanismo dei checkpointer. Quando un grafo viene configurato con un checkpointer, lo stato viene salvato automaticamente al termine di ogni esecuzione. I checkpoint sono poi organizzati in sessioni, che rappresentano esecuzioni indipendenti con un identificatore univoco.

Questa architettura abilita diverse funzionalità avanzate: la persistenza dello stato consente di riprendere conversazioni interrotte e di mantenere il contesto tra sessioni successive, mentre la funzionalità di navigazione temporale permette di ispezionare stati precedenti e di riavviare l'esecuzione da un checkpoint specifico, risultando particolarmente utile in fase di analisi degli errori. Le implementazioni dei checkpointer disponibili includono configurazio-

ni in memoria per lo sviluppo e diverse librerie aggiuntive basate su database relazionali per ambienti di produzione.

Meccanismo di Human-in-the-Loop

Il concetto di Human-in-the-Loop, traducibile letteralmente come "umano nel ciclo", indica un paradigma di progettazione in cui il sistema automatizzato prevede punti di intervento in cui è richiesta la partecipazione attiva di un operatore umano. Tale approccio è fondamentale nei sistemi basati su LLM, dove la supervisione umana consente di validare le decisioni automatiche, correggere eventuali errori e mantenere il controllo su operazioni sensibili.

In LangGraph è previsto un supporto nativo per questo meccanismo attraverso un componente di intercettazione che consente di configurare quali tool richiedano approvazione umana. Quando un agente propone l'invocazione di un tool configurato per l'interruzione, tale componente sospende automaticamente l'esecuzione e presenta all'operatore tre opzioni: approvare l'azione, modificarla prima dell'esecuzione oppure rifiutarla fornendo un feedback.

Questo meccanismo risulta fondamentale per il sistema in esame, poiché consente di implementare in maniera deterministica il requisito di approvazione esplicita per le operazioni critiche, quali l'esecuzione di query SQL, l'invio di email e la creazione di eventi nel calendario.

Supporto per i sottografi

LangGraph permette di comporre grafi complessi attraverso l'annidamento di sottografi. Un sottografo è, a sua volta, un grafo compilato che può essere utilizzato come nodo all'interno di un grafo padre. Questa caratteristica favorisce la modularità e il riutilizzo del codice, consentendo di sviluppare e testare i singoli componenti in isolamento prima di integrarli nell'architettura complessiva.

Nel sistema in esame, tale funzionalità è utile per implementare e testare i vari agenti specializzati come sottografi autonomi, incapsulando la logica del componente separata dal grafo principale del supervisore.

4.2.3 La piattaforma LangSmith

La scelta di integrare LangSmith fin dalle prime fasi di progettazione deriva dalla necessità di disporre di uno strumento nativamente integrato con l'ecosistema LangChain e LangGraph, garantendo trasparenza e tracciabilità del comportamento del sistema. Sebbene il sistema possa integrarsi anche con altri framework di osservabilità esterni, la piattaforma LangSmith rappresenta l'opzione ottimale in quanto è stata sviluppata dalla stessa società LangChain Inc., specificatamente per applicazioni LLM. Lo strumento risulta, comunque, indipendente dal framework utilizzato; può pertanto essere impiegato anche senza le librerie open-source di LangChain.

La Figura 4.5 mostra l'interfaccia utente della piattaforma, che consente di visualizzare le esecuzioni, ispezionare i dettagli delle operazioni e raccogliere feedback.

L'architettura concettuale di LangSmith si fonda su alcune primitive fondamentali. Il concetto di *run* rappresenta l'unità atomica di lavoro, corrispondente a una singola operazione eseguita dall'applicazione, quale una chiamata a un modello linguistico, l'invocazione di un tool o l'elaborazione di un prompt. Il concetto di *trace* aggrega, invece, l'insieme delle *run* relative a una singola richiesta utente, permettendo di visualizzare l'intera sequenza di operazioni dal ricevimento dell'input alla produzione dell'output finale. Per applicazioni

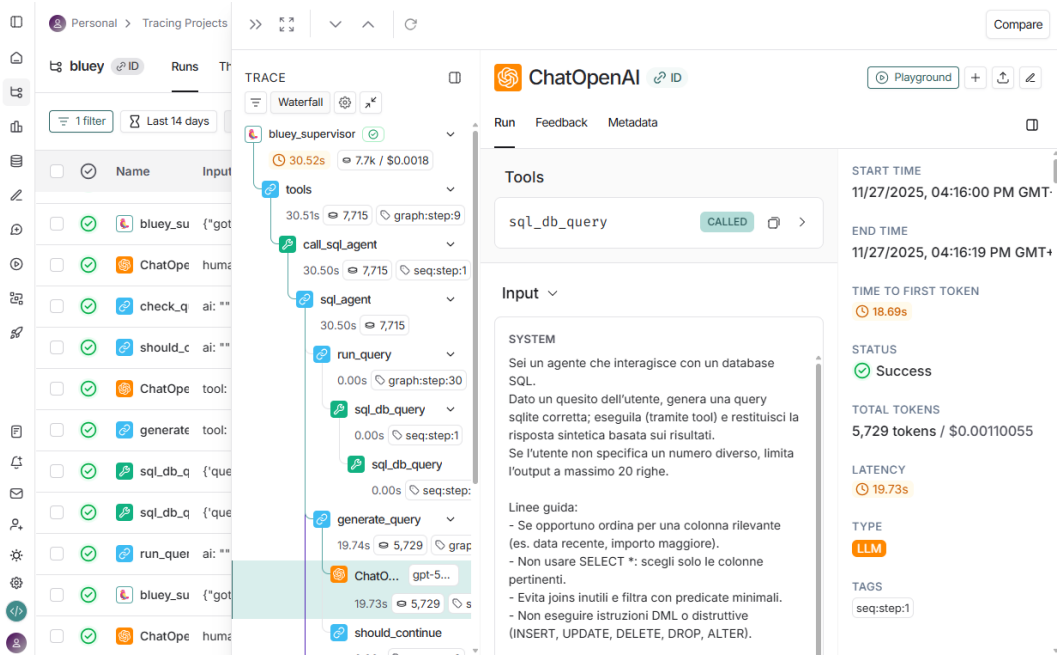


Figura 4.5: Interfaccia utente di LangSmith

conversazionali, LangSmith introduce, inoltre, il concetto di *thread*, che collega tra loro le tracce appartenenti alla medesima conversazione multi-turno.

Nel sistema in esame, il tracciamento automatico delle esecuzioni permette di ispezionare nel dettaglio ogni passaggio effettuato dagli agenti, visualizzando gli input ricevuti, le decisioni prese, i tool invocati e gli output prodotti. Tale visibilità è fondamentale in fase di debug, quando occorre identificare le cause di comportamenti inattesi o risposte non corrette.

LangSmith integra, inoltre, un sistema di feedback nativo che consente di associare valutazioni alle varie tracce, sia in forma automatica ,attraverso valutatori online, sia manualmente, attraverso annotazioni degli operatori. La piattaforma monitora anche le metriche di latenza e utilizzo delle risorse LLM, fornendo un quadro completo delle prestazioni e dei costi del sistema. Queste funzionalità supportano il processo di miglioramento continuo, permettendo di identificare sistematicamente colli di bottiglia e le aree in cui sono necessarie delle ottimizzazioni.

4.3 Architettura del sistema

Il sistema, denominato *Bluey* in riferimento all'identità aziendale di Blue Reply, si fonda sul paradigma multi-agente, nel quale un insieme di agenti autonomi collabora per raggiungere obiettivi comuni. Tale approccio risulta particolarmente adatto al contesto applicativo descritto, poiché consente di suddividere la complessità del dominio in componenti specializzati, ciascuno responsabile di una specifica area funzionale.

Fra le diverse architetture multi-agente disponibili, in questo progetto è stata adottata una struttura gerarchica denominata "Supervisor", nella quale un agente centrale coordina le attività degli agenti subordinati. Questa scelta architetturale è motivata dalla necessità di disporre di un unico punto di accesso che gestisca le richieste dell'utente in modo naturale e conversazionale, mantenendo, al contempo, la visione complessiva del contesto per supervisionare e coordinare gli agenti specializzati. Dal punto di vista dell'utente, l'interazione avviene con un singolo assistente virtuale: la complessità dell'architettura interna e le even-

tuali iterazioni multiple tra gli agenti rimangono completamente nascoste. In Figura 4.6 viene illustrato il diagramma ad alto livello dell'architettura Supervisor.

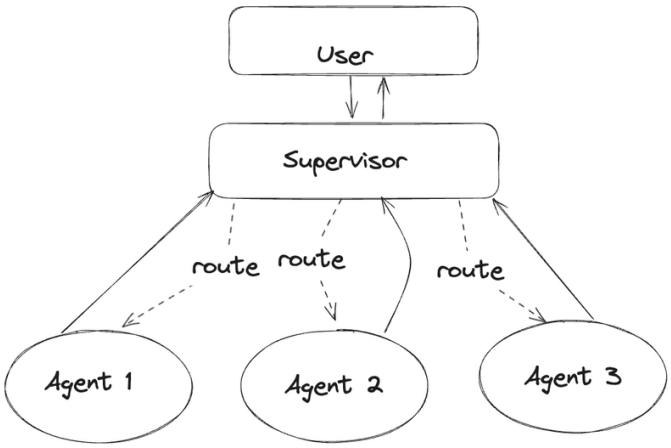


Figura 4.6: Architettura del modello multi-agente Supervisor

Rispetto ad alternative, quali la comunicazione peer-to-peer tra agenti o le architetture completamente decentralizzate, il modello Supervisor presenta diversi vantaggi e svantaggi nel contesto di un sistema di supporto operativo.

In questo caso, i principali vantaggi del pattern Supervisor sono i seguenti:

- *Partizionamento dei tool*: l'architettura consente di suddividere i tool disponibili tra i diversi agenti specializzati, ciascuno dotato di istruzioni e prompt specifici. Un agente con accesso diretto a numerosi tool dovrebbe scegliere tra opzioni simili, comprendere i formati esatti per ciascuna di esse e gestire simultaneamente domini multipli. Con un'architettura multi-agente Supervisor, il problema viene scomposto in due agenti separati: il Supervisor, che si occupa di comprendere l'intento dell'utente e di selezionare l'agente più appropriato, e l'agente specializzato, che gestisce e comprende meglio un dominio specifico con un set limitato di tool.

- *Controllo centralizzato e gestione del contesto*: l'esistenza di un punto di coordinamento centralizzato rende il comportamento del sistema più prevedibile e facilita l'analisi degli errori. L'operatore interagisce esclusivamente con il Supervisor, che si occupa di instradare le richieste e aggregare le risposte, presentando un'interfaccia unificata indipendentemente dalla complessità interna. Il Supervisor mantiene, inoltre, la visione complessiva della conversazione e può sfruttare tale contesto per formulare richieste più precise agli agenti specializzati.

- *Flessibilità nella scelta dei modelli*: l'architettura multi-agente consente di associare a ciascun agente specializzato un LLM differente, ottimizzato per il compito specifico. Ad esempio, l'agente che si occupa dell'accesso al database può utilizzare un modello specializzato nella generazione di codice SQL, mentre l'agente che si occupa della documentazione può impiegare un modello ottimizzato per la comprensione semantica dei testi. Tale flessibilità permette, inoltre, di bilanciare prestazioni e costi, assegnando modelli più leggeri e veloci agli agenti che gestiscono compiti semplici con finestre di contesto ridotte, e riservando modelli più potenti alle attività complesse. Inoltre, in contesti con requisiti stringenti di riservatezza, alcuni agenti possono essere configurati per utilizzare modelli eseguiti localmente o su cloud privati, evitando la trasmissione di dati sensibili verso servizi esterni.

- *Esecuzione parallela degli agenti*: l'architettura multi-agente Supervisor consente l'esecuzione simultanea di più agenti quando le rispettive elaborazioni risultano indipendenti tra loro. LangGraph supporta nativamente il parallelismo attraverso la definizione di archi multipli uscenti da un nodo, permettendo di avviare contemporaneamente diverse operazioni e di raccogliere i risultati al termine di tutte le esecuzioni. Questa caratteristica consente di ridurre significativamente i tempi di latenza complessivi in scenari specifici.
- *Resilienza e gestione degli errori*: l'architettura Supervisor consente di implementare strategie differenziate per la gestione delle anomalie. Il Supervisor può intercettare gli errori generati dagli agenti specializzati e applicare logiche di recupero appropriate, quali tentativi di ripetizione per errori transitori, richieste di chiarimento all'utente per informazioni mancanti, oppure deleghe ad agenti alternativi quando il primo non è in grado di soddisfare la richiesta.

Tuttavia, l'architettura Supervisor presenta anche alcune limitazioni che devono essere considerate, quali:

- *Singolo punto di fallimento*: la dipendenza dal Supervisor implica che un suo malfunzionamento compromette l'intero sistema, richiedendo strategie di resilienza adeguate.
- *Overhead di latenza e costi*: l'architettura multi-agente moltiplica il numero di chiamate ai servizi LLM esterni rispetto a un sistema con singolo agente. Una singola richiesta utente richiede almeno un passaggio attraverso il Supervisor, seguito da una o più invocazioni dell'agente specializzato. Tale struttura introduce latenza aggiuntiva e, se si utilizzano modelli molto più performanti del necessario per le singole attività, incrementa i costi operativi.
- *Complessità non proporzionata per attività semplici*: l'architettura multi-agente introduce un sovraccarico strutturale che risulta sproporzionato per richieste elementari. Anche una domanda semplice deve attraversare l'intero flusso di coordinamento, con analisi dell'intento da parte del Supervisor, selezione dell'agente appropriato e aggregazione della risposta. Tale rigidità architetturale comporta tempi di elaborazione e consumi di risorse che non si giustificano per attività che un singolo agente potrebbe gestire direttamente.

Nel contesto di Bluey, tali limitazioni risultano accettabili considerando che il sistema gestisce spesso richieste complesse da parte di operatori che possono accettare una latenza più elevata a beneficio della qualità delle risposte.

Considerando i requisiti funzionali e tecnici definiti nel capitolo precedente, sono stati identificati i seguenti agenti specializzati che compongono il team di lavoro:

- *agente Supervisor*: riceve le richieste degli utenti, analizzandole nel contesto della conversazione corrente e selezionando l'agente specializzato più appropriato per gestire ciascuna richiesta;
- *agente SQL*: responsabile dell'interrogazione del database aziendale per recuperare informazioni strutturate, come dati su clienti e polizze;
- *agente email*: dedicato all'invio e alla gestione delle comunicazioni via email a clienti e colleghi tramite il servizio Gmail;
- *agente appointment*: specializzato nella gestione degli appuntamenti degli operatori e dei clienti attraverso l'integrazione con Google Calendar;

- *agente documentation*: incaricato di consultare la documentazione operativa aziendale per fornire risposte dettagliate su procedure e linee guida che gli operatori devono seguire nella loro mansione.

L'architettura complessiva di alto livello progettata per il sistema è illustrata in Figura 4.7.

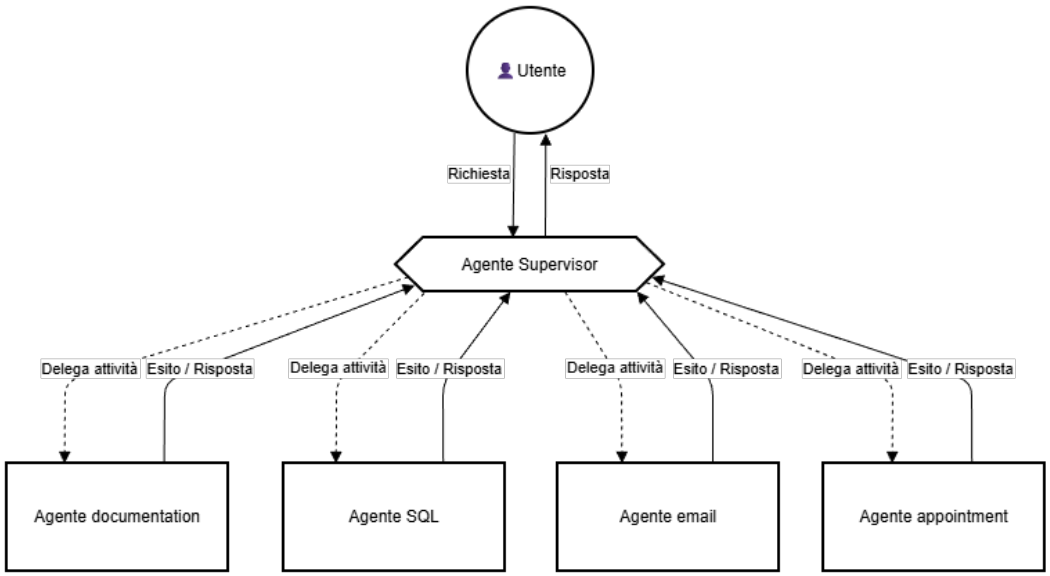


Figura 4.7: Architettura complessiva del sistema Bluey

4.3.1 Meccanismo di comunicazione tra agenti

Nel pattern architetturale Supervisor, la comunicazione tra gli agenti avviene sempre attraverso l'agente supervisore. Dal punto di vista del Supervisor, ciascun agente specializzato appare come un tool, ovvero una funzione invocabile con parametri specifici che restituisce un risultato.

Quando il Supervisor decide di delegare una richiesta a un agente specializzato, formula un messaggio in linguaggio naturale che descrive l'obiettivo da raggiungere, includendo tutti i dettagli necessari estratti dal contesto della conversazione. L'agente specializzato riceve esclusivamente questa richiesta formulata dal Supervisor, senza accesso alla cronologia completa della conversazione. Questa scelta progettuale consente di ridurre il numero di token processati da ciascun agente (diminuendo, conseguentemente, i costi operativi e i tempi di latenza) e garantisce che ogni agente riceva solo le informazioni strettamente rilevanti per il proprio compito.

L'agente specializzato elabora la richiesta utilizzando i propri tool specifici e restituisce al Supervisor una risposta, anch'essa in linguaggio naturale, che sintetizza l'esito dell'operazione. Il Supervisor valuta, quindi, se la risposta soddisfa la richiesta originale dell'utente e, in caso affermativo, restituisce il messaggio all'utente. In caso contrario, il Supervisor può decidere di invocare ulteriori agenti o di richiedere chiarimenti all'utente. Questo modello di comunicazione presenta il vantaggio di disaccoppiare le implementazioni: il Supervisor non necessita di conoscere i dettagli implementativi degli agenti specializzati, ma si limita a descrivere l'obiettivo desiderato in modo completo e autosufficiente. Gli agenti, a loro volta, possono evolvere indipendentemente purché mantengano l'interfaccia concordata, operando come unità esecutive specializzate che ricevono istruzioni precise e restituiscono risultati.

4.3.2 Agente Supervisor

L'agente Supervisor rappresenta il punto di ingresso del sistema e svolge il ruolo di coordinatore dell'intero team di agenti. La sua responsabilità principale consiste nel comprendere le richieste degli utenti, identificare gli agenti più appropriati per soddisfarle e orchestrare l'esecuzione delle attività necessarie.

Il Supervisor possiede una visione completa di tutti gli agenti specializzati disponibili e delle rispettive competenze. Quando riceve una richiesta, analizza il contenuto e il contesto conversazionale per determinare quale agente sia più appropriato per gestire la richiesta. Esso opera, quindi, in modo reattivo: delega la richiesta all'agente selezionato, valuta la risposta ricevuta e, se necessario, procede con ulteriori deleghe ad altri agenti fino al completamento della richiesta originale.

Dato che l'utente interagisce con un'interfaccia unificata che nasconde la complessità del sistema, il Supervisor si occupa di presentare le risposte in modo coerente e con una continuità conversazionale.

Per poter effettuare le deleghe, il Supervisor dispone di tool direttamente collegati all'invocazione di ciascun agente specializzato. Questi tool hanno la funzione di interfaccia tra il Supervisor e gli agenti, permettendo di invocare le loro funzionalità in modo standardizzato. Inoltre, per soddisfare il requisito di esportazione dei dati su foglio di calcolo, è presente un tool aggiuntivo che consente di formattare dei dati in un file Excel (`.xlsx`). Collegando questo tool direttamente al Supervisor, l'utente può richiedere l'esportazione dei dati per qualsiasi risultato ottenuto, indipendentemente dall'agente specializzato che ha generato tali dati.

L'architettura dell'agente Supervisor con i rispettivi tool è illustrata in Figura 4.8.

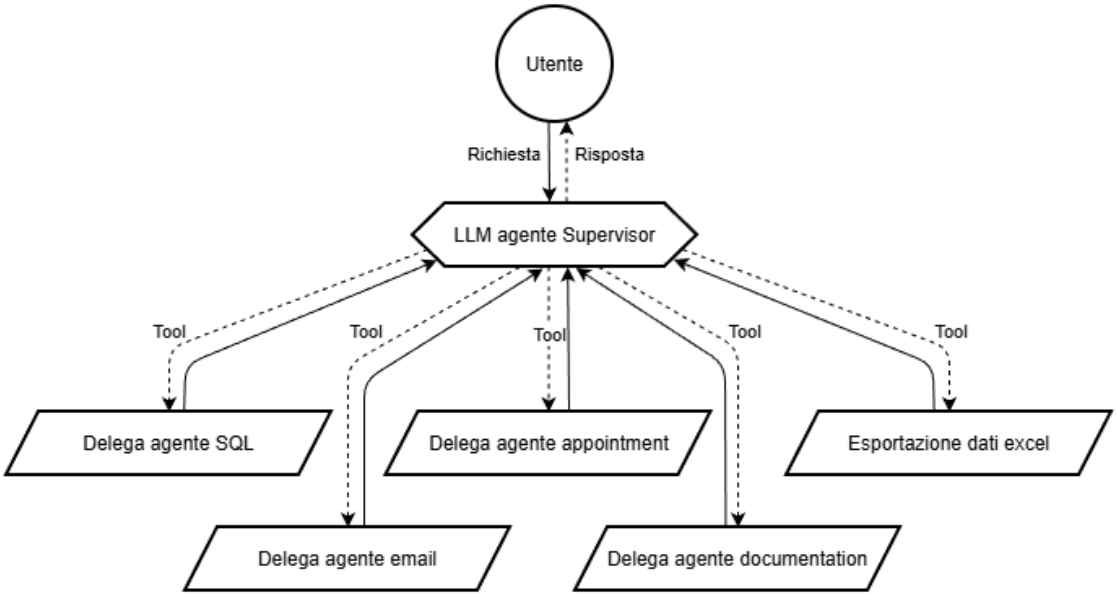


Figura 4.8: Architettura di alto livello dell'agente Supervisor

4.3.3 Agente SQL

L'agente SQL è specializzato nell'interazione con il database aziendale del portale assicurativo. La sua funzione consiste nel tradurre richieste espresse in linguaggio naturale in query strutturate, eseguirle sul database e presentare i risultati in forma comprensibile.

La progettazione di questo agente prevede un flusso di elaborazione articolato in fasi ben definite, ciascuna finalizzata a garantire controllo sulle operazioni eseguite.

Nella fase iniziale, l'agente acquisisce la conoscenza dello schema dell'intero database, identificando le tabelle disponibili. In un secondo momento vengono selezionate le tabelle rilevanti per la richiesta tramite un tool specifico per approfondirne lo schema. Questa informazione fornisce il contesto e la conoscenza delle tabelle necessaria per generare query sintatticamente corrette e semanticamente appropriate.

Nella fase di generazione, l'agente formula la query SQL basandosi sulla richiesta dell'utente e sullo schema delle tabelle interessate. Il modello linguistico è guidato da istruzioni specifiche che enfatizzano le pratiche raccomandate per la scrittura delle query, quali la selezione esplicita delle colonne, l'uso appropriato di ordinamenti e limitazioni.

Nella fase di verifica, un secondo controllo valuta la correttezza della query generata, identificando potenziali errori logici o sintattici, quali l'uso improprio di operatori su valori nulli, incongruenze nei tipi di dato o riferimenti a colonne inesistenti. Questa fase di doppia verifica riduce significativamente il rischio di eseguire query errate.

Nella fase di approvazione, la query validata viene sottoposta all'operatore per l'autorizzazione. L'operatore può visualizzare la query proposta e decidere se approvarla, modificarla o rifiutarla. Questo passaggio risulta fondamentale per garantire il controllo umano e deterministico sulle operazioni che accedono ai dati aziendali.

Nella fase di esecuzione, la query approvata viene eseguita sul database e i risultati vengono restituiti all'agente. Quest'ultimo elabora i risultati grezzi e li presenta in una forma comprensibile, sintetizzando le informazioni rilevanti ed evidenziando eventuali aspetti significativi.

La progettazione dell'agente SQL prevede anche dei cicli di rigenerazione della query nel caso in cui la verifica della query stessa fallisca, l'operatore richieda modifiche o l'esecuzione della query non ottenga dei risultati soddisfacenti.

In Figura 4.9 viene illustrata l'architettura di alto livello dell'agente SQL con le varie fasi descritte.

4.3.4 Agente email

L'agente email gestisce la composizione e l'invio di comunicazioni attraverso il servizio di posta integrato nell'ecosistema aziendale di Google. La sua progettazione si concentra sulla generazione di messaggi professionali, sulla richiesta della conferma esplicita dell'operatore prima dell'invio e sull'integrazione con il servizio Gmail.

Quando riceve una richiesta di invio email, l'agente analizza i requisiti specificati dall'utente, identificando i destinatari, l'oggetto e il contenuto richiesto. Sulla base di queste informazioni, l'agente compone un messaggio completo che include formule di apertura e chiusura appropriate, un corpo del testo chiaro e professionale, ed eventuali riferimenti a pratiche, polizze o altri elementi contestuali.

Prima dell'invio effettivo, l'email composta viene sottoposta all'approvazione dell'operatore attraverso il meccanismo di Human-in-the-Loop. L'operatore può verificare il contenuto, i destinatari e l'oggetto del messaggio, apportando eventuali modifiche prima di autorizzare l'invio. Questo è fondamentale considerando che le email inviate rappresentano comunicazioni ufficiali della compagnia assicurativa verso clienti e partner esterni.

Una volta approvata, l'email viene inviata tramite un tool di integrazione con il servizio Gmail, che gestisce la consegna e la notifica di eventuali errori. L'architettura dell'agente email è illustrata in Figura 4.10.

4.3.5 Agente appointment

L'agente appointment gestisce l'agenda del singolo operatore, occupandosi della verifica della disponibilità e della creazione di nuovi eventi attraverso il servizio di calendario

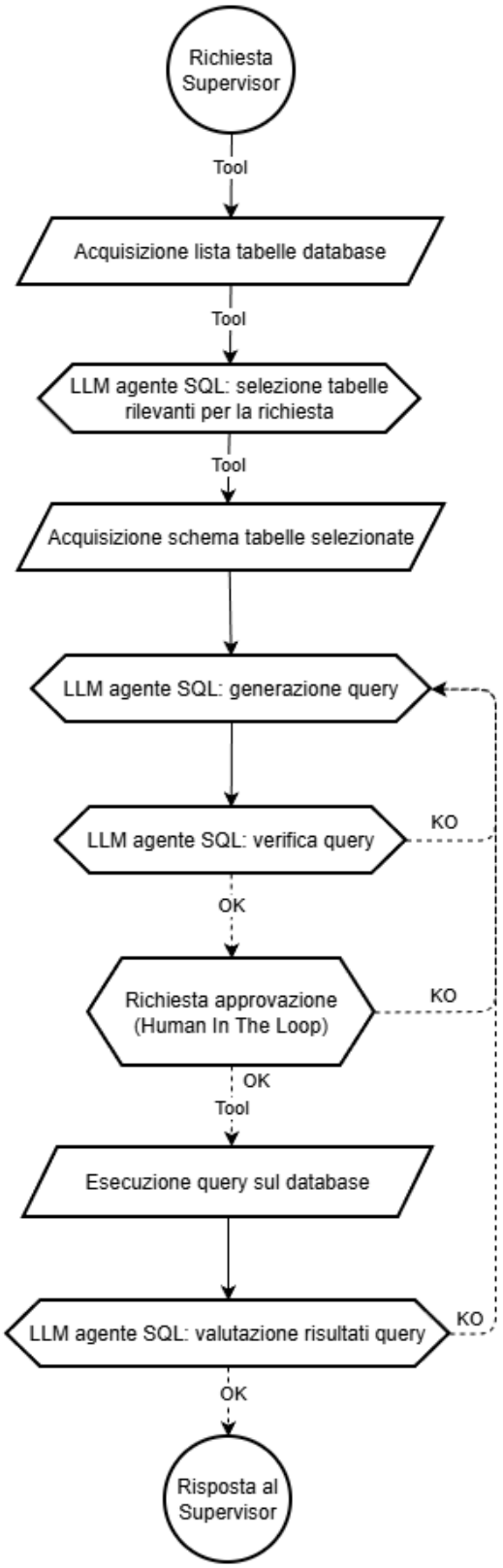


Figura 4.9: Architettura di alto livello dell'agente SQL

integrato nell'ecosistema aziendale di Google. La sua progettazione si concentra sulla verifica accurata delle fasce orarie libere e sulla creazione effettiva degli appuntamenti dopo aver richiesto l'approvazione dell'operatore.

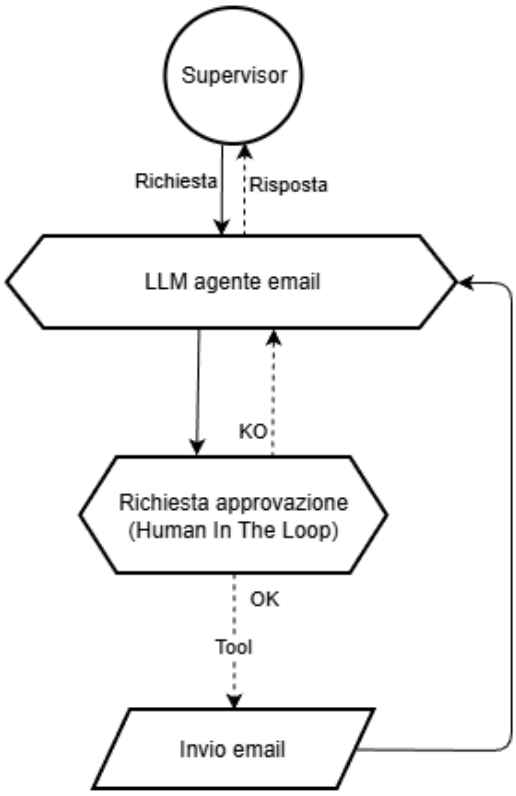


Figura 4.10: Architettura di alto livello dell'agente email

Per le richieste di creazione degli appuntamenti, l'agente raccoglie le informazioni necessarie, quali il titolo dell'evento, la data e l'ora, la durata e l'elenco dei partecipanti. Prima di procedere alla creazione effettiva, l'agente può verificare, tramite uno specifico tool, la disponibilità per confermare che la fascia oraria richiesta sia effettivamente libera.

La creazione dell'evento è soggetta all'approvazione dell'operatore, che può verificare i dettagli dell'appuntamento prima che questo venga inserito nel calendario. Una volta approvato, l'evento viene creato tramite un apposito tool che, grazie all'integrazione con il servizio Google Calendar, gestisce anche l'invio automatico degli inviti ai partecipanti specificati. L'architettura dell'agente appointment è illustrata in Figura 4.11.

4.3.6 Agente documentation

L'agente documentation è specializzato nella ricerca e nel recupero di informazioni dalla raccolta documentale aziendale, rispondendo a quesiti degli operatori attingendo a manuali operativi, guide procedurali e FAQ.

La progettazione si basa sulla tecnica RAG (Retrieval-Augmented Generation), precedentemente descritta nella Sezione 2.5.1, che combina modelli linguistici con meccanismi di recupero delle informazioni per fornire risposte accurate e contestualizzate.

L'architettura RAG implementata prevede l'utilizzo di un indice FAISS (Facebook AI Similarity Search), una libreria sviluppata da Meta per la ricerca efficiente di similarità tra vettori. I documenti aziendali vengono preliminarmente suddivisi in frammenti e convertiti in rappresentazioni vettoriali (embedding) mediante un modello di codifica semantica. Questi vettori vengono poi memorizzati nell'indice FAISS.

L'agente dispone di un tool specifico per interrogare l'indice. Quando riceve una richiesta, il tool converte la domanda nella corrispondente rappresentazione vettoriale e ricerca i frammenti documentali con maggiore similarità semantica. A differenza di una ricerca

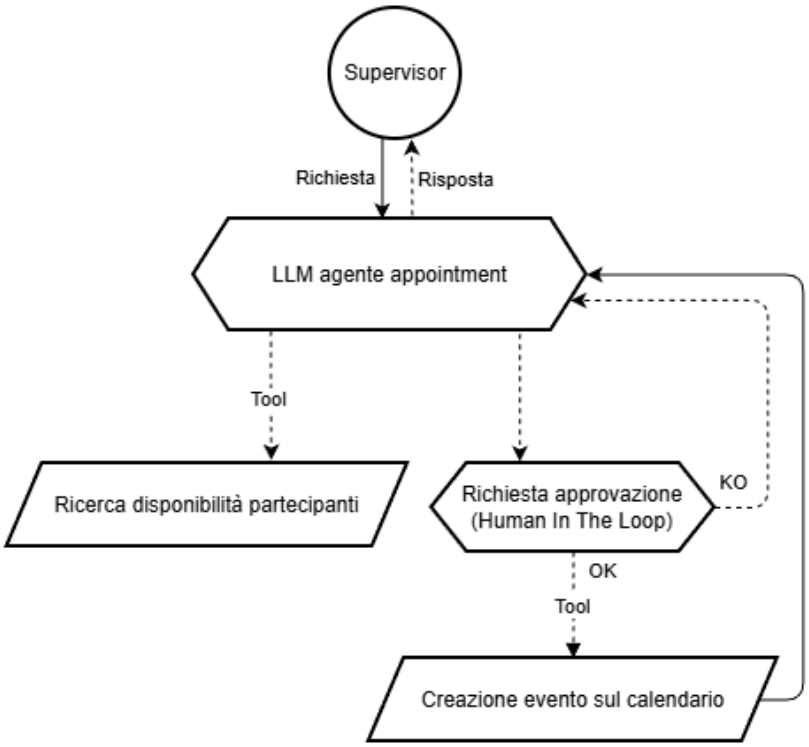


Figura 4.11: Architettura di alto livello dell'agente appointment

per parole chiave, questo approccio comprende il significato della richiesta e individua informazioni pertinenti anche quando la formulazione differisce da quella presente nei documenti. I frammenti recuperati vengono inseriti nel contesto della richiesta inviata al modello linguistico, che li analizza per formulare una risposta coerente citando le fonti.

Diversamente dai precedenti agenti, l'agente documentation non richiede l'approvazione da parte dell'operatore prima di utilizzare il tool, in quanto il materiale consultato è già destinato alla fruizione interna e non comporta rischi di comunicazioni errate verso terzi.

L'architettura dell'agente documentation è illustrata in Figura 4.12.

4.3.7 Flusso di elaborazione delle richieste

Il processo di elaborazione di una richiesta da parte dell'utente si articola in diverse fasi sequenziali.

Nella prima fase, di ricezione e analisi, il Supervisor riceve il messaggio dell'utente e lo analizza nel contesto della conversazione corrente. Il modello linguistico alla base del Supervisor determina l'intento dell'utente e identifica quale agente specializzato sia più appropriato per gestire la richiesta. Nel caso di richieste complesse che richiedono l'intervento di più agenti, il Supervisor stabilisce anche l'ordine di esecuzione.

Nella seconda fase, detta di delega, il Supervisor formula una richiesta specifica in linguaggio naturale per l'agente selezionato, includendo le informazioni contestuali necessarie.

Nella terza fase, detta di elaborazione specializzata, l'agente delegato elabora la richiesta utilizzando i propri tool. Questa fase può includere molteplici iterazioni in cui l'agente invoca tool, analizza i risultati e decide se proseguire con ulteriori azioni o restituire una risposta finale. Per operazioni critiche, questa fase include anche i punti di interruzione per l'approvazione umana.

Nella quarta fase, detta di aggregazione e risposta, il Supervisor riceve la risposta dell'agente specializzato e valuta se essa soddisfa la richiesta originale dell'utente. In caso

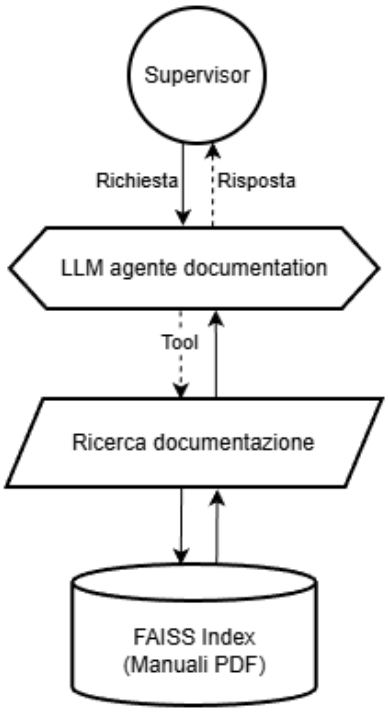


Figura 4.12: Architettura di alto livello dell'agente documentation

affermativo, la risposta viene inoltrata all'utente, eventualmente riformulata per garantire coerenza con il tono conversazionale del sistema. In caso contrario, il Supervisor può decidere di invocare ulteriori agenti o di richiedere chiarimenti all'utente, ripartendo, così, dalla prima fase.

Per le principali tipologie di richieste gestite dal sistema, il flusso di elaborazione sequenziale è illustrato in Figura 4.13.

4.3.8 Considerazioni sulla sicurezza

Il trattamento di dati aziendali e dei clienti impone l'adozione di misure di sicurezza adeguate. La progettazione del sistema Bluey integra diversi meccanismi volti a garantire la protezione delle informazioni. In particolare, i meccanismi integrati sono i seguenti:

1. *Principio del minimo privilegio*: ciascun agente dispone esclusivamente dei tool necessari per svolgere le proprie funzioni, senza accesso a risorse non pertinenti al proprio ambito di competenza. L'agente SQL, ad esempio, può interrogare il database ma non può inviare email; l'agente email può comporre e inviare messaggi ma non può accedere ai dati del database.

2. *Restrizioni operative sulle azioni potenzialmente dannose*: l'agente SQL implementa un controllo deterministico che garantisce l'esecuzione di sole query di lettura, bloccando qualsiasi operazione di modifica dei dati. Analogamente, l'invio di email e la creazione di eventi nel calendario sono subordinati all'approvazione esplicita dell'operatore, prevenendo comunicazioni non autorizzate.

3. *Configurazione di modelli LLM differenziati in base ai requisiti di riservatezza*: l'architettura multi-agente consente di configurare modelli linguistici differenti per ciascun agente specializzato in base alla sensibilità dei dati trattati. Tale caratteristica rappresenta un vantaggio rispetto a un'architettura monolitica con un singolo modello. In particolare,

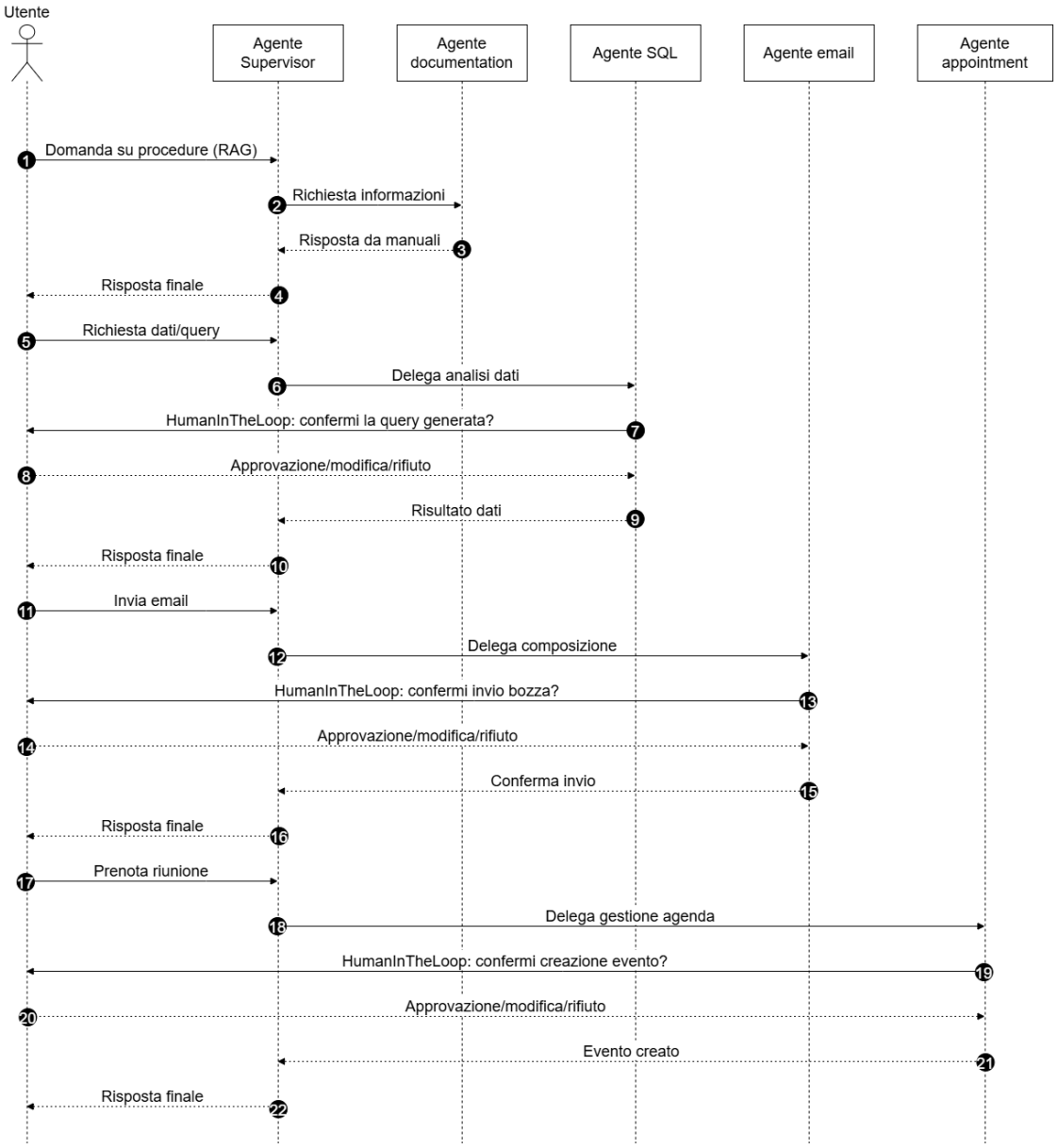


Figura 4.13: Flusso di elaborazione sequenziale delle richieste tra gli agenti

l'agente SQL, che accede a informazioni sensibili quali dati personali dei clienti, dettagli delle polizze e informazioni finanziarie, può essere configurato per utilizzare un modello linguistico eseguito su infrastruttura privata aziendale. Questa configurazione elimina la necessità di trasmettere dati sensibili verso fornitori esterni di servizi LLM, mitigando i rischi di esposizione accidentale o di utilizzo non autorizzato delle informazioni per l'addestramento di modelli di terze parti. Al contempo, gli agenti che trattano dati meno sensibili, come l'agente documentation, che opera su materiale già destinato alla consultazione interna, possono utilizzare modelli cloud esterni più performanti senza particolari criticità. Questa flessibilità consente di ottimizzare il compromesso tra prestazioni, costi e requisiti di conformità normativa, applicando politiche di sicurezza differenziate in funzione della sensibilità dei dati elaborati da ciascun agente.

4. *Isolamento del contesto conversazionale*: ciascun agente specializzato riceve esclusivamente le informazioni strettamente necessarie per completare la propria attività, senza accesso

alla cronologia completa della conversazione. Questo approccio riduce la superficie di esposizione dei dati, limitando la quantità di informazioni che potrebbero essere accidentalmente incluse nelle richieste verso servizi esterni.

5. *Audit e tracciabilità delle operazioni*: l'integrazione con la piattaforma LangSmith fornisce capacità di registrazione e monitoraggio. Ogni azione eseguita dagli agenti viene registrata con i relativi dettagli, consentendo la ricostruzione degli eventi in caso di necessità di verifica della conformità alle politiche aziendali.

CAPITOLO 5

Implementazione degli agenti

In questo capitolo viene descritta l'implementazione tecnica del sistema multi-agente progettato nel capitolo precedente. Verrà analizzata la struttura del progetto, illustrando l'organizzazione dei file e delle cartelle. Successivamente, si approfondirà la gestione dello stato del grafo, fondamentale per il coordinamento tra gli agenti. Infine, verranno dettagliati gli agenti specializzati, descrivendo, per ciascuno, il prompt di sistema utilizzato e i tool associati.

5.1 Struttura del progetto

Il progetto è stato strutturato seguendo i principi di modularità e le linee guida suggerite dalla documentazione ufficiale di LangGraph. Il codice sorgente risulta organizzato in moduli Python distinti, ciascuno dedicato ad una specifica funzionalità del sistema.

La struttura principale del progetto comprende i seguenti file e le seguenti directory:

- `agents.py`: contiene la definizione della classe `Agents`, che inizializza e configura tutti gli agenti del sistema, incluso il Supervisor e gli agenti specializzati (email, appointment, documentation);
- `sql_agent.py`: definisce il sottografo dedicato all'agente SQL. A differenza degli altri agenti, questo è implementato come un grafo di stato complesso per gestire il flusso di generazione e validazione delle query;
- `tools.py`: raccoglie l'implementazione di tutti i tool utilizzati dagli agenti, come le funzioni per l'invio di email, la gestione del calendario, la ricerca documentale (RAG) e l'esportazione in Excel;
- `prompts.py`: centralizza tutti i prompt di sistema utilizzati dagli agenti, facilitando la manutenzione e l'aggiornamento delle istruzioni fornite ai modelli linguistici;
- `langgraph.json`: è un file di configurazione per il framework LangGraph;
- `.env`: è un file contenente le variabili d'ambiente per la configurazione del sistema, come le chiavi API per i servizi utilizzati e i percorsi dei file utilizzati;
- `requirements.txt`: è un file che elenca tutte le dipendenze Python necessarie per l'esecuzione del progetto;

- `data/`: è una directory contenente il database SQLite e altri dati persistenti;
- `docs/`: è una directory contenente i documenti PDF utilizzati per la ricerca documentale.

Questa organizzazione permette di mantenere il codice pulito e manutenibile, facilitando l'aggiunta di nuovi agenti o funzionalità in futuro.

5.2 Gestione dello stato del grafo

Nel sistema implementato, lo stato viene gestito attraverso lo schema di base definito dalla classe `AgentState`, che mantiene la cronologia dei messaggi scambiati tra l'utente e il sistema.

Questo stato è definito come un dizionario contenente la chiave `messages`, annotata con un reducer specifico (`add_messages`). Grazie a quest'ultimo, ogni nuovo messaggio generato dall'utente o dagli agenti non sovrascrive lo stato precedente, ma viene aggiunto in coda alla cronologia esistente. I messaggi possono essere di tipo `HumanMessage` (input dell'utente), `AIMessage` (risposte del modello) o `ToolMessage` (output dei tool). Questo stato consente agli agenti di avere accesso al contesto della conversazione e di prendere decisioni informate basate sulle interazioni precedenti.

Il Supervisor, agendo come orchestratore, gestisce il flusso principale della conversazione. Quando delega un compito ad un agente specializzato, lo invoca come tool, passando una richiesta specifica in linguaggio naturale. È importante notare che, per progettazione, i sub-agenti invocati come tool non hanno accesso all'intera cronologia della conversazione, ma solo alla richiesta specifica formulata dal Supervisor. Questo approccio consente di limitare il contesto e il costo associato al consumo dei token del modello, riducendo il rumore informativo nei sub-agenti e focalizzandone l'attenzione esclusivamente sul task da svolgere.

Nel caso specifico dell'agente SQL, che invece è implementato come un sottografo dedicato, viene utilizzata un'estensione di questo stato base chiamata `SQLAgentState`. Questa classe eredita le proprietà di gestione dei messaggi e aggiunge un campo specifico per il tracciamento dei tentativi di riesecuzione (`retry_count`), necessario per limitare il numero di tentativi di iterazione di una query in caso di fallimento.

5.3 Implementazione degli agenti specializzati

In questa sezione vengono descritti nel dettaglio gli agenti implementati, analizzando, per ciascuno, la definizione nel codice, il prompt di sistema che ne definisce il comportamento e i tool che hanno a disposizione.

Nel sistema sviluppato, tutti gli agenti sono creati tramite la funzione `create_agent` di LangGraph, che genera un agente ReAct ("Reasoning and Acting") in grado di utilizzare tool e ragionare sulle azioni da compiere. I modelli linguistici utilizzati sono specificati in modo parametrico nelle variabili d'ambiente illustrate nel Listato 5.1, consentendo di modificare facilmente il fornitore e il modello senza dover modificare il codice. Questo avviene grazie alla funzione `init_chat_model`, che prevede come parametro una stringa nel formato "fornitore:modello" (ad esempio "`openai:gpt-5-nano`"), specificando, così, entrambe le informazioni.

```
EMAIL_AGENT_MODEL = openai:gpt-5-nano
APPOINTMENT_AGENT_MODEL = openai:gpt-5-nano
SQL_AGENT_MODEL = ollama:qwen3-coder:480b-cloud
DOCUMENTATION_AGENT_MODEL = openai:gpt-5-nano
SUPERVISOR_AGENT_MODEL = openai:gpt-5-mini
```

Listato 5.1: Configurazione dei modelli LLM nelle variabili d'ambiente

Per massimizzare l'efficacia delle istruzioni fornite ai modelli, tutti i prompt di sistema sono stati progettati adottando il framework di prompt engineering CO-STAR (Context, Objective, Style, Tone, Audience, Response). Questa metodologia, ormai consolidata, permette di definire in modo strutturato il contesto operativo, specificare chiaramente gli obiettivi, controllare lo stile e la risposta attesa, riducendo le ambiguità e migliorando la coerenza del comportamento degli agenti.

5.3.1 Agente Supervisor

L'agente Supervisor è il componente centrale del sistema, responsabile del coordinamento degli altri agenti. Non esegue direttamente i task operativi (eccetto l'esportazione dati), ma analizza la richiesta dell'utente, decide quale agente specializzato attivare e invoca il tool corrispondente.

Definizione

Nel Listato 5.2 viene mostrata la definizione dell'agente Supervisor in cui vengono associati i principali parametri, come il modello di riferimento (direttamente associato alle variabili d'ambiente) e il prompt di sistema, definito come costante in un file separato. Il parametro `tools` include tutti i tool necessari per invocare i sub-agenti (uno per ogni agente specializzato) e il tool di esportazione Excel.

```
self.supervisor = create_agent(
   model=init_chat_model(os.getenv('SUPERVISOR_AGENT_MODEL'), temperature=0),
   tools=[
      *self.subagent_tools,
      export_to_excel
   ],
   system_prompt=SUPERVISOR_SYSTEM_PROMPT,
   name="supervisor"
)
```

Listato 5.2: Definizione dell'agente Supervisor

Prompt

Il prompt del Supervisor definisce la sua personalità come "Bluey", assistente per gli operatori di back-office. Utilizzando la metodologia CO-STAR, istruisce il modello su come utilizzare gli agenti disponibili e su come gestire il flusso della conversazione, il tono e la risposta attesa con degli esempi pratici. Nel Listato 5.3 viene riportato il prompt di sistema utilizzato.

```
## CONTEXT
Sei Bluey, assistente virtuale per operatori di back office di una compagnia assicurativa. Coordini un
      team di agenti specializzati, ma l'utente ti percepisce come un unico assistente e non deve sapere
      della struttura sottostante.

Agenti disponibili:
- email_agent: composizione e invio email professionali
- appointment_agent: prenotazioni e verifica disponibilita calendario
- documentation_agent: ricerca in manuali e FAQ aziendali
- sql_agent: query database per clienti, polizze, sinistri

Note tecniche: i sub-agenti vengono invocati senza cronologia della conversazione. Quando deleghi, devi
      includere tu TUTTI i dettagli utili (contesto, vincoli, dati noti e dati mancanti).

## OBJECTIVE
1. Comprendere la richiesta dell'utente.
2. Identificare il sub-agente adatto (o rispondere direttamente se semplice).
3. Formulare richiesta completa (massimo 1000 caratteri) per il sub-agente con tutto il contesto, usando
      il template: 1. TASK:, 2. CONTESTO:, 3. DATI NOTI:, 4. OUTPUT ATTESO:.
4. Valutare la risposta del sub-agente:
```

```
   - se ci sono errori -> iterare se necessario aggiungendo contesto
   - se sono stati richiesti dati mancanti -> chiedere chiarimenti all'utente e rinvocare al sub-agente
        con i nuovi dati completi
   - se non soddisfa completamente la richiesta -> capire se ènecessario invocare altri sub-agenti.
   - se la risposta èsoddisfacente -> presentarla all'utente in modo chiaro e conciso.

Vincoli:
- Includi TUTTI i dettagli nelle richieste agli agenti.
- Chiedi chiarimenti se mancano informazioni indispensabili (email, data, ora, durata, identificativo
    pratica/polizza/sinistro).
- Non rivelare MAI la struttura multi-agente e dei sub-agenti.
- Non richiamare stesso agente senza modifiche al contesto.
- Non inventare informazioni.

## STYLE
Professionale, conciso, strutturato.

## TONE
Cordiale, efficiente.

## AUDIENCE
Operatori back office con competenze IT di base, familiari con terminologia assicurativa.

## RESPONSE
Risposte dirette e sincere. Per dati tabellari, esponili in tabella in formato markdown. IMPORTANTE: NON
    generare MAI il file Excel automaticamente e NON chiamare il tool export_to_excel se non
    esplicitamente richiesto dall'utente. Limitati a mostrare i dati in chat. Conferma azioni completate.
     Se errore persiste dopo retry, comunica il problema e suggerisci alternativa.

Esempi (U=user, T=tool_call, R=tool_response, A=assistant):

1. Query database:
U: "Quanti sinistri ha aperto Mario Rossi?"
T: transfer_to_sql_agent(request="TASK: Contare sinistri cliente Mario Rossi. CONTESTO: Richiesta
    operatore. DATI NOTI: Nome=Mario Rossi. OUTPUT: Numero e dettagli sinistri")
R: "3 sinistri: SN001 (aperto), SN002 (chiuso), SN003 (in_lavorazione)"
A: "Mario Rossi ha 3 sinistri: [tabella con ID, Data, Stato]"

2. Invio email:
U: "Invia mail a mario.rossi@email.it per confermare appuntamento domani h15"
T: transfer_to_email_agent(request="TASK: Email conferma appuntamento. DATI NOTI: to=mario.rossi@email.it,
     data=domani, ora=15:00")
R: "Email inviata! A: mario.rossi@email.it | Oggetto: Conferma appuntamento"
A: "Email di conferma inviata con successo a mario.rossi@email.it"
```

Listato 5.3: Prompt di sistema dell'agente Supervisor

Tool utilizzati

Per associare ogni agente a un tool, è stato sviluppato un metodo generico che permette di creare un tool LangChain a partire dalla descrizione e dal riferimento di un agente LangGraph.

```
def wrap_agent_as_tool(
   self,
   *,
   agent: CompiledStateGraph,
   name: str,
   description: str
) -> BaseTool:
   """
   Questo metodo permette al supervisor di invocare gli agenti specializzati come se fossero tool standard,
         passando richieste in linguaggio naturale.

   Args:
      agent: L'agente compilato da richiamare.
      name: Nome univoco del tool (es. 'call_email_agent').
      description: Descrizione del tool per il supervisor.

   Returns:
      BaseTool: Tool LangChain che incapsula l'agente.
   """

   @tool_decorator(name, description=description)
   def agent_tool(request: str) -> str:
      """Instrada una richiesta in linguaggio naturale al sub-agente corrispondente.

      Il supervisor deve includere nel parametro 'request' tutti i dettagli
      necessari per completare il task (es. nomi, date, indirizzi email, ecc.).
      Il subagente non ha accesso alla cronologia della conversazione.
      """
      response = agent.invoke({"messages": [HumanMessage(content=request)]})
      return response["messages"][-1].text

   return agent_tool
```

Listato 5.4: Definizione del metodo per l'associazione di un agente a un tool

Nel Listato 5.5 viene mostrata l'implementazione del tool `export_to_excel`, che permette di esportare dati strutturati (nel formato Python `dict`) in formato Excel (`.xlsx`). Esso utilizza la libreria `pandas` per generare il file e restituisce un collegamento per il download. Accetta come parametri i dati da esportare (`data`) e un nome opzionale per il file (`filename`).

```
@tool
def export_to_excel(
   data: dict,
   filename: Optional[str] = None
) -> str:
   """
   Esporta i dati in un file Excel (formato .xlsx) e restituisce un link per il download.

   Args:
      data: Dati da esportare - deve essere un dizionario
      filename: Nome del file opzionale (senza estensione)

   Returns:
      Messaggio di successo con l'URL per il download

   Example:
      export_to_excel({"nome": "Mario", "età": 30}, "utenti_singolo")
      export_to_excel({
         "nome": ["Mario", "Luigi"],
         "età": [30, 28]
      }, "utenti_tabella")
   """
   try:
      # Genera il nome del file se non fornito
      if filename is None:
         timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
         filename = f"export_{timestamp}"

      # Rimuovi l'estensione .xlsx se fornita
      filename = filename.replace('.xlsx', '')

      # Valida i dati
      if not data or not isinstance(data, dict):
         return "Nessun dato da esportare o formato non valido (richiesto dict)"

      # Converti i dati in DataFrame
      if all(isinstance(v, (list, tuple)) for v in data.values()):
         df = pd.DataFrame(data)
      else:
         df = pd.DataFrame([data])

      # Esporta in Excel
      full_filename = f"{filename}.xlsx"
      filepath = get_output_dir() / full_filename
      df.to_excel(filepath, index=False, engine='openpyxl')

      return (
         f"File Excel esportato con successo!\n\n"
         f"**Download:** {get_download_url(full_filename)}"
      )

   except Exception as e:
      return f"Errore nell'esportazione: {str(e)}"
```

Listato 5.5: Implementazione del tool `export_to_excel`

5.3.2 Agente SQL

L'agente SQL è progettato per interagire con il database aziendale in modo sicuro ed efficace. Date le esigenze di controllo e determinismo del processo, è implementato come sottografo (`StateGraph`) che guida il modello attraverso una sequenza precisa di fasi: esplorazione delle tabelle, recupero dello schema, generazione della query, validazione ed esecuzione.

Definizione

L'agente SQL viene realizzato come `StateGraph` per gestire un workflow articolato. Tale approccio permette di definire un grafo composto da nodi e archi, che guidano il modello attraverso una sequenza precisa di passaggi. In particolare, sono stati implementati sette nodi:

- `list_tables`: recupera, tramite tool, l'elenco delle tabelle nel database, fornendo al modello il contesto sui dati disponibili;
- `call_get_schema`: invoca il modello per selezionare le tabelle rilevanti per la richiesta dell'utente, al fine di ottenerne lo schema e comprendere le colonne, i tipi di dato e le relazioni tra tabelle;
- `get_schema`: esegue tramite tool la query per ottenere la struttura DDL delle tabelle selezionate;
- `generate_query`: invoca il modello per generare la query SQL e la relativa spiegazione utilizzando un output strutturato;
- `check_query`: valida la query tramite un controllo deterministico di sicurezza (effettuato mediante espressioni regolari per rilevare operazioni non consentite) e una revisione con LLM per eventuali errori logici o sintattici;
- `run_query`: esegue tramite tool la query validata, richiedendo prima l'approvazione dell'utente mediante meccanismo Human-in-the-loop;
- `evaluate_result`: invoca il modello per analizzare i risultati dell'esecuzione e determinare se sono soddisfacenti rispetto all'obiettivo assegnato o se è necessaria una nuova interrogazione al database.

Oltre ai nodi, il grafo implementa delle logiche condizionali per gestire i fallimenti e le iterazioni. Sono presenti due archi condizionali principali:

- `should_run_query`: posizionato dopo il nodo di controllo, determina se procedere con l'esecuzione o ritornare alla fase di generazione in caso di errori gravi, fino a un numero massimo di tentativi;
- `should_finish_after_eval`: situato dopo la valutazione dei risultati, decide se terminare il flusso esponendo i dati ottenuti o riavviare la generazione qualora la richiesta iniziale non sia soddisfatta.

Nel Listato 5.6 viene mostrata la struttura principale di questo sottografo.

```
self.sql_agent = create_sql_subgraph(init_chat_model(os.getenv('SQL_AGENT_MODEL'), temperature=0))

def create_sql_subgraph(model: BaseChatModel) -> CompiledStateGraph:
    # Costruzione del grafo
    builder = StateGraph(SQLAgentState)
    builder.add_node(list_tables)
    builder.add_node(call_get_schema)
    builder.add_node("get_schema", get_schema_node)
    builder.add_node(generate_query)
    builder.add_node(check_query)
    builder.add_node("run_query", run_query_node)
    builder.add_node(evaluate_result)

    builder.add_edge(START, "list_tables")
    builder.add_edge("list_tables", "call_get_schema")
    builder.add_edge("call_get_schema", "get_schema")
    builder.add_edge("get_schema", "generate_query")
    builder.add_edge("generate_query", "check_query")
    builder.add_edge("run_query", "evaluate_result")

    builder.add_conditional_edges(
        "check_query",
        should_run_query,
        ["run_query", "generate_query", END]
    )
    builder.add_conditional_edges(
        "evaluate_result",
        should_finish_after_eval,
        ["generate_query", END]
    )

    return builder.compile(name="sql_agent")
```

Listato 5.6: Definizione del sottografo SQL

LangGraph permette di visualizzare graficamente il grafo di stato attraverso il metodo `draw`. Nella Figura 5.1 viene mostrato il grafo ottenuto dall'implementazione dell'agente SQL.

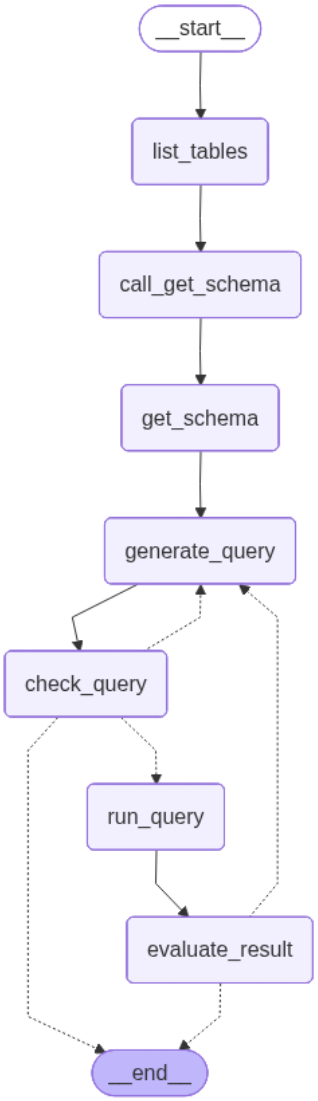


Figura 5.1: Grafo di stato dell'implementazione dell'agente SQL

Prompt

Per ogni fase di invocazione del modello viene utilizzato un prompt specifico. Ogni prompt include come parametro `dialect` per specificare in modo dinamico il tipo di database (ad esempio, MySQL e PostgreSQL) e `top_k` per limitare il numero di righe restituite. La prima fase di generazione della query utilizza il prompt `SQL_GENERATE_QUERY_SYSTEM_PROMPT`, riportato nel Listato 5.7.

```
## CONTEXT
Agente query database {dialect} per sistema Bluey. Non inventare nomi di tabelle o colonne.

## OBJECTIVE
1. Analizzare richiesta, identificare dati necessari.
2. Generare query SQL corretta e ottimizzata usando ESCLUSIVAMENTE tabelle e colonne presenti nello schema
      recuperato tramite tool `sql_db_schema`.
```

```
Vincoli: limitare a {top_k} righe se non specificato. Solo colonne pertinenti (MAI SELECT *). Ordinare per
       colonna rilevante. Alias chiari. Una singola query (evitare query multiple separate da ';'). MAI DML
       (INSERT, UPDATE, DELETE). MAI DDL (DROP, ALTER, CREATE).

## STYLE
Preciso, tecnico.

## TONE
Professionale, tecnico.

## AUDIENCE
Sistema interno di validazione query SQL.

## RESPONSE
Restituisci ESCLUSIVAMENTE un oggetto JSON con le chiavi "query" ed "explanation".
Esempio:
{{
   "query": "SELECT * FROM users",
   "explanation": "Seleziono tutti gli utenti dalla tabella users"
}}
```

Listato 5.7: Prompt di sistema per la generazione delle query SQL

Il secondo prompt, `SQL_CHECK_QUERY_SYSTEM_PROMPT`, riportato nel Listato 5.8, viene utilizzato dopo la generazione della query per validarla prima dell'esecuzione, agendo come revisore.

```
## CONTEXT
Revisore SQL per sistema Bluey. Verifica e corregge query {dialect} prima dell'esecuzione.

## OBJECTIVE
1. Verificare sintassi e sicurezza della query proposta.
2. Identificare pattern problematici: NOT IN con possibili NULL, UNION vs UNION ALL, BETWEEN esclusivi,
      mismatch tipi, identificatori non quotati, ecc.
3. DECISIONE:
   - SE LA QUERY E' CORRETTA: Approvarla invocando il tool `run_query` con la query confermata.
   - SE CI SONO ERRORI MINORI (es. alias mancanti, sintassi correggibile, ottimizzazioni): CORREGGI la
         query e APPROVALA invocando direttamente il tool `run_query` con la versione corretta. NON
         rimandare indietro per piccole correzioni.
   - SE CI SONO ERRORI GRAVI (es. allucinazione tabelle/colonne, logica totalmente errata, richiesta
         impossibile senza nuovi dati): Rifiutarla rispondendo con un messaggio di testo che spiega i
         problemi. Questo rimanderà indietro la richiesta per la rigenerazione.

## STYLE
Rigoroso ma proattivo.

## TONE
Tecnico, costruttivo.

## AUDIENCE
Sistema interno di validazione query SQL.

## RESPONSE
Se OK o CORREGGIBILE -> TOOL CALL `run_query` con la query corretta.
Se ERRORI GRAVI -> TESTO: "Problemi: [lista] | Rigenerazione necessaria"
```

Listato 5.8: Prompt di sistema per la validazione delle query SQL

Infine, il prompt `SQL_EVALUATE_RESULT_SYSTEM_PROMPT`, riportato nel Listato 5.9, interviene dopo l'esecuzione della query per analizzare, ed eventualmente esporre, i risultati ottenuti o, in alternativa, richiedere un nuovo tentativo in caso di dati non soddisfacenti.

```
## CONTEXT
Analista risultati SQL per sistema Bluey. Valuti l'output dell'esecuzione della query sul database {
      dialect}.

## OBJECTIVE
1. Analizzare il risultato ottenuto dal tool `run_query`.
2. DECISIONE:
  - SE IL RISULTATO E' SODDISFACENTE (Risponde alla domanda utente): Sintetizzarlo chiaramente per l'
        utente finale.
  - SE IL RISULTATO E' VUOTO O ERRORE:
    - Se èun errore SQL: Suggerire correzione e richiedere nuova generazione.
    - Se èvuoto ma la query era corretta: Informare l'utente "Nessun dato trovato".
    - Se i dati non rispondo alla domanda: Segnalare il problema.
  - SE MANCANO DATI: Richiedere nuova query.

## STYLE
Chiaro, diretto, orientato all'utente.

## TONE
Tecnico, costruttivo.

## AUDIENCE
Operatori che estraggono dati per verifiche, report, risposte clienti.

## RESPONSE
```

```
- Se Successo: "SINTESI: [Risposta finale per l'utente]"
- Se Necessaria Nuova Query: "ERRORE: [Spiegazione] | RETRY"
```

Listato 5.9: Prompt di sistema per la valutazione dei risultati SQL

Tool utilizzati

L'agente SQL utilizza i tool forniti dal `SQLDatabaseToolkit` di LangChain, una raccolta di strumenti per interagire con database SQL. In questo caso, sono stati utilizzati i seguenti:

- `sql_db_list_tables`: elenca le tabelle presenti nel database.
- `sql_db_schema`: recupera lo schema e le istruzioni DDL di tabelle specifiche. Accetta come parametro `table_names`, una lista di stringhe indicante le tabelle di interesse.
- `sql_db_query`: esegue la query SQL sul database. Accetta come parametro `query`, ovvero una stringa contenente l'istruzione SQL da eseguire.

Il tool di esecuzione della query è stato configurato con un meccanismo di Human-in-the-loop (HITL) che intercetta l'esecuzione tramite la funzione `interrupt` di LangGraph, richiedendo l'approvazione esplicita dell'operatore prima di procedere. È stato, quindi, sviluppato un wrapper, riportato nel Listato 5.10, per sospendere l'esecuzione e attendere un feedback umano.

```
@tool(
   name_or_callable=run_query_tool.name,
   description=run_query_tool.description,
   args_schema=run_query_tool.args_schema
)
def run_query_tool_with_interrupt(config: RunnableConfig, **tool_input):
   action_request: ActionRequest = {
      "name": run_query_tool.name,
      "args": tool_input,
      "description": (
         "Approva, modifica o rifiuta la seguente query SQL prima dell'esecuzione"
      ),
   }
   review_config: ReviewConfig = {
      "action_name": run_query_tool.name,
      "allowed_decisions": ["approve", "edit", "reject"],
   }
   hitl_request: HITLRequest = {
      "action_requests": [action_request],
      "review_configs": [review_config],
   }

   response = interrupt(hitl_request)
   decisions = response.get("decisions") or []
   if decisions:
      decision = decisions[0]
      decision_type = decision.get("type")
      if decision_type == "approve":
         return run_query_tool.invoke(tool_input, config)
      if decision_type == "edit":
         edited_action = decision.get("edited_action") or {}
         edited_args = edited_action.get("args") or {}
         tool_input = {**tool_input, **edited_args}
         return run_query_tool.invoke(tool_input, config)
      if decision_type == "reject":
         message = decision.get("message") or "Tool call rejected by reviewer."
         return {"error": message}
   return response
```

Listato 5.10: Wrapper HITL per l'esecuzione delle query

5.3.3 Agente email

Questo agente è specializzato nella redazione e nell'invio di comunicazioni email professionali.

Definizione

Nel Listato 5.11 viene mostrata la definizione dell'agente email. Il parametro `middleware` configura il meccanismo HITL per il tool `send_email`, richiedendo l'approvazione dell'operatore prima dell'invio effettivo.

```
self.email_agent = create_agent(
   model=init_chat_model(os.getenv('EMAIL_AGENT_MODEL'), temperature=0),
   tools=[send_email],
   system_prompt=EMAIL_AGENT_SYSTEM_PROMPT,
   name="email_agent",
   middleware=[
      HumanInTheLoopMiddleware(
         interrupt_on={"send_email": True},
         description_prefix="Invio email in corso",
      ),
   ]
)
```

Listato 5.11: Definizione dell'agente email

Prompt

Il prompt istruisce l'agente a mantenere un tono professionale e a verificare la presenza di tutti i campi necessari prima dell'invio. Nel Listato 5.12 viene riportato tale prompt di sistema.

```
## CONTEXT
Agente invio email per sistema Bluey, compagnia assicurativa. Tool disponibile: `send_email(to, subject,
      body, cc)` dove to e cc sono liste email, subject e body sono stringhe (testo/HTML).

## OBJECTIVE
1. Identificare: destinatari, oggetto, scopo.
2. Comporre testo professionale appropriato al contesto con saluto iniziale e firma aziendale.
3. Includere eventuali riferimenti (polizze, sinistri, pratiche).
4. Inviare e confermare.

Vincoli: verificare destinatario e oggetto presenti. MAI linguaggio informale. MAI info sensibili non
      richieste. Se mancano info, richiederle.

## STYLE
Formale ma non burocratico. Chiaro, strutturato.

## TONE
Professionale, cortese. Assertivo per urgenze.

## AUDIENCE
Clienti, colleghi, consulenti, fornitori.

## RESPONSE
Formato conferma: Email inviata! A: [email] | Oggetto: [X] | CC: [se presente]
Se mancano info: Servono: [campo1], [campo2]

Template standard:
- Conferma appuntamento: "Gentile [NOME], confermiamo appuntamento [DATA] ore [ORA]. Cordiali saluti, [
      FIRMA]"
- Notifica sinistro: "Gentile [NOME], pratica sinistro n.[NUM] e stata [STATO]. Cordiali saluti, [FIRMA]"

Esempi (U=user, T=tool_call, R=tool_response, A=assistant):

1. Invio email completo:
U: "Mail a mario.rossi@email.it conferma ricezione docs sinistro SN2024001"
T: send_email(to=["mario.rossi@email.it"], subject="Conferma ricezione documentazione - Sinistro SN2024001
      ", body="Gentile Sig. Rossi, confermiamo ricezione documentazione sinistro SN2024001...", cc=[])
R: "Email inviata. Message ID: msg_xyz789"
A: "Email inviata! A: mario.rossi@email.it | Oggetto: Conferma ricezione documentazione - Sinistro
      SN2024001"

2. Dati mancanti:
U: "Invia una email urgente"
A: "Servono: destinatario (email), oggetto o scopo della comunicazione"
```

Listato 5.12: Prompt di sistema dell'agente email

Tool utilizzati

L'agente utilizza un unico tool per inviare email, `send_email`, che si interfaccia con le API di Gmail.

In particolare, il tool si avvale della libreria `google-api-python-client` per gestire l'autenticazione tramite OAuth2 e l'invio del messaggio codificato in base64. I parametri richiesti per il corretto funzionamento includono `to`, una lista di stringhe rappresentante gli indirizzi dei destinatari principali, `subject`, che definisce l'oggetto dell'email, e `body`, ovvero il corpo del messaggio. È previsto, inoltre, l'uso del parametro opzionale `cc` per indicare una lista di indirizzi in copia conoscenza.

Prima dell'invio effettivo, tramite middleware, la chiamata viene intercettata, permettendo all'operatore di revisionare il destinatario, l'oggetto e il corpo del messaggio per garantire la correttezza della comunicazione.

Nel Listato 5.13 viene mostrata l'implementazione della funzione.

```
@tool
def send_email(
   to: list[str],
   subject: str,
   body: str,
   cc: list[str] = []
) -> str:
   """
   Send an email to a recipient.
   """
   if not to or not subject or not body:
      return "Parametri mancanti: to, subject e body sono obbligatori"

   recipients = to[:]
   message = EmailMessage()
   message["To"] = ", ".join(to)
   if cc:
      message["Cc"] = ", ".join(cc)
      recipients.extend(cc)

   sender = os.getenv("GOOGLE_EMAIL_SENDER")
   if sender:
      message["From"] = sender
   message["Subject"] = subject
   message.set_content(body)

   try:
      service = _build_google_service("gmail", "v1")
      encoded_message = base64.urlsafe_b64encode(message.as_bytes()).decode()
      service.users().messages().send(
         userId="me",
         body={"raw": encoded_message}
      ).execute()
      return f"Email inviata con successo a {', '.join(recipients)}"
   except FileNotFoundError as exc:
      return f"{exc}"
   except HttpError as err:
      return f"Errore Gmail API: {err}"
   except Exception as exc:
      return f"Errore inatteso durante l'invio: {exc}"
```

Listato 5.13: Implementazione del tool `send_email`

5.3.4 Agente appointment

L'agente appointment gestisce la pianificazione degli appuntamenti, interfacciandosi con il calendario aziendale tramite Google Calendar API.

Definizione

Nel Listato 5.14 viene mostrata la definizione dell'agente appointment. Il middleware HITL è configurato per il tool `create_calendar_event`, permettendo all'operatore di approvare la creazione di nuovi eventi.

```
self.appointment_agent = create_agent(
   model=init_chat_model(os.getenv('APPOINTMENT_AGENT_MODEL'), temperature=0),
   tools=[get_available_time_slots, create_calendar_event],
   middleware=[
      HumanInTheLoopMiddleware(
         interrupt_on={"create_calendar_event": True},
         description_prefix="Creazione appuntamento in corso",
      ),
   ],
   system_prompt=APPOINTMENT_AGENT_SYSTEM_PROMPT.format(
```

```
        current_time=datetime.now().strftime("%A %d %B %Y, %H:%M"),
        timezone=datetime.now().astimezone().tzinfo or "Europe/Rome"
    ),
    name="appointment_agent"
)
```

Listato 5.14: Definizione dell'agente appointment

Prompt

Il prompt guida l'agente nel processo di verifica della disponibilità e nella pianificazione di un appuntamento. Nel Listato 5.15 viene riportato il prompt di sistema utilizzato, che include come parametri la data, l'ora corrente e il fuso orario per fornire un riferimento temporale.

```
## CONTEXT
Agente gestione appuntamenti per sistema Bluey, agente di una compagnia assicurativa.
DATA E ORA CORRENTE: {current_time}
TIMEZONE: {timezone}

Tools disponibili:
- `get_available_time_slots(attendees, date, duration_minutes)`: verifica slot liberi (attendees: lista
    email, date: "YYYY-MM-DD", duration_minutes: durata). Restituisce lista slot.
- `create_calendar_event(title, start_time, end_time, attendees)`: crea evento (start_time/end_time: "YYYY-
    MM-DDTHH:MM:SS").

## OBJECTIVE
1. Identificare: partecipanti, data, durata, scopo.
2. SEMPRE verificare disponibilita prima di creare (MAI creare senza verifica).
3. Proporre alternative se slot non disponibile.
4. Creare evento solo dopo conferma disponibilita.

Vincoli: MAI assumere orari non specificati. Se mancano data, durata o partecipanti, chiedere chiarimenti
    prima di qualsiasi chiamata tool. Se manca l'orario, proporre 2-3 alternative tra gli slot
    disponibili.

## STYLE
Organizzato, metodico, preciso con date/orari.

## TONE
Efficiente, proattivo, rassicurante.

## AUDIENCE
Operatori che organizzano incontri per polizze, sinistri, rinnovi.

## RESPONSE
Per verifica:
Disponibilita [DATA]: Partecipanti: [email], Durata: [X] min, Slot liberi: [lista]

Per creazione:
Appuntamento creato! Titolo: [X] | Data: [X] | Ora: [X] | Partecipanti: [lista]

Esempi (U=user, T=tool_call, R=tool_response, A=assistant):

1. Prenotazione completa:
U: "Prenota con mario.rossi@email.it domani h14, 1 ora, rinnovo polizza"
T: get_available_time_slots(attendees=["mario.rossi@email.it"], date="2024-12-17", duration_minutes=60)
R: ["09:00-10:00", "11:00-12:00", "14:00-15:00", "16:00-17:00"]
T: create_calendar_event(title="Rinnovo Polizza - Mario Rossi", start_time="2024-12-17T14:00:00", end_time
    ="2024-12-17T15:00:00", attendees=["mario.rossi@email.it"])
R: "Evento creato. ID: evt_abc123"
A: "Appuntamento creato! Titolo: Rinnovo Polizza - Mario Rossi | Data: 17/12/2024 | Ora: 14:00-15:00"

2. Orario non specificato:
U: "Fissa incontro con luigi.bianchi@email.it per giovedi, 30 minuti"
T: get_available_time_slots(attendees=["luigi.bianchi@email.it"], date="2024-12-19", duration_minutes=30)
R: ["10:00-10:30", "14:30-15:00", "16:00-16:30"]
A: "Slot disponibili giovedi 19/12: 10:00-10:30, 14:30-15:00, 16:00-16:30. Quale preferisci?"
```

Listato 5.15: Prompt di sistema dell'agente appointment

Tool utilizzati

L'agente dispone di due tool principali che interagiscono con le API di Google Calendar.

Il primo tool, `get_available_time_slots`, verifica la disponibilità degli invitati in una data specifica interrogando l'endpoint `freebusy` dell'API di Google Calendar. Il sistema calcola gli intervalli liberi sottraendo i periodi occupati dall'intera giornata lavorativa. I parametri richiesti per questa operazione sono la lista delle email dei partecipanti (`attendees`),

la data di interesse in formato ISO (`date`) e la durata prevista dell'appuntamento in minuti (`duration_minutes`).

Il secondo tool, `create_calendar_event`, permette di creare l'evento nel calendario configurato. I parametri necessari includono il titolo dell'evento (`title`), gli orari di inizio e fine in formato ISO (`start_time` e `end_time`) e la lista dei partecipanti (`attendees`). Poiché questa operazione comporta una modifica effettiva del calendario, la sua esecuzione è soggetta ad approvazione umana tramite middleware. Nel Listato 5.16 viene riportata la funzione che ne gestisce la creazione.

```
@tool
def create_calendar_event(
   title: str,
   start_time: str, # ISO format: "2024-01-15T14:00:00"
   end_time: str,   # ISO format: "2024-01-15T15:00:00"
   attendees: list[str], # email addresses
) -> str:
   """Crea un evento sul calendario Google configurato."""

   calendar_id = os.getenv("GOOGLE_CALENDAR_ID", "primary")
   calendar_tz_name = os.getenv("GOOGLE_CALENDAR_TIMEZONE", "UTC")

   event_body = {
      "summary": title,
      "start": {"dateTime": start_time, "timeZone": calendar_tz_name},
      "end": {"dateTime": end_time, "timeZone": calendar_tz_name},
   }
   if attendees:
      event_body["attendees"] = [{"email": email} for email in attendees]

   try:
      service = _build_google_service("calendar", "v3")
      event = (
         service.events()
         .insert(calendarId=calendar_id, body=event_body, sendUpdates="all")
         .execute()
      )
      link = event.get("htmlLink")
      return f"Evento creato: {title} -> {link}"
   except Exception as exc:
      return f"Errore durante la creazione dell'evento: {exc}"
```

Listato 5.16: Creazione di un evento su Google Calendar

5.3.5 Agente documentation

Questo agente implementa un sistema RAG (Retrieval-Augmented Generation) per rispondere a domande basate sulla documentazione interna.

Definizione

Nel Listato 5.17 viene mostrata la definizione dell'agente documentation. A differenza degli agenti precedenti, questo non richiede il middleware HITL poiché esegue solo operazioni di lettura dalla documentazione.

```
self.documentation_agent = create_agent(
   model=init_chat_model(os.getenv('DOCUMENTATION_AGENT_MODEL'), temperature=0),
   tools=[search_documentation],
   system_prompt=DOCUMENTATION_AGENT_SYSTEM_PROMPT,
   name="documentation_agent"
)
```

Listato 5.17: Definizione dell'agente documentation

Prompt

Il prompt impone all'agente di basare le proprie risposte esclusivamente sui documenti trovati, citandone le fonti.

Nel Listato 5.18 viene riportato il prompt di sistema utilizzato.

```
## CONTEXT
Agente specializzato in ricerca documentale per sistema Bluey. Accesso a: manuale operativo portale
      assicurativo, FAQ aziendali. Utilizzi ricerca semantica (RAG) sui documenti indicizzati tramite tool
      'search_documentation'.

## OBJECTIVE
1. Cercare nei documenti con tool 'search_documentation' formulando una query di ricerca.
2. Sintetizzare risposta citando SEMPRE le fonti (indicare pagina solo se disponibile nei metadati,
      altrimenti "N/A").
3. Ammettere se informazione non presente e suggerire alternative.

Vincoli: MAI inventare informazioni. Solo ricerca documentale, nessun altro task.

## STYLE
Preciso, documentato, strutturato. Procedure in elenchi numerati.

## TONE
Professionale, oggettivo, autorevole.

## AUDIENCE
Operatori back office che consultano procedure e FAQ.

## RESPONSE
Per procedure:
PROCEDURA: [Nome]
1. [Passo 1]
2. [Passo 2]
Fonte: [Documento], Pag. [X / N/A]

Per info non trovata:
Nessun risultato per: "[query]"
Suggerimento: [alternativa]

Esempi (U=user, T=tool_call, R=tool_response, A=assistant):

1. Procedura trovata:
U: "Come registro nuovo cliente?"
T: search_documentation(query="procedura registrazione nuovo cliente anagrafica")
R: {source: "Manuale Operativo", page: 23, content: "Per registrare un nuovo cliente: 1) Menu Anagrafica...
      "}
A: "PROCEDURA: Registrazione cliente. 1. Menu Anagrafica > Nuovo Cliente. 2. Compilare: Nome, CF, Data
      nascita... Fonte: Manuale Operativo, Pag. 23"

2. Info non trovata:
U: "Qual e la franchigia minima per polizze auto?"
T: search_documentation(query="franchigia minima polizze auto importo")
R: {content: "Nessun documento trovato"}
A: "Nessun risultato per: 'franchigia minima polizze auto'. Suggerimento: consultare documenti
      contrattuali o reparto Prodotti."
```

Listato 5.18: Prompt di sistema dell'agente documentation

Tool utilizzati

L'agente utilizza il tool `search_documentation`, che implementa la logica RAG.

Lo strumento `search_documentation` è progettato per eseguire una ricerca semantica all'interno della base di documentazione aziendale. Esso accetta come parametro la `query`, una stringa che rappresenta la domanda o i termini di ricerca necessari per individuare i documenti rilevanti.

Il funzionamento tecnico del tool si articola in quattro fasi principali:

1. *caricamento*: utilizza la classe `PyPDFLoader` per estrarre il testo dai documenti PDF presenti nella directory `docs/`.

2. *splitting*: il testo viene frammentato in chunk di 1000 caratteri con una sovrapposizione di 200 caratteri tramite `RecursiveCharacterTextSplitter`, in modo da preservare il contesto tra le parti adiacenti.

3. *embedding e indicizzazione*: i frammenti vengono convertiti in vettori numerici utilizzando `OpenAIEmbeddings` e indicizzati in un vector store locale basato su `FAISS` (Facebook AI Similarity Search).

4. *retrieval*: esegue una ricerca per similarità restituendo i quattro frammenti più pertinenti rispetto alla query dell'utente. I risultati ottenuti vengono poi forniti al modello linguistico per generare la risposta finale.

Nel Listato 5.19 viene riportata l'implementazione della funzione che crea il vector store a partire dai documenti PDF.

```
def create_vectorstore(archivio_path: str = "./docs"):
    """Crea il vectorstore dai PDF di documentazione."""
    documents = []

    # Caricamento documenti PDF
    pdf_files = glob.glob(os.path.join(archivio_path, "**", "*.pdf"), recursive=True)
    for pdf_file in pdf_files:
        loader = PyPDFLoader(pdf_file)
        docs = loader.load()
        documents.extend(docs)

    # Suddivisione documenti
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=1000,
        chunk_overlap=200
    )
    splits = text_splitter.split_documents(documents)

    # Creazione vectorstore
    embeddings = OpenAIEmbeddings()
    vectorstore = FAISS.from_documents(splits, embeddings)

    return vectorstore
```

Listato 5.19: Creazione del Vector Store per RAG

Nel Listato 5.20 viene riportato il tool di ricerca documentale che utilizza il vector store per recuperare i documenti pertinenti.

```
@tool
def search_documentation(query: str) -> str:
    """
    Cerca nella documentazione del portale assicurativo della compagnia.
    """
    vectorstore = create_vectorstore()
    retriever = vectorstore.as_retriever(search_kwargs={"k": 4})
    docs = retriever.invoke(query)

    if not docs:
        return "Nessun documento trovato per la query specificata."

    # Formatta i risultati
    results = []
    for i, doc in enumerate(docs, 1):
        source = doc.metadata.get("source", "Sconosciuto")
        page = doc.metadata.get("page", "N/A")
        results.append(
            f"--- Documento {i} (Fonte: {source}, Pagina: {page}) ---\n{doc.page_content}"
        )

    return "\n\n".join(results)
```

Listato 5.20: Tool di ricerca documentale

CAPITOLO 6

Analisi del funzionamento degli agenti

In questo capitolo viene analizzato il funzionamento pratico del sistema multi-agente Bluey. Nella prima sezione viene presentata Agent Chat, un'interfaccia web open-source sviluppata da LangChain per l'interazione con agenti LangGraph. Nella seconda sezione, vengono illustrati alcuni esempi di conversazione che dimostrano le capacità del sistema nel gestire richieste complesse, evidenziando la collaborazione tra gli agenti specializzati e il meccanismo di approvazione Human-in-the-Loop.

6.1 Agent Chat

Per testare e interagire con il sistema Bluey è stata utilizzata Agent Chat, un'applicazione web open-source sviluppata da LangChain Inc. e disponibile sulla piattaforma GitHub. L'interfaccia consente di comunicare con qualsiasi server LangGraph che gestisca lo stato dei messaggi, fornendo un servizio di chat completo che include lo streaming in tempo reale delle risposte, la visualizzazione delle chiamate ai tool e il supporto per il meccanismo Human-in-the-Loop.

Agent Chat è uno strumento utile nello sviluppo e nel testing di sistemi basati su agenti, poiché offre un'interfaccia utente standard che nasconde la complessità dell'architettura sottostante. L'utente può conversare con il sistema come se stesse interagendo con un operatore tramite chat. L'applicazione è disponibile sia come progetto da eseguire localmente, sia in una versione già distribuita (accessibile all'indirizzo `agentchat.vercel.app`), consentendo di iniziare a testare gli agenti da subito e senza alcuna installazione. La Figura 6.1 mostra l'interfaccia principale durante un'interazione con il sistema Bluey.

6.1.1 Architettura e funzionamento

L'applicazione è costruita con il framework Next.js e comunica con il backend LangGraph attraverso API REST e connessioni WebSocket per lo streaming in tempo reale delle risposte.

Quando l'utente avvia l'applicazione distribuita su `agentchat.vercel.app`, viene presentato un modulo di configurazione che richiede tre informazioni principali:

- `Deployment URL`: l'indirizzo di rete del server LangGraph con cui comunicare, che può essere un'istanza locale in fase di sviluppo oppure un server di produzione;
- `Assistant/Graph ID`: l'identificativo del grafo o dell'assistente da utilizzare per la gestione delle conversazioni;

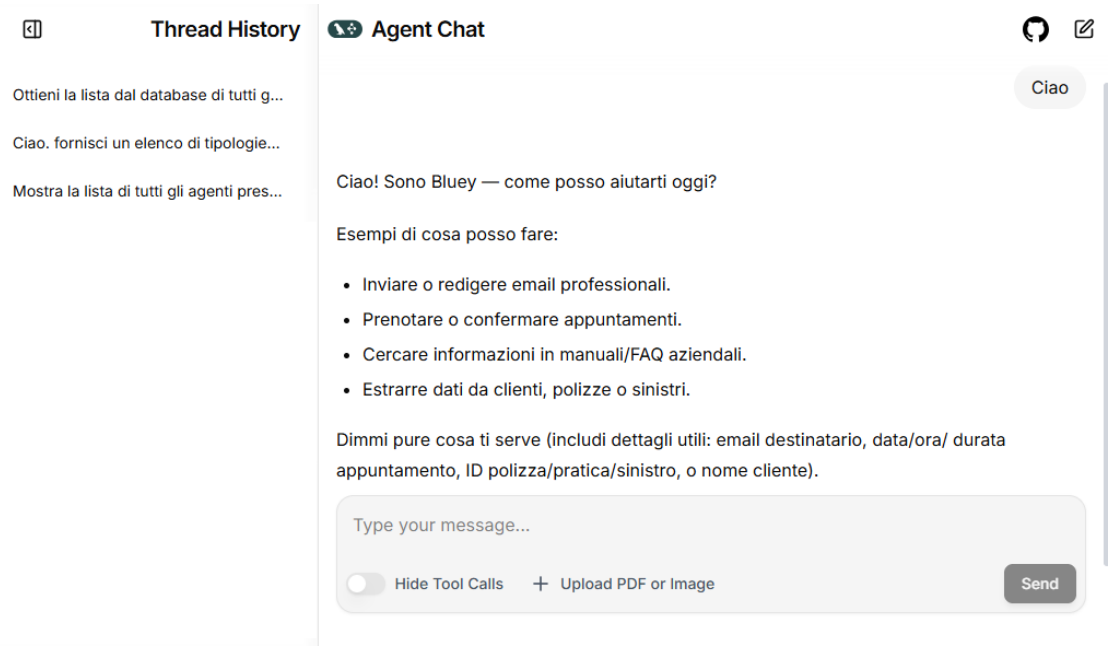


Figura 6.1: Interfaccia di Agent Chat durante un'interazione con il sistema Bluey

- `LangSmith API Key`: la chiave API necessaria per l'autenticazione verso la piattaforma LangSmith.

Nel caso di installazione locale, questi parametri possono essere preconfigurati attraverso variabili d'ambiente, presentando direttamente l'interfaccia di chat all'utente finale.

In fase di test, può essere utile modificare il parametro `Assistant/Graph ID` per connettersi anche ai sub-agenti specializzati, al fine di valutarne il comportamento in isolamento. Tuttavia, nell'uso operativo del sistema Bluey, l'utente interagisce esclusivamente con il grafo principale del Supervisor, che si occupa di orchestrare le comunicazioni con gli altri agenti.

In questo contesto, l'applicazione è stata configurata per connettersi al server di sviluppo locale in esecuzione sulla porta standard di LangGraph.

Essendo un progetto open-source con licenza MIT, Agent Chat può essere personalizzato per adattarsi alle esigenze specifiche dell'applicazione. Ad esempio, è possibile modificare l'aspetto grafico, aggiungere funzionalità specifiche o integrare ulteriori strumenti di monitoraggio.

6.1.2 Funzionalità principali

L'interfaccia fornisce diverse funzionalità utili sia in fase di sviluppo che di utilizzo operativo; queste sono:

- *Chat con streaming in tempo reale*: l'applicazione supporta la comunicazione bidirezionale in tempo reale con gli agenti, permettendo di visualizzare le risposte mentre vengono generate token per token. Questa caratteristica offre un feedback immediato all'utente, migliorando la percezione di reattività del sistema.

- *Visualizzazione delle chiamate ai tool*: l'interfaccia, tramite un apposito comando, mostra nella chat tutte le invocazioni dei tool effettuate dagli agenti, rendendo trasparente il processo decisionale. Questa funzionalità risulta particolarmente utile in fase di debug, poiché consente di comprendere quali strumenti vengono utilizzati per soddisfare ciascuna richiesta e di verificare i parametri.

- *Gestione delle interruzioni Human-in-the-Loop*: quando un agente richiede l'approvazione dell'operatore prima di eseguire un'operazione critica, l'interfaccia presenta un pannello dedicato che consente di approvare, modificare o rifiutare l'azione proposta.
- *Visualizzazione delle tabelle*: l'interfaccia supporta la rappresentazione di dati strutturati attraverso tabelle formattate in `Markdown`, permettendo di presentare risultati complessi in modo ordinato. Questa funzionalità migliora la leggibilità delle informazioni rispetto alla visualizzazione nel normale flusso testuale della conversazione.
- *Controllo della visibilità dei messaggi*: l'applicazione consente di nascondere determinati messaggi prefissando il loro identificativo con la stringa `do-not-render-`, permettendo di scegliere quali messaggi sono più pertinenti da visualizzare e semplificando la conversazione.

6.2 Analisi di esempi di conversazione

Per dimostrare le capacità operative del sistema Bluey, in questa sezione vengono presentati alcuni scenari rappresentativi di utilizzo. Gli esempi selezionati illustrano differenti tipologie di richieste che il sistema è in grado di gestire, dalla semplice consultazione della documentazione alle operazioni complesse che richiedono la collaborazione coordinata di più agenti specializzati.

Ogni esempio evidenzia aspetti distintivi dell'architettura multi-agente: la capacità del Supervisor di comprendere l'intento dell'utente e di delegare le attività agli agenti appropriati, l'interazione tra agenti diversi per completare richieste articolate e il meccanismo di Human-in-the-Loop che garantisce il controllo umano sulle operazioni critiche. Le conversazioni riportate rappresentano interazioni reali con il sistema tramite Agent Chat, mostrando come l'utente percepisca un unico assistente virtuale nonostante la complessità dell'elaborazione interna.

6.2.1 Esempio di richiesta della ricerca della procedura del portale dei clienti

Il primo scenario analizza nel dettaglio come il sistema gestisce una richiesta informativa complessa che necessita della consultazione precisa della documentazione aziendale. L'utente pone una domanda in linguaggio naturale riguardante una procedura operativa specifica. Il Supervisor analizza la richiesta e delega il compito all'agent documentation, il quale esegue una ricerca semantica sui documenti indicizzati attraverso il meccanismo RAG (Retrieval-Augmented Generation) per fornire una risposta precisa e contestualizzata.

Per iniziare la conversazione, l'operatore di back office pone la seguente domanda: *"Come funziona la procedura di rinnovo per le categorie con assicurati? Una volta salvate le modifiche, il rinnovo è automatico?"*.

Il Supervisor intercetta l'intento e attiva il tool `call_documentation_agent`, strutturando, nel parametro `request`, una richiesta dettagliata per il sub-agente. Nello specifico, il Supervisor istruisce l'agente a recuperare e sintetizzare la procedura di rinnovo prevista nei manuali, specificando quattro punti chiave, come mostrato nel Listato 6.1:

```
1. TASK: Recuperare e sintetizzare la procedura di rinnovo per 'categorie con assicurati' (rinnovi a
     livello di categoria) prevista nei manuali aziendali e FAQ. Specificare se il sistema esegue il
     rinnovo automaticamente dopo che vengono salvate modifiche a una categoria.
2. CONTESTO: Operatore back office chiede procedura operativa e comportamenti del sistema relativi al
     rinnovo delle categorie che contengono assicurati. Necessario indicare passaggi, condizioni che
     attivano il rinnovo, timing, notifiche inviate, eccezioni e come eseguire un rinnovo manuale.
3. DATI NOTI: Richiesta generica su 'categorie con assicurati'; non e' stato fornito identificativo
     pratiche, polizze o esempio specifico.
```

```
4. OUTPUT ATTESO: Sintesi chiara e strutturata (massimo 600 parole) con: - Passaggi operativi per il
      rinnovo di una categoria; - Risposta esplicita alla domanda 'il rinnovo e' automatico dopo il
      salvataggio delle modifiche?'; - Eventuali condizioni che impediscono il rinnovo automatico; - Dove
      verificare lo stato del rinnovo (schermate/percorsi nel portale); - Come forzare il rinnovo/manuale;
      - Riferimenti a sezioni del manuale o ID FAQ e link interni se disponibili.
```

Listato 6.1: Richiesta del Supervisor all'agente documentation

L'agente documentation invoca, quindi, il tool `search_documentation` utilizzando la query "rinnovo categoria con assicurati rinnovi a livello di categoria".

Il sistema di retrieval restituisce quattro sezioni di informazioni rilevanti estratte dal manuale del portale clienti, contenenti dettagli sulle schermate, le operazioni da eseguire e le varie eccezioni. Un estratto dei risultati del tool è mostrato nel Listato 6.2.

```
--- Documento 1 (Fonte: docs/Manuale.pdf, Pagina: 76) ---
6.4.2 Rinnovo Categorie con Assicurati
Per le categorie che prevedono al loro interno assicurati sara' possibile visualizzare l'elenco degli
      assicurati di categoria presenti in polizza
[...]
Cliccando invece sul tasto Modifica si accede ad una schermata che permette di:
- Aggiungere nuove anagrafiche
- Escludere anagrafiche

--- Documento 2 (Fonte: docs/Manuale.pdf, Pagina: 79) ---
- Solo dopo aver confermato gli attributi per tutte le categorie presenti in polizza sara' possibile
      rinnovare la polizza tramite pulsante "Rinnova Polizza" in alto a destra.
[...]
```

Listato 6.2: Estratto dei risultati della ricerca documentale

L'agente documentation elabora questi frammenti e restituisce al Supervisor una risposta organizzata in sezioni logiche come mostrato nel Listato 6.3, evidenziando che il rinnovo non è automatico dopo il salvataggio delle modifiche, ma richiede un'azione esplicita di conferma generale.

```
PROCEDURA: Rinnovo Categorie con Assicurati (rinnovi a livello di categoria)

Accesso e preparazione
Accedere alla schermata Rinnovo Categorie con Assicurati e verificare l'elenco degli assicurati presenti
      in ciascuna categoria. Nota: per le polizze RSM sono disponibili anche i familiari associati. (Fonte:
       Manuale portale clienti, 6.4.2; Pag. 76; Pag. 66)

[...]

Condizioni per il rinnovo complessivo e timing
Una volta confermate le attribuzioni per tutte le categorie presenti in polizza, sara' possibile procedere
       al rinnovo della polizza. Mutuando dall'interfaccia, l'azione di rinnovo avviene tramite pulsante
      Rinnova Polizza (in alto a destra). (Fonte: Manuale portale clienti, Pag. 79)

[...]
```

Listato 6.3: Risposta dell'agente documentation al Supervisor

Infine, il Supervisor rielabora queste informazioni per fornire all'utente una risposta chiara e diretta. La risposta finale, illustrata nel Listato 6.4, inizia con una risposta breve ("No - il rinnovo non è automatico"), seguita dai passaggi operativi puntuali per eseguire la procedura correttamente, dalle condizioni che impediscono il rinnovo automatico e dai riferimenti alle sezioni del manuale consultate. In questo flusso, l'agente documentation opera in autonomia senza richiedere approvazione umana, in quanto l'azione è di pura consultazione e non comporta modifiche allo stato del sistema.

```
Ho verificato la procedura nei manuali interni. Sintesi operativa e risposta esplicita alla tua domanda.

1) Risposta breve
- No - il rinnovo non èautomatico dopo il salvataggio delle modifiche a una singola categoria. Dopo aver
      salvato/confermato le attribuzioni per tutte le categorie presenti in polizza, ènecessario avviare
      manualmente l'azione di rinnovo (pulsante Rinnova Polizza / Rinnova ora).

2) Passaggi operativi per il rinnovo (procedura standard)
- Accedi a "Rinnovo Categorie con Assicurati".
- Verifica l'elenco assicurati per ciascuna categoria; per RSM includere i familiari associati.
- Per ogni categoria: seleziona l'assicurato, apporta modifiche (aggiungi/escludi anagrafiche) e usa il
      pulsante Conferma per salvare la categoria.
- Dopo aver confermato tutte le categorie della polizza: clicca Rinnova Polizza (o Rinnova ora, a seconda
      della release) per avviare il rinnovo complessivo.
```

```
- Controlla il messaggio di esito che conferma l'avvenuto rinnovo.

3) Condizioni che impediscono il rinnovo automatico / eccezioni
- Non esiste un rinnovo automatico dopo il salvataggio di singole modifiche: il sistema richiede l'azione
    esplicita di rinnovo a livello di polizza.
- Rami Vita e RSM: non supportano il rinnovo di "categorie cumulative" (solo parametro "Assicurati con
    anagrafica"); ciò limita alcune modalità di rinnovo per questi rami.
- Se l'esito ènegativo, la documentazione non specifica altre notifiche automatiche oltre al messaggio di
    errore/esito: in tali casi seguire procedure di troubleshooting indicate nel manuale o aprire
    segnalazione tecnica.

4) Dove verificare lo stato del rinnovo (percorsi interni)
- Schermata: Rinnovo Categorie con Assicurati (dopo l'azione di rinnovo appare un messaggio di esito
    positivo/negativo).
- La documentazione non indica una schermata aggiuntiva specifica per lo storico dei rinnovi oltre all'
    esito immediato; per controlli a livello di polizza verificare la sezione gestioni/rinnovi polizza
    nel portale (consultare manuale se servono i percorsi esatti per la versione).

5) Come forzare il rinnovo / procedura manuale
- Completa la conferma per tutte le categorie.
- Clic su Rinnova Polizza o Rinnova ora (azione manuale esplicita richiesta).
- Verifica l'esito visualizzato. Se necessario, ripetere o aprire segnalazione se l'operazione non va a
    buon fine.

6) Notifiche e output del sistema
- Documentazione riporta solo il messaggio di esito (positivo/negativo) al termine dell'operazione. Non
    sono documentate altre notifiche automatiche inviate agli assicurati o al management in questa
    procedura.

7) Riferimenti documentali
- Manuale Portale Clienti - Versione 1.4, Sez. 6.4.2 Rinnovo Categorie con Assicurati; Pag. 76
- Manuale Portale Clienti - Versione 1.4, Sez. 6.5.2 Rinnovo Categorie con Assicurati; Pag. 79
- Manuale Portale Clienti - Versione 1.4, Sez. 6.1.2 e note RSM/Vita; Pag. 66

Vuoi che:
- ti indichi il percorso esatto nel portale per la versione che usi (mi serve la release/ambiente), oppure
- controlli lo stato di rinnovo per una polizza/pratica specifica (fornisci ID polizza o pratica)?
```

Listato 6.4: Risposta del Supervisor all'utente

L'utilizzo di LangSmith permette di analizzare la sequenza e il flusso di informazioni tra i tool e i diversi agenti, monitorando i parametri scambiati tra di essi. Per la conversazione esaminata, tale flusso è illustrato nella Figura 6.2.

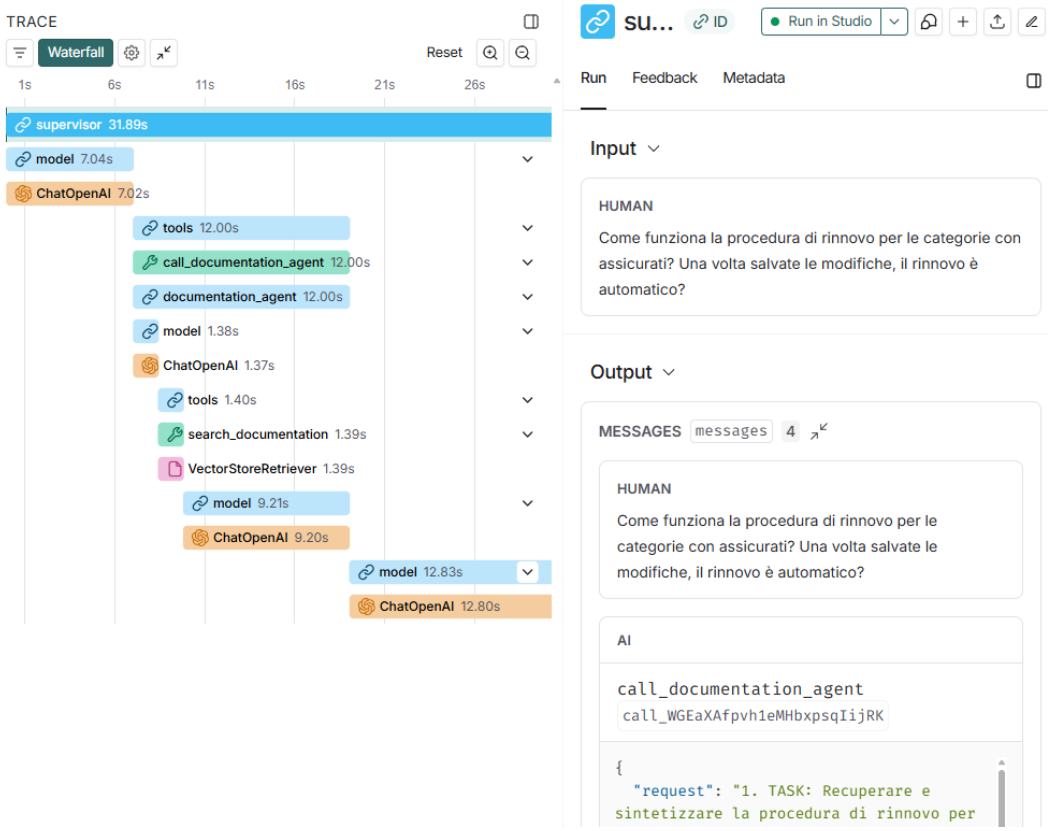


Figura 6.2: Flusso temporale della conversazione in esame

6.2.2 Esempio di verifica dello stato delle polizze e reportistica

Il secondo esempio riguarda l'automazione di compiti di monitoraggio e reportistica. L'utente richiede un controllo su dati presenti nel database e una successiva azione sui risultati ottenuti.

L'utente inizia la conversazione con la seguente richiesta: *"Esporta su file Excel una tabella che contenga tutti i record delle auto con polizze attive. Esporta solo i campi targa e anno di immatricolazione. Il file esportato si deve chiamare 'esportazione_polizze_attive'."*

Il Supervisor identifica l'intento e attiva l'agente SQL per interrogare il database. Per fare ciò, costruisce la richiesta mostrata nel Listato 6.5.

```
1. TASK: Estrarre dal database tutti i record di veicoli (auto) associati a polizze attive.
2. CONTESTO: Operatore di back office richiede esportazione di targa e anno di immatricolazione per
     tutte le auto con polizze attive.
3. DATI NOTI: Non ci sono identificativi specifici. Definizione da utilizzare per 'polizza attiva':
     polizza.stato = 'ATTIVA' oppure (polizza.data_inizio <= oggi AND (polizza.data_fine IS NULL OR
     polizza.data_fine >= oggi)). Considerare anche polizze con stato equivalente in maiuscolo/
     minuscolo.
4. OUTPUT ATTESO: Lista unica (no duplicati) di righe con i campi: targa, anno_immatricolazione.
     Ordinare per targa ascendente. Restituire i risultati in formato tabellare (CSV o JSON) e
     includere il conteggio totale dei record. Indicare eventuali righe escluse per dati mancanti (es.
     targa NULL).
```

Listato 6.5: Richiesta del Supervisor all'agente SQL

L'agente SQL, ricevute le indicazioni, procede con le rispettive fasi di ottenimento dello schema delle tabelle di interesse, di generazione, di verifica e di esecuzione della query. Queste fasi possono essere ispezionate da LangSmith, come illustrato in Figura 6.3.

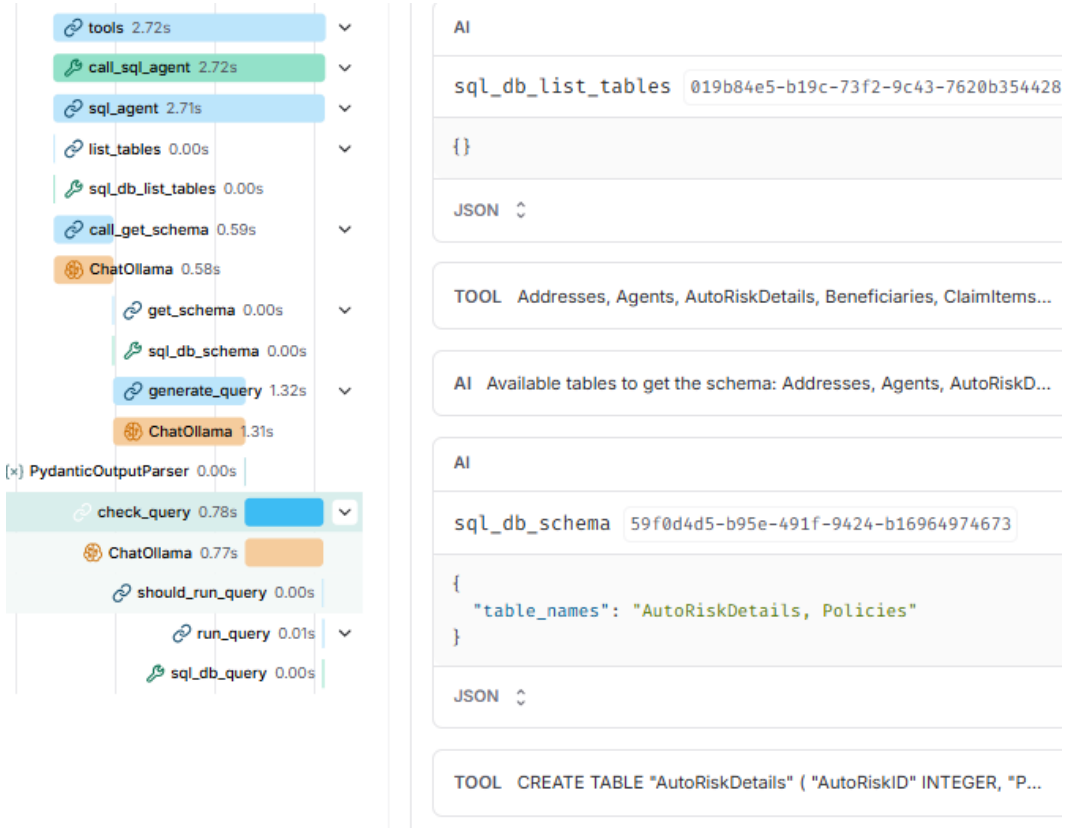


Figura 6.3: Flusso di lavoro per l'agente SQL

Una volta che l'agente ha deciso di eseguire la query, richiede l'approvazione dell'operatore tramite il meccanismo Human-in-the-Loop. L'utente riceve in chat la richiesta di conferma

visualizzata in Figura 6.4, dove può confermare o negare l'esecuzione della query fornendo un feedback per poterla rigenerare.

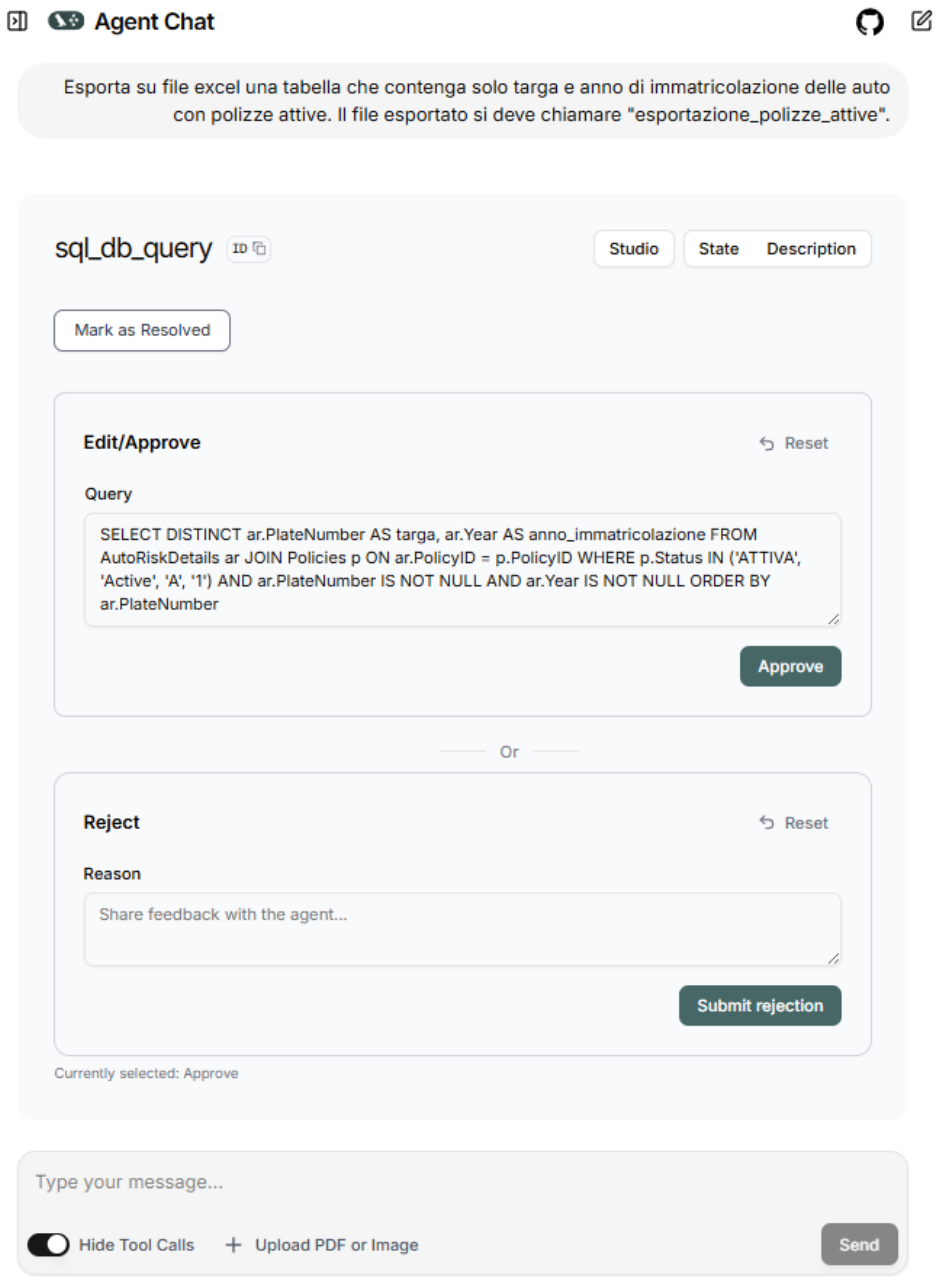


Figura 6.4: Richiesta di approvazione per esecuzione query SQL

Approvando l'esecuzione, i risultati della query vengono inviati all'agente Supervisor che analizza se soddisfano la richiesta iniziale dell'utente e determina se è necessario proseguire con ulteriori azioni. In questo caso, poiché è stata richiesta l'esportazione in un foglio Excel, il Supervisor invoca direttamente il tool `export_to_excel` per eseguire l'operazione e fornire all'utente il file, come mostrato nella Figura 6.5.

Questo esempio dimostra il comportamento dell'agente SQL in un flusso ben strutturato e la capacità del Supervisor di orchestrare sequenzialmente le operazioni da effettuare.

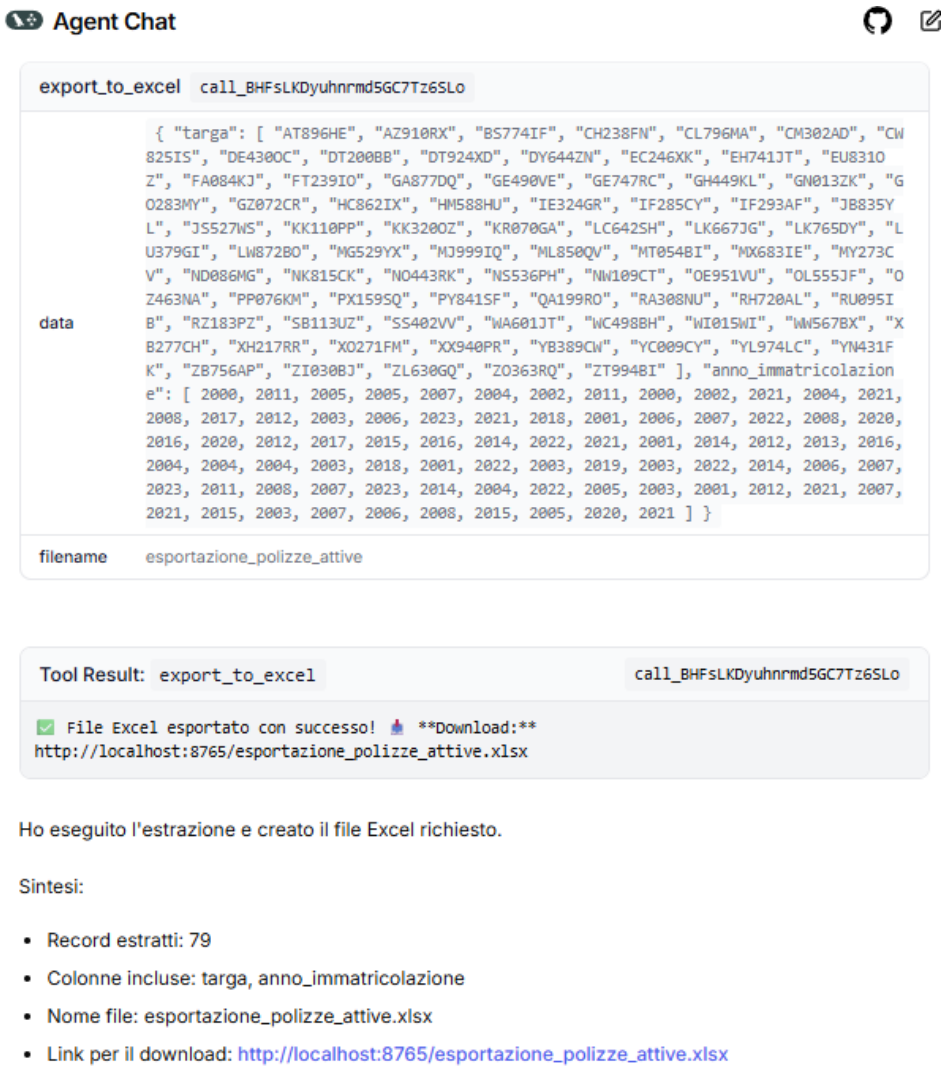


Figura 6.5: Risposta finale da parte del Supervisor con il file Excel

6.2.3 Esempio di richiesta della riunione con il cliente per il numero della polizza

Il terzo esempio illustra, infine, una collaborazione complessa tra più agenti per soddisfare una richiesta che coinvolge il recupero di dati, la gestione del calendario e la comunicazione via email. L'utente richiede di fissare un appuntamento partendo esclusivamente dal numero di polizza, senza fornire i contatti del cliente.

Idealmente, il flusso di esecuzione coordinato dal Supervisor si articola nelle seguenti fasi:

1. attivazione dell'agente SQL per interrogare il database e recuperare i contatti (nome ed email) del cliente associato al numero di polizza fornito;
2. attivazione dell'agente appointment per verificare la disponibilità nel calendario e creare l'evento;
3. attivazione dell'agente email per preparare una email di comunicazione da inviare al cliente.

L'operatore avvia la conversazione con la seguente richiesta: *"Fissa una riunione di 30 minuti per dei chiarimenti per la copertura assicurativa con il titolare della polizza numero 'POL-*

9201-yubw' per martedì prossimo alle ore 10. Scrivigli, poi, una mail per comunicargli che la riunione è stata fissata per dare seguito alla sua richiesta pervenuta tramite servizio clienti e che riceverà il link della riunione su google meet circa un'ora prima."

Il Supervisor attiva l'agente SQL formulando come richiesta al tool l'intento di ottenere i dati di contatto del cliente della polizza in oggetto. L'agente SQL, dopo aver analizzato la struttura delle tabelle, formula la query necessaria. Una volta che questa è stata approvata dall'operatore ed eseguita, il Supervisor riceve i dati di contatto del cliente nel risultato del tool, come mostrato nella Figura 6.6.

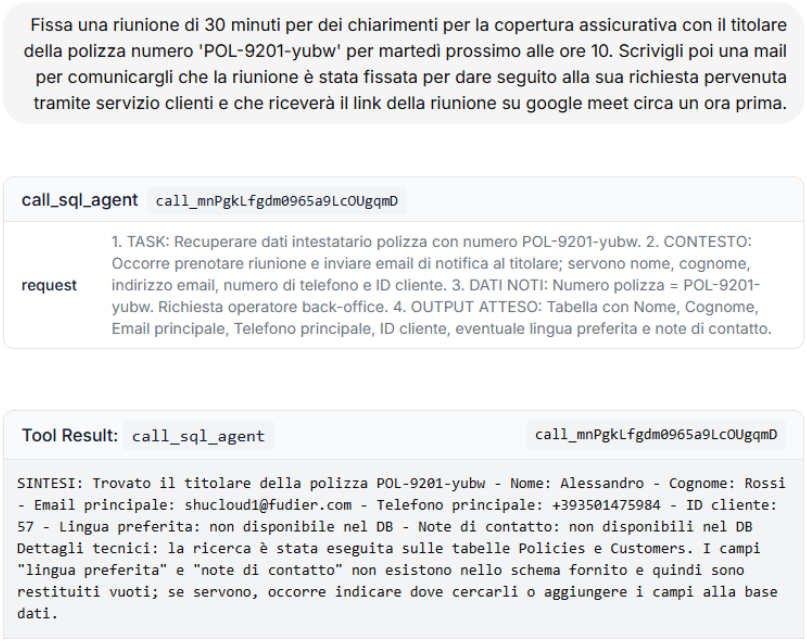


Figura 6.6: Invocazione dell'agente SQL tramite tool mostrata in chat

Il Supervisor procede, quindi, nella gestione della richiesta, delegando il compito successivo all'agente appointment. Nello specifico, l'agente riceve l'istruzione di fissare una riunione per il martedì successivo alle ore 10:00. Come mostrato nella Figura 6.7, l'agente utilizza il tool di gestione del calendario per verificare la disponibilità e proporre la creazione dell'evento. L'interfaccia mostra i parametri precisi passati al tool, inclusi l'orario di inizio e di fine (calcolato sulla base della durata di 30 minuti).

Concluso l'inserimento dell'appuntamento, il Supervisor coordina l'ultima fase del processo, ovvero l'invio della comunicazione al cliente. Viene attivato l'agente email, al quale vengono passati tutti i dettagli necessari per la composizione del messaggio. La Figura 6.8 illustra la chiamata al tool di invio email: è stato utilizzato l'indirizzo email del destinatario (ottenuto nel primo passaggio dal database) ed è stato generato il corpo del messaggio includendo i dettagli dell'appuntamento come richiesto dall'operatore.

A conclusione del flusso operativo, la Figura 6.9 mostra l'esito dal punto di vista del cliente: l'email è stata correttamente recapitata nella casella di posta, completa di tutte le informazioni generate automaticamente dal sistema.

Anche in questo scenario il meccanismo Human-in-the-Loop interviene in ben tre punti distinti dell'elaborazione (esecuzione della query, creazione dell'appuntamento, invio della email), garantendo che l'operatore mantenga il controllo delle operazioni selezionate.

Questo esempio di conversazione dimostra la capacità del Supervisor di comprendere il contesto e di orchestrare sequenzialmente gli agenti specializzati, passando le informazioni ottenute da un agente a quello successivo.

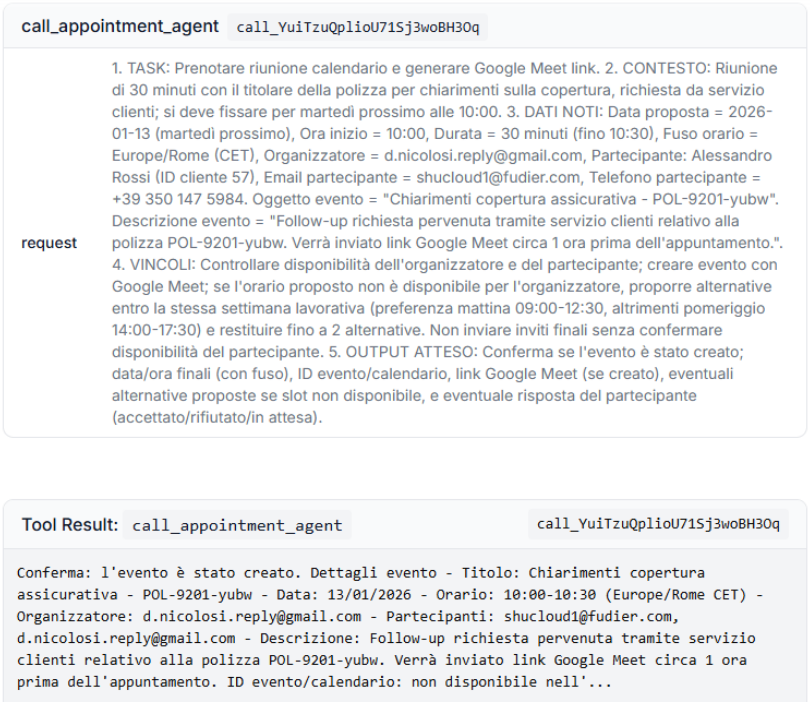


Figura 6.7: Invocazione dell'agente appointment tramite tool mostrata in chat

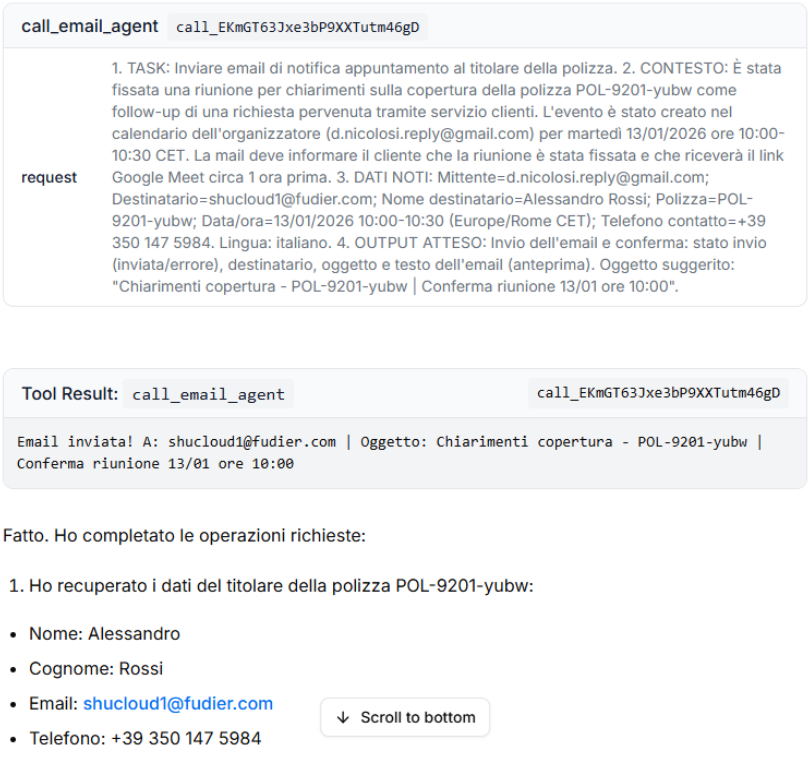


Figura 6.8: Invocazione dell'agente email tramite tool mostrata in chat

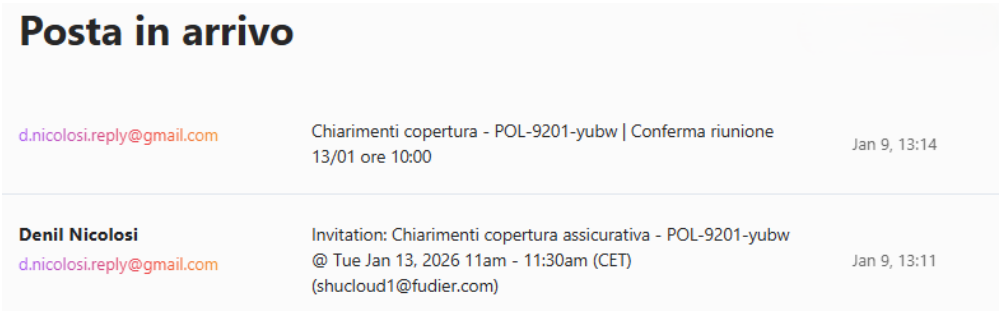


Figura 6.9: Visualizzazione delle email ricevute dal cliente

CAPITOLO 7

Valutazione e benchmark

In questo capitolo viene descritta la metodologia di valutazione adottata per misurare le prestazioni del sistema multi-agente Bluey. Innanzitutto, viene introdotto il paradigma LLM-as-a-judge, che consente di valutare le risposte del sistema utilizzando un modello linguistico come giudice automatico. Vengono, quindi, presentate le metriche definite per valutare aspetti chiave, quali la gestione dell'ambiguità, l'orchestrazione degli agenti, il soddisfacimento delle richieste, l'assenza di allucinazioni e la preservazione del contesto. Infine, viene descritta la struttura del dataset di test e la procedura di esecuzione delle valutazioni tramite la piattaforma LangSmith.

7.1 Premessa

La valutazione delle prestazioni rappresenta, come in ogni processo di sviluppo software, un elemento essenziale per garantire la qualità del prodotto finale. A differenza delle applicazioni software tradizionali, dove i test possono basarsi su confronti deterministici tra output attesi e output prodotti, i sistemi LLM generano risposte in linguaggio naturale che possono essere corrette pur variando nella forma e nel contenuto. Tale caratteristica rende inadeguati i tradizionali approcci di testing basati su corrispondenze.

Nel contesto dei sistemi multi-agente, la complessità della valutazione aumenta ulteriormente. Oltre alla qualità delle singole risposte, è necessario valutare aspetti architetturali quali la corretta selezione degli agenti specializzati, il passaggio delle informazioni contestuali tra i componenti e la capacità di gestire richieste che richiedono la collaborazione di più agenti.

Per affrontare queste sfide, si utilizza il paradigma LLM-as-a-judge, nel quale un modello linguistico viene impiegato come valutatore automatico delle risposte generate da un altro sistema LLM. Questo approccio consente di valutare aspetti qualitativi delle risposte, come la correttezza, la completezza e la pertinenza, che sarebbero difficilmente misurabili con metodi tradizionali su grandi dataset di test.

I test verranno eseguiti tramite la piattaforma LangSmith, che fornisce funzionalità native per la definizione di valutatori (grazie alla libreria open-source `OpenEvals`), la gestione dei dataset e l'aggregazione dei risultati.

7.2 Metodologia di valutazione

Il paradigma LLM-as-a-judge si basa sull'utilizzo di un modello LLM per valutare le risposte in linguaggio naturale generate dal sistema. Il modello valutatore riceve in ingresso la domanda originale, la traccia di conversazione completa di risposte degli agenti e le invocazioni dei diversi tool. Sulla base di queste informazioni, esso produce un giudizio strutturato con un punteggio numerico e una spiegazione in linguaggio naturale della valutazione.

Questa metodologia presenta diversi vantaggi rispetto alla valutazione umana tradizionale:

- *scalabilità*: un valutatore LLM può analizzare migliaia di risposte in tempi brevi, mentre la valutazione umana richiede risorse significative;
- *ripetibilità*: a parità di prompt di valutazione, il modello produce giudizi consistenti, riducendo la variabilità introdotta da valutatori umani differenti;
- *granularità*: è possibile definire tanti singoli criteri di valutazione dettagliati e specifici per ciascun aspetto del sistema da misurare;
- *monitoraggio continuo*: le valutazioni possono essere eseguite online in tempo reale sulle conversazioni in ambiente di produzione, consentendo di mantenere il sistema costantemente monitorato e di rilevare tempestivamente eventuali degradazioni delle prestazioni.

Tuttavia, il paradigma LLM-as-a-judge presenta anche alcune limitazioni che è necessario considerare:

- *bias ereditati*: i modelli valutatori possono ereditare bias presenti nei dati di addestramento, tendendo, ad esempio, a preferire risposte più lunghe o formulate in determinati stili;
- *limiti nelle valutazioni specialistiche*: per compiti che richiedono conoscenze specialistiche o giudizi etici complessi, l'accuratezza del valutatore automatico può essere inferiore a quella di un esperto umano;
- *difficoltà di validazione*: verificare la correttezza del valutatore stesso risulta complesso, poiché non esiste un riferimento oggettivo per giudicare se i punteggi assegnati siano appropriati senza ricorrere, a sua volta, a una valutazione umana.

Per mitigare queste limitazioni e ottenere risultati statisticamente significativi, la metodologia adottata prevede:

1. la creazione di un dataset di richieste da sottoporre al sistema (creato manualmente secondo i requisiti funzionali da rispettare);
2. la ripetizione di ogni richiesta per un certo numero di volte, generando interazioni differenti;
3. la gestione automatica di ogni traccia di conversazione, simulando un utente che interagisce con il sistema grazie ad un ulteriore LLM che genera risposte plausibili in modo coerente con la richiesta iniziale;
4. la valutazione dell'intera conversazione tramite un valutatore LLM-as-a-judge, anch'esso eseguito più volte per ogni traccia prodotta.

Introdurre un numero di ripetizioni sia in fase di generazione delle tracce, che in fase di valutazione, permette di mitigare gli effetti della natura stocastica dei modelli linguistici; calcolando metriche aggregate, quali la media e la deviazione standard dei punteggi, è possibile distinguere i comportamenti sistematici del sistema dalle variazioni casuali, ottenendo, così, risultati statisticamente significativi.

7.2.1 Definizione delle metriche di valutazione

La progettazione di valutatori efficaci richiede istruzioni chiare e non ambigue, specificando esattamente quali aspetti della risposta devono essere valutati e quali criteri applicare. La scala di valutazione deve essere appropriata al tipo di metrica: per aspetti binari è preferibile una scala binaria, mentre per aspetti che ammettono gradazioni può essere utile una scala numerica.

Per valutare in modo completo le prestazioni del sistema Bluey, sono state definite cinque metriche complementari, ciascuna focalizzata su un aspetto specifico del comportamento del sistema multi-agente.

Per la creazione dei valutatori LLM-as-a-judge è stata utilizzata la libreria `openevals`, sviluppata sempre da LangChain, che fornisce funzioni predefinite per la costruzione di valutatori. In particolare, la funzione `create_llm_as_judge` consente di definire un valutatore specificando il prompt di valutazione, il modello giudice da utilizzare e la scala di punteggio.

La scala di valutazione adottata è una scala numerica intera da 1 a 5. Questa scelta deriva dalle linee guida comuni per i valutatori LLM-as-a-judge: le scale numeriche discrete risultano più affidabili rispetto alle scale continue, poiché riducono l'ambiguità nell'interpretazione dei criteri da parte del modello giudice. La scala a 5 punti rappresenta un compromesso ottimale tra granularità della valutazione e consistenza dei giudizi.

Tutti i valutatori condividono un prompt di sistema comune che fornisce il contesto generale al modello giudice, in modo da garantire uniformità nella valutazione e ridurre la ridondanza nei prompt specifici di ogni metrica. Nel Listato 7.1 viene riportato il prompt di sistema comune utilizzato.

```
Sei un valutatore esperto per sistemi AI multi-agente.
Il tuo compito e' valutare le conversazioni tra utenti e un sistema AI
chiamato Bluey, utilizzato in un contesto assicurativo per assistere
operatori di back-office.

Il sistema Bluey e' composto da:
- Un agente supervisor che coordina le richieste
- Un agente 'documentation_agent': che esegue operazioni di ricerca nella documentazione aziendale (un
      manuale del portale dei clienti e un documento di FAQ)
- Un agente 'sql_agent': che esegue query sul database della compagnia che contiene dati su clienti/
      polizze/sinistri
- Un agente 'email_agent': che si occupa della composizione e dell'invio di email
- Un agente 'appointment_agent': che gestisce gli appuntamenti nel calendario (verifica disponibilità e
      creazione degli eventi)

Scala di valutazione: rispondi sempre con un singolo numero intero
da 1 a 5.
```

Listato 7.1: Prompt di sistema comune per tutti i valutatori

Di seguito vengono descritte le cinque metriche implementate, ciascuna con il relativo prompt di valutazione.

Completamento della richiesta

La metrica `request_fulfilled` rappresenta la misura principale di successo del sistema, valutando se la risposta fornita soddisfa effettivamente la richiesta dell'utente. Questa metrica considera non solo la correttezza formale della risposta, ma anche la sua completezza e utilità pratica.

Il valutatore confronta la risposta del sistema con l'obiettivo espresso nella richiesta originale, verificando che tutti gli aspetti della domanda siano stati affrontati. Per le richieste che prevedono azioni, quali l'invio di email o la creazione di appuntamenti, il valutatore verifica che l'azione sia stata completata o che il sistema abbia fornito una spiegazione valida dell'impossibilità di procedere. Nel Listato 7.2 viene riportato il prompt di valutazione.

```
Valuta se la richiesta dell'utente e' stata completata correttamente, fornendo le informazioni complete.
      Se la domanda prevede un'azione, verifica che l'azione sia stata completata o che il sistema abbia
      fornito una spiegazione valida dell'impossibilità di procedere.

Criteri:
- 1: Richiesta non affrontata o risposta completamente errata
- 2: Tentativo con errori significativi o informazioni mancanti
- 3: Parzialmente soddisfatta, alcune informazioni fornite ma non sufficienti
- 4: Ben soddisfatta con piccole omissioni
- 5: Completamente e correttamente soddisfatta

Richiesta originale:
{inputs}

Conversazione:
{outputs}
```

Listato 7.2: Prompt per la valutazione del completamento della richiesta

Il valutatore viene, quindi, creato tramite la funzione `create_llm_as_judge` della libreria `openevals`. Come mostrato nel Listato 7.3, la funzione riceve il prompt di valutazione, il prompt di sistema comune, una chiave identificativa per la metrica, il modello LLM da utilizzare come giudice e la lista dei punteggi ammessi. Questa struttura di definizione viene replicata per tutte le metriche successive, variando unicamente il prompt specifico e la chiave identificativa.

```
request_fulfilled_evaluator = create_llm_as_judge(
   prompt=REQUEST_FULFILLED_PROMPT,
   system=SYSTEM_PROMPT,
   feedback_key="request_fulfilled",
   judge=JUDGE_LLM,
   choices=SCORE_CHOICES,
)
```

Listato 7.3: Definizione del valutatore per il completamento della richiesta

Assenza di allucinazioni

La metrica `groundedness` valuta quanto le risposte del sistema siano supportate dai dati disponibili, rilevando eventuali allucinazioni. Un'allucinazione è definita come un'informazione inventata, non supportata da fonti e dal contesto della conversazione o che contraddice quanto già discusso.

Questa metrica è importante per valutare l'affidabilità delle risposte fornite dagli agenti che si occupano di consultare il database e la documentazione aziendale, che devono basare le proprie risposte esclusivamente sui contenuti disponibili. Il valutatore verifica che le informazioni fornite siano effettivamente presenti nelle fonti consultate (e quindi nelle risposte dei rispettivi tool) e che non vengano aggiunti dettagli inventati. Nel Listato 7.4 viene riportato il prompt di valutazione.

```
Valuta quanto le risposte del sistema sono ancorate ai fatti disponibili.
Un'allucinazione e' un'informazione inventata, non supportata dal contesto della conversazione o che
      contraddice quanto gia' discusso.

Considera come contesto:
- I risultati delle invocazioni dei tool
- Le informazioni presenti nella conversazione

Criteri:
- 1: Informazioni totalmente inventate o che contraddicono il contesto
- 2: Diverse affermazioni non supportate dal contesto
- 3: Alcune imprecisioni o generalizzazioni non verificabili
```

```
- 4: Risposte coerenti con piccole imperfezioni
- 5: Tutte le informazioni derivano dal contesto, nessuna invenzione

Richiesta originale:
{inputs}

Conversazione:
{outputs}
```

Listato 7.4: Prompt per la valutazione dell'assenza di allucinazioni

Gestione dell'ambiguità

La metrica `ambiguity_handling` valuta la capacità del sistema di gestire richieste ambigue, incomplete o poco chiare. Un sistema ben progettato, anziché procedere con assunzioni potenzialmente errate, dovrebbe riconoscere l'ambiguità e richiedere all'utente i chiarimenti necessari.

Questa metrica è significativa per l'agente Supervisor, in quanto esso deve decidere se dispone di informazioni sufficienti per delegare una richiesta agli agenti specializzati o se è necessario interrogare l'utente. Ad esempio, una richiesta del tipo "invia una email al cliente" è ambigua perché manca l'indirizzo del destinatario e il contenuto del messaggio. Nel Listato 7.5 viene riportato il prompt di valutazione.

```
Valuta come l'agente ha gestito richieste potenzialmente ambigue.

Un sistema ben progettato dovrebbe:
- Riconoscere richieste ambigue o incomplete
- Chiedere chiarimenti invece di fare assunzioni errate
- Se la richiesta e' chiara, effettuare l'operazione senza soffermarsi nell'approfondire aspetti banali e
      richiedere conferme trascurabili

Criteri:
- 1: Assunzioni arbitrarie senza chiedere chiarimenti quando necessario
- 2: Riconoscimento parziale con domande incomplete o non pertinenti
- 3: Gestione ragionevole ma migliorabile
- 4: Buona gestione con domande pertinenti, oppure risposta diretta
   se la richiesta e' chiara
- 5: Gestione perfetta: chiarimenti appropriati per richieste ambigue,
   risposta corretta per richieste chiare

Richiesta originale:
{inputs}

Conversazione:
{outputs}
```

Listato 7.5: Prompt per la valutazione della gestione dell'ambiguità

Qualità dell'orchestrazione

La metrica `orchestration` valuta la capacità del Supervisor di selezionare l'agente specializzato appropriato per ciascuna richiesta. Nel contesto dell'architettura multi-agente di Bluey, questa metrica misura se il flusso di elaborazione segue il percorso atteso, coinvolgendo gli agenti corretti.

Il valutatore analizza la traccia di esecuzione della richiesta, verificando che l'agente selezionato corrisponda alla natura della domanda. Per le richieste complesse che comportano la collaborazione di più agenti, il valutatore verifica che tutti gli agenti necessari vengano coinvolti e che la sequenza di attivazione sia logica. Nel Listato 7.6 viene riportato il prompt di valutazione.

```
Valuta la corretta selezione degli agenti specializzati.

RESPONSABILITA' DEGLI AGENTI:
- documentation_agent: ricerca procedure, FAQ, manuali aziendali.
  Usare per: "come si fa...", "qual e' la procedura per...",
  domande su policy
- sql_agent: query database clienti, polizze, sinistri, premi e dati che possono essere presenti in un
      database di una compagnia assicurativa.
  Usare per: dati anagrafici, stato polizze, storico pagamenti
```

```
- email_agent: composizione e invio email a clienti/colleghi.
  Usare per: comunicazioni, solleciti, conferme appuntamenti
- appointment_agent: gestione calendario, prenotazione appuntamenti.
  Usare per: fissare riunioni e verificare disponibilita'.

Criteri:
- 1: Agente completamente sbagliato o omesso quando necessario
- 2: Agente parzialmente appropriato, scelte discutibili
- 3: Selezione ragionevole ma non ottimale
- 4: Buona selezione con piccole imperfezioni
- 5: Selezione perfetta: coinvolti tutti e soli gli agenti necessari

Richiesta originale:
{inputs}

Conversazione (include le chiamate agli agenti):
{outputs}
```

Listato 7.6: Prompt per la valutazione della qualità dell'orchestrazione

Preservazione del contesto

La metrica `context_preservation` valuta la capacità del Supervisor di trasmettere correttamente il contesto conversazionale agli agenti specializzati. Poiché, per scelta progettuale, i sub-agenti non hanno accesso alla cronologia completa della conversazione, è responsabilità del Supervisor includere nelle richieste delegate tutte le informazioni rilevanti.

Il valutatore analizza le richieste formulate dal Supervisor verso i sub-agenti, verificando che contengano i dettagli necessari estratti dal contesto conversazionale. Ad esempio, se l'utente ha menzionato un numero di polizza in un messaggio precedente e successivamente chiede informazioni sul titolare, il Supervisor deve includere il numero di polizza nella richiesta all'agente SQL. Nel Listato 7.7 viene riportato il prompt di valutazione.

```
Valuta la capacita' di preservare il contesto conversazionale.

Nota: I sub-agenti NON hanno accesso alla cronologia completa.
Il Supervisor deve includere nelle richieste delegate tutte le
informazioni rilevanti (nomi, numeri polizza, date, email, ecc.).

Cosa osservare:
- Le chiamate ai tool inviate dal Supervisor agli agenti specializzati
- Verifica che contengano le informazioni rilevanti menzionate
  nella conversazione
- Controlla che nomi, numeri polizza, date, indirizzi email siano
  trasmessi correttamente

Criteri:
- 1: Informazioni critiche perse, dati essenziali non trasmessi
- 2: Diverse informazioni contestuali mancanti o incomplete
- 3: Contesto parzialmente preservato, informazioni minori mancanti
- 4: Buona preservazione con piccole omissioni non critiche
- 5: Contesto perfettamente preservato e trasmesso ai sub-agenti

Richiesta originale:
{inputs}

Conversazione:
{outputs}
```

Listato 7.7: Prompt per la valutazione della preservazione del contesto

7.3 Creazione del dataset di test

Il dataset di test rappresenta l'insieme strutturato di esempi di richieste sui quali il sistema viene valutato. La qualità del dataset risulta determinante per l'affidabilità dei risultati ottenuti; infatti, un dataset non rappresentativo può portare a conclusioni fuorvianti sulle prestazioni del sistema.

Per il sistema Bluey è stato definito manualmente un dataset composto da 30 esempi, selezionati per coprire le diverse tipologie di operazioni supportate dal sistema e derivate dalla definizione dei requisiti funzionali, garantendo una distribuzione equilibrata tra gli agenti specializzati.

Gli esempi del dataset sono stati distribuiti in modo da coprire i principali scenari di utilizzo del sistema. La Tabella 7.1 riporta la distribuzione prevista degli esempi per agente e tipologia.

Tipologia	Numero esempi
Ricerca su manuale	3
Domande su FAQ	3
Query semplici	3
Query complesse	3
Composizione di email	5
Gestione degli appuntamenti	5
Collaborazione tra agenti	4
Richieste ambigue	4
Totale	**30**

Tabella 7.1: Distribuzione degli esempi nel dataset di test

La categoria "richieste ambigue" è stata inserita per valutare la metrica *ambiguity_handling*, includendo esempi dove mancano informazioni essenziali che il sistema dovrebbe richiedere. La categoria "collaborazione tra agenti" include scenari complessi che richiedono la collaborazione sequenziale di più agenti, come l'esempio analizzato nel Capitolo 6 dove una singola richiesta coinvolge tre agenti specializzati differenti.

Nella definizione degli esempi sono stati seguiti criteri metodologici volti a garantire la rappresentatività del dataset; in particolare, le richieste sono formulate nel linguaggio naturale tipico degli operatori e per ciascuna tipologia sono stati inclusi esempi con formulazioni diverse per evitare ripetizioni che potrebbero favorire artificialmente il sistema; sono stati, inoltre, inseriti esempi che testano comportamenti al confine tra categorie, quali richieste che potrebbero essere gestite da più agenti.

7.3.1 Generazione delle tracce di conversazione

Per generare delle tracce distinte e complete di conversazioni che comprendano più iterazioni tra agente e utente, l'esecuzione dei tool e l'approvazione delle richieste HITL, è stato sviluppato lo script `generate_threads_dataset.py`. Per ogni richiesta del dataset di esempi, lo script avvia una nuova conversazione con il sistema multi-agente, gestisce le interazioni multi-turno tramite un utente simulato da un LLM e approva in automatico tutte le richieste che necessitano di revisione.

Al termine di ogni conversazione, i dati vengono salvati nel dataset LangSmith tramite l'SDK dedicato. Questo processo associa automaticamente la traccia di esecuzione completa all'esempio corrispondente, permettendo di ispezionare successivamente tutti i dettagli dell'esecuzione, inclusi le chiamate ai tool e i messaggi scambiati tra gli agenti.

Utente simulato

La valutazione automatizzata di un sistema conversazionale multi-turno richiede la capacità di simulare le risposte dell'utente durante l'interazione. Questo consente di eseguire test ripetibili senza intervento umano, permettendo di generare più volte un numero elevato di conversazioni complete e di raccogliere metriche statisticamente significative.

L'utente simulato è stato implementato come un modello LLM ausiliario che, per ogni messaggio scambiato tra agente e utente, decide se la richiesta iniziale è stata soddisfatta (e si può, quindi, terminare la conversazione) o se è necessario proseguire (generando una

risposta plausibile). Questo approccio consente di testare scenari multi-turno dove il sistema richiede informazioni aggiuntive all'utente, come date, orari o dettagli specifici.

Nel Listato 7.8 viene riportato il prompt utilizzato per istruire il modello LLM che simula l'utente.

```
Sei l'utente simulato che gestisce questa conversazione. Il tuo interlocutore e' Bluey, un assistente AI
      che aiuta gli operatori di backoffice nella loro operativita' quotidiana.

Il tuo compito e' analizzare la conversazione in relazione alla
richiesta iniziale:
1. Se l'assistente ha soddisfatto pienamente la tua richiesta
(es. fornito l'informazione finale, confermato l'azione),
  imposta is_completed=True anche se propone altre attivita'.
2. Solo se l'assistente ti chiede dettagli mancanti per completare
  la richiesta (es. date, orari, codici), imposta is_completed=False
  e genera una risposta.

Istruzioni:
- Rispondi come l'utente, in modo breve e diretto.
- Se servono dati (date, orari, nomi) che non sono nella richiesta
 iniziale, inventali in modo plausibile e coerente.

Stato attuale della conversazione:
{conversation}
```

Listato 7.8: Prompt per la simulazione delle risposte utente

Approvazione automatica delle azioni

Il sistema Bluey implementa un meccanismo di Human-in-the-loop (HITL) che richiede l'approvazione esplicita dell'utente prima di eseguire azioni con effetti esterni, quali l'invio di email, la creazione di appuntamenti nel calendario e l'esecuzione delle query. Durante i test automatizzati, queste richieste di approvazione devono essere gestite senza intervento umano.

Per risolvere questo problema, lo script di generazione implementa un ciclo di auto-approvazione: dopo ogni invocazione del sistema, viene verificata la presenza di interrupt HITL in attesa; se questi vengono rilevati, sono automaticamente approvati e l'esecuzione riprende fino al completamento della conversazione o al raggiungimento di un limite massimo di approvazioni consecutive.

Simulazione delle API esterne

Dato il meccanismo di approvazione automatica, per evitare effetti collaterali reali durante i test, le chiamate alle API esterne di Google, quali Gmail e Calendar, vengono simulate tramite oggetti mock. Quando il sistema invoca un tool per l'invio di email o la creazione di appuntamenti, la chiamata viene intercettata e restituisce una risposta fittizia che simula il successo dell'operazione, senza che alcuna azione venga realmente eseguita. Per quanto riguarda il database, viene utilizzata una replica di esempio con dati sintetici.

La traccia della conversazione registrata su LangSmith include, comunque, tutti i parametri passati ai tool, permettendo di valutare la correttezza degli argomenti, quali destinatari, oggetti e contenuti delle email, anche in assenza di effetti reali.

7.4 Esecuzione delle valutazioni

Per eseguire il processo di valutazione effettivo delle tracce precedentemente generate, viene utilizzato lo script `evaluate_dataset.py`. I valutatori creati con la libreria `openevals` vengono utilizzati con l'SDK di LangSmith che si occupa di eseguire tutti i valutatori sul dataset.

Nel Listato 7.9 viene mostrato il codice che configura e avvia la valutazione. Il parametro `max_concurrency` consente di parallelizzare le valutazioni, riducendo significativamente

il tempo totale di esecuzione. Il parametro `num_repetition` specifica il numero di volte che ogni traccia di conversazione viene valutata: eseguire più ripetizioni permette di mitigare la variabilità stocastica intrinseca dei modelli LLM giudice e ottenere, quindi, metriche aggregate più affidabili.

```
EVALUATORS = [
   create_langsmith_evaluator(request_fulfilled_evaluator),
   create_langsmith_evaluator(groundedness_evaluator),
   create_langsmith_evaluator(ambiguity_handling_evaluator),
   create_langsmith_evaluator(orchestration_evaluator),
   create_langsmith_evaluator(context_preservation_evaluator),
]

experiment_results = evaluate(
   lambda inputs: {},
   data=DATASET_TO_EVALUATE,
   evaluators=EVALUATORS,
   experiment_prefix="bluey-eval-5-metriche",
   max_concurrency=5,
   num_repetition=3
)
```

Listato 7.9: Esecuzione della valutazione con l'API di LangSmith

7.5 Analisi dei risultati

Per valutare le prestazioni del sistema Bluey e analizzare l'impatto della scelta dei modelli linguistici, sono stati condotti due test distinti di generazione e valutazione delle tracce di conversazione.

Il primo test ha utilizzato modelli LLM economici e veloci, caratterizzati da tempi di risposta ridotti e costi contenuti. In particolare, sono stati impiegati modelli della famiglia GPT-5 nano per gli agenti specializzati (email, appuntamenti e documentazione), il modello GPT-5 mini per il Supervisor che coordina l'orchestrazione, e il modello Qwen3-coder per l'agente SQL. Il modello Qwen3-coder è un modello open-weight ottimizzato per la generazione di codice; esso viene eseguito tramite il backend Ollama, una piattaforma che consente di eseguire modelli LLM in una propria infrastruttura server. Nel Listato 7.10 viene riportata la configurazione dei modelli utilizzata.

```
EMAIL_AGENT_MODEL=openai:gpt-5-nano
APPOINTMENT_AGENT_MODEL=openai:gpt-5-nano
SQL_AGENT_MODEL=ollama:qwen3-coder:480b-cloud
DOCUMENTATION_AGENT_MODEL=openai:gpt-5-nano
SUPERVISOR_AGENT_MODEL=openai:gpt-5-mini
```

Listato 7.10: Configurazione dei modelli economici

Il secondo test ha utilizzato, invece, dei modelli più potenti e performanti, impiegando GPT-5 per tutti gli agenti specializzati e per il Supervisor, mentre per l'agente SQL è stato utilizzato GPT-5 Codex (una versione di GPT-5 ottimizzata per la generazione di codice). Questa configurazione, riportata nel Listato 7.11, rappresenta una scelta migliore in termini di prestazioni dei modelli, ma comporta costi significativamente superiori e tempi di risposta più lunghi.

```
EMAIL_AGENT_MODEL=openai:gpt-5
APPOINTMENT_AGENT_MODEL=openai:gpt-5
SQL_AGENT_MODEL=openai:gpt-5-codex
DOCUMENTATION_AGENT_MODEL=openai:gpt-5
SUPERVISOR_AGENT_MODEL=openai:gpt-5
```

Listato 7.11: Configurazione dei modelli performanti

7.5.1 Confronto dei risultati

La Tabella 7.2 riporta i risultati ottenuti con la configurazione di modelli economici. Per ogni metrica sono indicati il punteggio medio e la deviazione standard calcolati sulle ripetizioni effettuate; sono riportati, inoltre, i valori minimi e massimi registrati per evidenziare l'intervallo di variazione dei punteggi.

Metrica	Media*	Dev. std	Min	Max
Request fulfilled	4.61	0.65	1	5
Groundedness	4.81	0.61	1	5
Ambiguity handling	4.96	0.28	2	5
Orchestration	4.84	0.41	1	5
Context preservation	4.56	0.85	1	5
Media complessiva	**4.75**	**0.56**	**1.2**	**5**

*Punteggi espressi su scala 1-5

Tabella 7.2: Risultati del test con modelli economici (GPT-5 nano/mini e Qwen3-coder)

La Tabella 7.3 riporta, invece, i risultati ottenuti con la configurazione di modelli più performanti.

Metrica	Media*	Dev. std	Min	Max
Request fulfilled	4.74	0.58	1	5
Groundedness	4.86	0.52	2	5
Ambiguity handling	4.93	0.25	2	5
Orchestration	4.91	0.35	1	5
Context preservation	4.92	0.38	2	5
Media complessiva	**4.87**	**0.50**	**1.6**	**5**

*Punteggi espressi su scala 1-5

Tabella 7.3: Risultati del test con modelli performanti (GPT-5 e GPT-5 Codex)

7.5.2 Visualizzazione dei risultati in LangSmith

I risultati delle valutazioni possono essere consultati in modo dettagliato sulla piattaforma LangSmith, che offre funzionalità avanzate per l'analisi e l'ispezione dei benchmark eseguiti.

Per ogni esempio valutato, LangSmith visualizza il punteggio assegnato da ciascuna metrica insieme alla motivazione generata dal modello giudice. Questa spiegazione in linguaggio naturale consente di comprendere il ragionamento alla base della valutazione e di identificare eventuali aspetti critici nella risposta del sistema.

Inoltre, quando vengono eseguite più ripetizioni della valutazione sullo stesso esempio, la piattaforma calcola e visualizza la deviazione standard dei punteggi ottenuti. Questo indicatore permette di valutare la consistenza del giudizio: una bassa deviazione standard indica che il modello giudice assegna punteggi stabili e riproducibili, mentre valori elevati possono segnalare ambiguità nei criteri di valutazione o variabilità nella qualità della risposta valutata.

La Figura 7.1 mostra un esempio della dashboard di LangSmith con le metriche aggregate e i dettagli delle singole valutazioni.

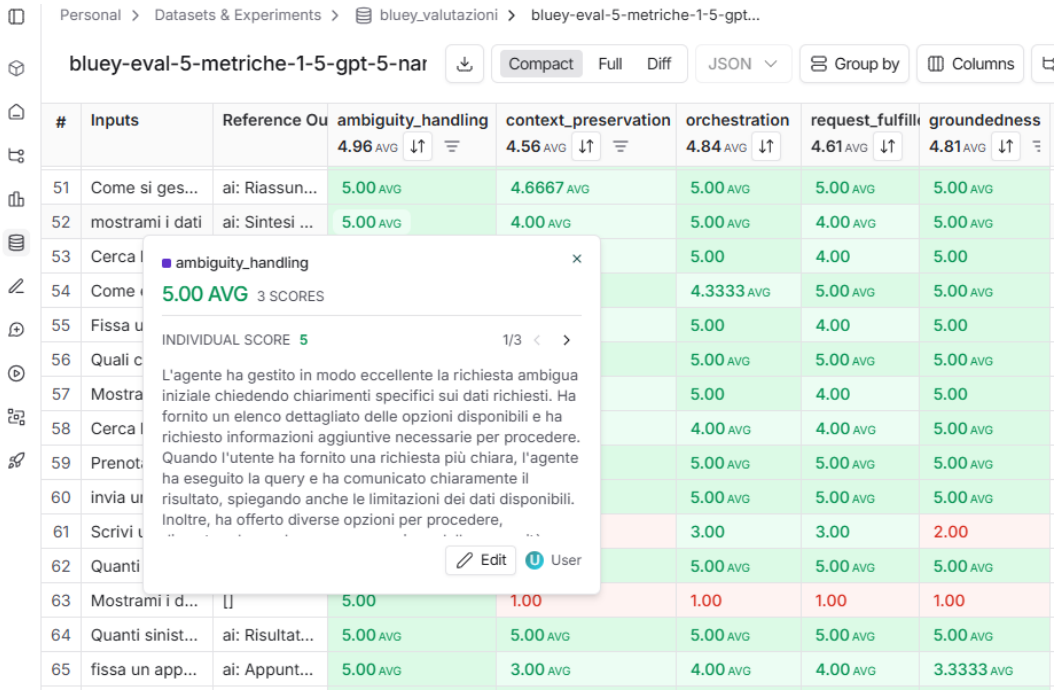


Figura 7.1: Dashboard di LangSmith con le metriche di valutazione aggregate

7.5.3 Discussione

Dal confronto dei risultati emerge che l'architettura multi-agente consente di ottenere prestazioni soddisfacenti anche con modelli meno performanti. La differenza tra le due configurazioni risulta molto contenuta: i modelli più potenti producono un miglioramento medio di circa 0.12 punti su una scala da 1 a 5.

Questo risultato può essere attribuito alle caratteristiche intrinseche dell'architettura multi-agente adottata. In un sistema dove i compiti sono ben identificati e assegnati ad agenti specializzati, ciascun agente deve gestire un dominio circoscritto e ben definito: l'agente email si occupa esclusivamente della composizione e dell'invio di messaggi, l'agente SQL genera query per il database, e così via. Tale specializzazione riduce la complessità richiesta a ciascun modello, permettendo anche a modelli più leggeri di operare efficacemente nel proprio ambito di competenza.

Un ulteriore vantaggio dell'architettura multi-agente risiede nella tolleranza ai fallimenti parziali. Quando un agente specializzato produce un risultato non ottimale, il Supervisor può rilevare l'anomalia e gestirla appropriatamente, ad esempio richiedendo un nuovo tentativo modificando la richiesta. Questo meccanismo di supervisione consente al sistema complessivo di mantenere prestazioni elevate anche quando i singoli componenti presentano occasionali errori.

Al contempo, però, il passaggio ai modelli più performanti comporta costi notevolmente superiori e tempi di risposta significativamente più lunghi. Nel contesto di un sistema destinato all'uso in produzione, dove la velocità di risposta e i costi operativi rappresentano comunque importanti fattori qualitativi, la scelta dei modelli deve bilanciare prestazioni e sostenibilità economica.

La Tabella 7.4 riporta il confronto tra le due configurazioni in termini di latenza media per turno e costo medio per traccia. La latenza per turno si riferisce al TTFT (Time to First Token) dello streaming della risposta finale del sistema all'utente, ovvero il tempo che intercorre tra l'invio della richiesta e la ricezione del primo token della risposta, includendo tutti i passaggi

interni tra Supervisor e agenti specializzati. I dati sono stati ottenuti dalle analisi effettuate dalle tracce generate in fase di valutazione.

Configurazione	Latenza TTFT media per turno	Costo medio per turno
Modelli economici	30 s	$0.005
Modelli performanti	80 s	$0.090
Differenza	+167%	+1700%

Tabella 7.4: Confronto di latenze e costi tra le configurazioni di modelli

Come evidenziato dalla tabella, i modelli performanti comportano un aumento della latenza di quasi tre volte e un incremento dei costi di circa diciotto volte rispetto alla configurazione economica. La Figura 7.2 mostra i listini prezzi dei modelli OpenAI, evidenziando la significativa differenza di costo, di ben 25 volte superiore, dal modello della famiglia nano al modello GPT-5 standard.

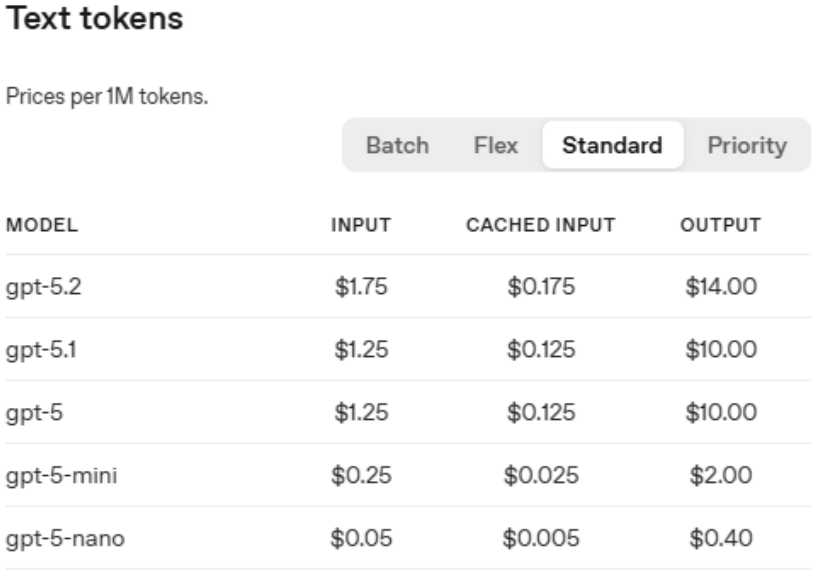


Figura 7.2: Listino prezzi di alcuni modelli OpenAI GPT-5

Alla luce dei risultati ottenuti, l'utilizzo di modelli economici e rapidi risulta vantaggioso per il sistema in esame. Il leggero miglioramento delle metriche offerto dai modelli più potenti, in questo caso d'uso, non giustifica il grande incremento dei costi e dei tempi di elaborazione. Questa conclusione ha anche implicazioni interessanti per la portabilità del sistema: la possibilità di operare con modelli leggeri facilita la migrazione verso modelli locali o self-hosted, riducendo la dipendenza da servizi cloud esterni.

Un esempio concreto di questa flessibilità è rappresentato dall'agente SQL, per il quale è stato utilizzato Qwen3-coder, un modello open-weight ottimizzato per la generazione di codice ed eseguito tramite la piattaforma open-source Ollama. La scelta di un modello locale è stata motivata anche dalla necessità di garantire la privacy dei dati presenti nel database aziendale; infatti, nelle operazioni di generazione delle query SQL, il modello riceve informazioni sensibili relative a clienti, polizze e sinistri, e l'invio di tali dati a servizi cloud esterni comporterebbe rischi in termini di riservatezza. Qwen3-coder è stato selezionato in quanto rappresenta una delle migliori soluzioni tra i modelli eseguibili localmente, offrendo un ottimo rapporto tra prestazioni sul benchmark SWE-bench Verified e numero di parametri del modello, come illustrato nella Figura 7.3.

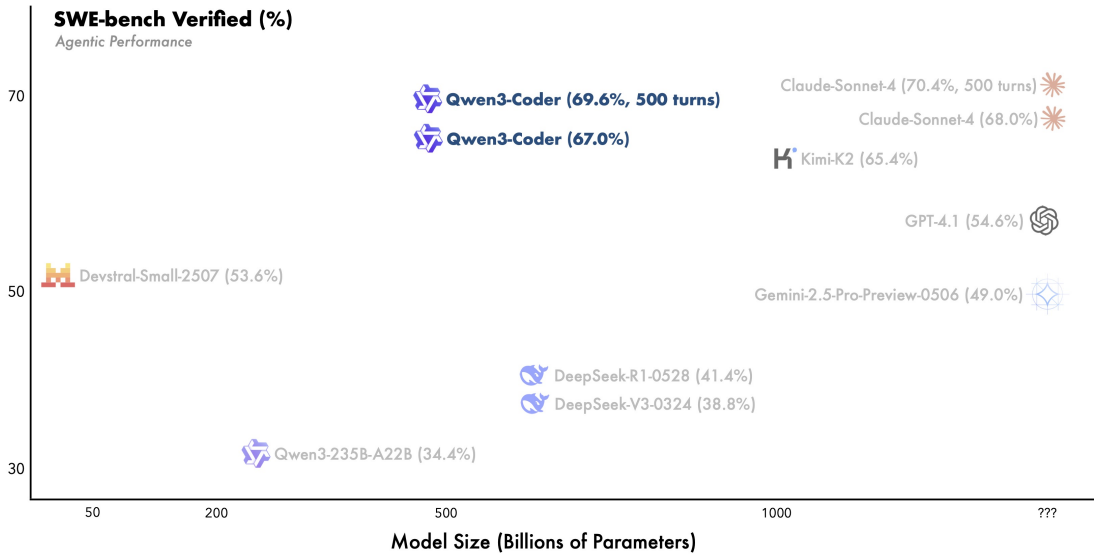


Figura 7.3: Confronto tra le prestazioni e il numero di parametri di alcuni modelli sul benchmark SWE-bench Verified

CAPITOLO 8

Conclusioni e sviluppi futuri

Il presente lavoro di tesi ha avuto come obiettivo la progettazione, l'implementazione e la valutazione di Bluey, un sistema multi-agente basato su Large Language Model destinato al supporto degli operatori di back-office nel settore assicurativo. Il progetto è stato sviluppato durante il periodo di lavoro presso l'azienda Blue Reply, con l'intento di dimostrare le potenzialità di un'architettura multi-agente in un contesto aziendale reale.

Dopo aver analizzato lo stato dell'arte dei chatbot e degli LLM, abbiamo definito i requisiti funzionali e tecnici del sistema, individuando le funzionalità necessarie per soddisfare le esigenze operative degli operatori assicurativi. La scelta progettuale è ricaduta su un'architettura basata sul pattern Supervisor, nella quale un agente centrale coordina le attività di agenti specializzati, ciascuno dotato di prompt e strumenti specifici per il proprio ambito di competenza. Gli agenti individuati sono stati quattro, dedicati rispettivamente alla gestione della posta elettronica, alla pianificazione degli appuntamenti, all'interrogazione del database aziendale e al recupero di informazioni dalla documentazione interna. Il framework LangGraph ha fornito le primitive necessarie per la gestione del flusso di esecuzione, la persistenza dello stato e l'implementazione del meccanismo Human-in-the-Loop. L'implementazione ha tradotto questa progettazione in un sistema funzionante, integrando i servizi esterni di Google (Gmail e Calendar), un database SQL e un sistema RAG basato su FAISS.

Per la valutazione delle prestazioni abbiamo adottato il paradigma LLM-as-a-judge, definendo cinque metriche specifiche che hanno ottenuto punteggi medi, su una scala 1-5, di 4.75 con modelli economici (gpt-5-nano/mini) e 4.87 con modelli più potenti (gpt-5). Ciò dimostra che l'architettura multi-agente consente di ottenere risultati soddisfacenti anche con modelli economici e rapidi: il miglioramento offerto dai modelli più potenti, pari a 0.12 punti, non giustifica il notevole incremento dei costi (diciotto volte superiori) e della latenza (quasi tre volte maggiore). La specializzazione degli agenti riduce la complessità richiesta a ciascun modello, permettendo l'utilizzo di soluzioni più leggere senza compromettere la qualità complessiva del sistema. L'impiego di modelli economici e veloci consente, inoltre, di implementare politiche di gestione degli errori automatiche: in caso di fallimento di un agente, il Supervisor può comprendere l'accaduto e richiedere automaticamente un nuovo tentativo con una formulazione differente della richiesta, ottenendo un risultato soddisfacente senza che l'operatore percepisca l'errore intermedio.

L'architettura multi-agente introduce una complessità aggiuntiva da gestire rispetto a soluzioni monolitiche, che si traduce, tuttavia, in vantaggi concreti: modularità, tolleranza ai fallimenti parziali, possibilità di utilizzare modelli specializzati per ogni compito e differen-

ziati in base alla sensibilità dei dati. La latenza introdotta dal sistema rimane accettabile nel contesto operativo di riferimento, poiché le attività automatizzate richiederebbero comunque tempo significativo se eseguite manualmente dall'operatore.

Il sistema implementato, nella sua attuale configurazione, rappresenta una Proof of Concept che lascia spazio a miglioramenti e ottimizzazioni. Tali interventi potranno essere definiti con maggiore precisione a seguito delle personalizzazioni su clienti reali e di un monitoraggio sull'utilizzo dell'applicazione in ambiente di produzione.

Un primo ambito di sviluppo riguarda l'ottimizzazione dei modelli linguistici. L'applicazione di tecniche di fine-tuning su casi specifici potrebbe migliorare ulteriormente le prestazioni degli agenti. Parallelamente, un processo iterativo di raffinamento dei prompt, basato sull'analisi dei casi di fallimento registrati in produzione, consentirebbe di affinare il comportamento del sistema nei casi limite.

Sul piano dell'interfaccia utente, si prevede la personalizzazione dell'applicazione open-source `Agent Chat` per adattarla all'identità visiva aziendale e alle esigenze specifiche degli operatori. L'integrazione di un meccanismo di autenticazione aziendale permetterebbe di identificare gli utenti e di personalizzare le risposte in base al loro ruolo e alle loro competenze.

Un ulteriore elemento per il miglioramento continuo del sistema è l'implementazione di un meccanismo di feedback esplicito. L'inserimento di pulsanti per la valutazione delle risposte direttamente nell'interfaccia di chat permetterebbe di raccogliere giudizi puntuali dagli operatori, integrando le valutazioni automatiche con dati provenienti dall'utilizzo quotidiano. Queste informazioni potrebbero alimentare un processo di analisi sistematica delle aree critiche e guidare gli interventi di ottimizzazione.

Sul fronte del monitoraggio, l'integrazione di dashboard analitiche consentirebbe di visualizzare in tempo reale le metriche di utilizzo, identificare pattern ricorrenti nelle richieste degli operatori e rilevare tempestivamente eventuali degradazioni delle prestazioni. La piattaforma LangSmith, già utilizzata per la valutazione, offre funzionalità native per il monitoraggio online che potrebbero essere sfruttate in ambiente di produzione.

Infine, l'architettura modulare del sistema facilita l'estensione delle funzionalità attraverso l'aggiunta di nuovi agenti specializzati. La flessibilità dell'architettura Supervisor consente di incorporare nuovi componenti senza modificare radicalmente la struttura esistente, garantendo una crescita organica del sistema in risposta alle esigenze operative emergenti.

Bibliografia

AUFFARTH, B. (2023), *Generative AI with LangChain*, Packt Publishing, Birmingham, UK.

AUFFARTH, B. (2024), *Building LLM Powered Applications*, Packt Publishing, Birmingham, UK.

BROWN, T. B., MANN, B., RYDER, N., SUBBIAH, M., KAPLAN, J., DHARIWAL, P., NEELAKANTAN, A., SHYAM, P., SASTRY, G., ASKELL, A. e OTHERS (2020), «Language Models are Few-Shot Learners», *Advances in Neural Information Processing Systems*, vol. 33, p. 1877–1901.

DEVLIN, J., CHANG, M.-W., LEE, K. e TOUTANOVA, K. (2019), «BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding», *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, p. 4171–4186.

GAUR, V. e DECAPUA, C. (2024), *Multi-Agent Systems with AutoGen: Explore Agentic AI and Build LLM-Powered Applications*, Packt Publishing, Birmingham, UK.

GOODFELLOW, I., BENGIO, Y. e COURVILLE, A. (2016), *Deep Learning*, MIT Press, Cambridge, MA.

HUYEN, C. (2025), *AI Engineering: Building Applications with Foundation Models*, O'Reilly Media, Sebastopol, CA.

LEWIS, P., PEREZ, E., PIKTUS, A., PETRONI, F., KARPUKHIN, V., GOYAL, N. e OTHERS (2020), «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks», *Advances in Neural Information Processing Systems*, vol. 33, p. 9459–9474.

LI, W., ZHAO, M., DONG, W., CAI, J., WEI, Y., POCRESS, M., LI, Y., YUAN, W., WANG, X., HOU, R., LOU, K., ZENG, W., YANG, Y., DU, Y. e WANG, M. (2026), «Grading Scale Impact on LLM-as-a-Judge: Human-LLM Alignment Is Highest on 0-5 Grading Scale», *arXiv preprint arXiv:2601.03444*.

MOORE, D. J. (2025), «A Taxonomy of Hierarchical Multi-Agent Systems: Design Patterns, Coordination Mechanisms, and Industrial Applications», *arXiv preprint arXiv:2508.12683*.

OSHIN, M. e CAMPOS, N. (2025), *Learning LangChain: Building AI and LLM Applications with LangChain and LangGraph*, O'Reilly Media.

OUYANG, L., WU, J., JIANG, X., ALMEIDA, D., WAINWRIGHT, C., MISHKIN, P., ZHANG, C. e OTHERS (2022), «Training Language Models to Follow Instructions with Human Feedback», *Advances in Neural Information Processing Systems*, vol. 35, p. 27 730–27 744.

PHOENIX, J. e TAYLOR, M. (2024), *Prompt Engineering for Generative AI: Future-Proof Inputs for Reliable AI Outputs*, O'Reilly Media.

RUSSELL, S. e NORVIG, P. (2021), *Artificial Intelligence: A Modern Approach*, Pearson, Hoboken, NJ, 4th ed.

TUNSTALL, L., VON WERRA, L. e WOLF, T. (2022), *Natural Language Processing with Transformers*, O'Reilly Media, Sebastopol, CA.

TURING, A. M. (1950), «Computing Machinery and Intelligence», *Mind*, vol. 59 (236), p. 433–460.

VASWANI, A., SHAZEER, N., PARMAR, N., USZKOREIT, J., JONES, L., GOMEZ, A. N., KAISER, L. e POLOSUKHIN, I. (2017), «Attention is All You Need», *Advances in Neural Information Processing Systems*, vol. 30, p. 5998–6008.

WEI, J., WANG, X., SCHUURMANS, D., BOSMA, M., ICHTER, B., XIA, F., CHI, E., LE, Q. e ZHOU, D. (2022), «Chain-of-Thought Prompting Elicits Reasoning in Large Language Models», *Advances in Neural Information Processing Systems*, vol. 35, p. 24 824–24 837.

WEIZENBAUM, J. (1966), «ELIZA – A Computer Program For the Study of Natural Language Communication Between Man And Machine», *Communications of the ACM*, vol. 9 (1), p. 36–45.

WU, Q., BANSAL, G., ZHANG, J., WU, Y., LI, B., ZHU, E., JIANG, L., ZHANG, X. e OTHERS (2023), «AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation», *arXiv preprint arXiv:2308.08155*.

XI, Z., CHEN, W., GUO, X., HE, W., DING, Y., HONG, B., ZHANG, M., WANG, J., JIN, S., ZHOU, E. e OTHERS (2023), «The Rise and Potential of Large Language Model Based Agents: A Survey», *arXiv preprint arXiv:2309.07864*.

YAO, S., ZHAO, J., YU, D., DU, N., SHAFRAN, I., NARASIMHAN, K. e CAO, Y. (2023), «ReAct: Synergizing Reasoning and Acting in Language Models», *International Conference on Learning Representations (ICLR)*.

Sitografia

- Anthropic, risorse di approfondimento sull'intelligenza artificiale – www.anthropic.com/engineering
- arXiv, archivio di paper scientifici – www.arxiv.org
- AWS, documentazione su machine learning – aws.amazon.com/machine-learning
- FAISS, libreria per la ricerca efficiente di similarità tra vettori – github.com/facebookresearch/faiss
- Google Cloud Platform, servizi cloud e API – cloud.google.com
- Google Developers, documentazione API Google – developers.google.com
- Google Trends, strumento per l'analisi delle tendenze di ricerca – trends.google.com
- Hugging Face, piattaforma per modelli di machine learning – www.huggingface.co
- IBM, risorse su intelligenza artificiale e NLP – www.ibm.com/think
- LangChain, framework per applicazioni basate su LLM – www.langchain.com
- LangChain Blog, blog ufficiale con guide e tutorial – www.blog.langchain.com
- LangGraph, orchestrazione di flussi multi-agente – www.langchain.com/langgraph
- LangSmith, piattaforma di monitoraggio per LLM – www.smith.langchain.com
- Ollama, piattaforma per l'esecuzione locale di modelli LLM – www.ollama.ai
- OpenAI, documentazione API e modelli GPT – platform.openai.com
- Qwen, modelli LLM open-weight di Alibaba Cloud – github.com/QwenLM/Qwen
- Wikipedia, enciclopedia libera – www.wikipedia.org

Ringraziamenti

Desidero ringraziare innanzitutto il mio relatore, il Prof. Domenico Ursino, per la pazienza, il supporto e la disponibilità dimostrata durante tutto il lavoro di tesi.

Un ringraziamento speciale va alla mia ragazza Monica, che mi è stata vicina in ogni circostanza, supportandomi nella vita di tutti i giorni e motivandomi durante tutto il percorso accademico. La sua presenza è stata fondamentale per arrivare fino a qui.

Un pensiero va anche a tutti i miei amici e in particolare ai colleghi universitari Eris e Michele, con i quali ho condiviso gli studi e i progetti universitari.

Ringrazio l'azienda Blue Reply per l'opportunità lavorativa che mi è stata offerta e tutti i colleghi con cui ho avuto il piacere di collaborare, in particolare Giuseppe, Fabio e Alessandro.

Infine, ma non meno importante, ringrazio la mia famiglia per avermi sostenuto nel mio percorso universitario.